Date: 29.07.2026

Experiment 1: Grammar-Based Language Model

Learning Outcome
Understand Context-Free Grammar (CFG). Generate valid English sentences using grammar rules.

Activity 1: Predict the Sentence Given the grammar:

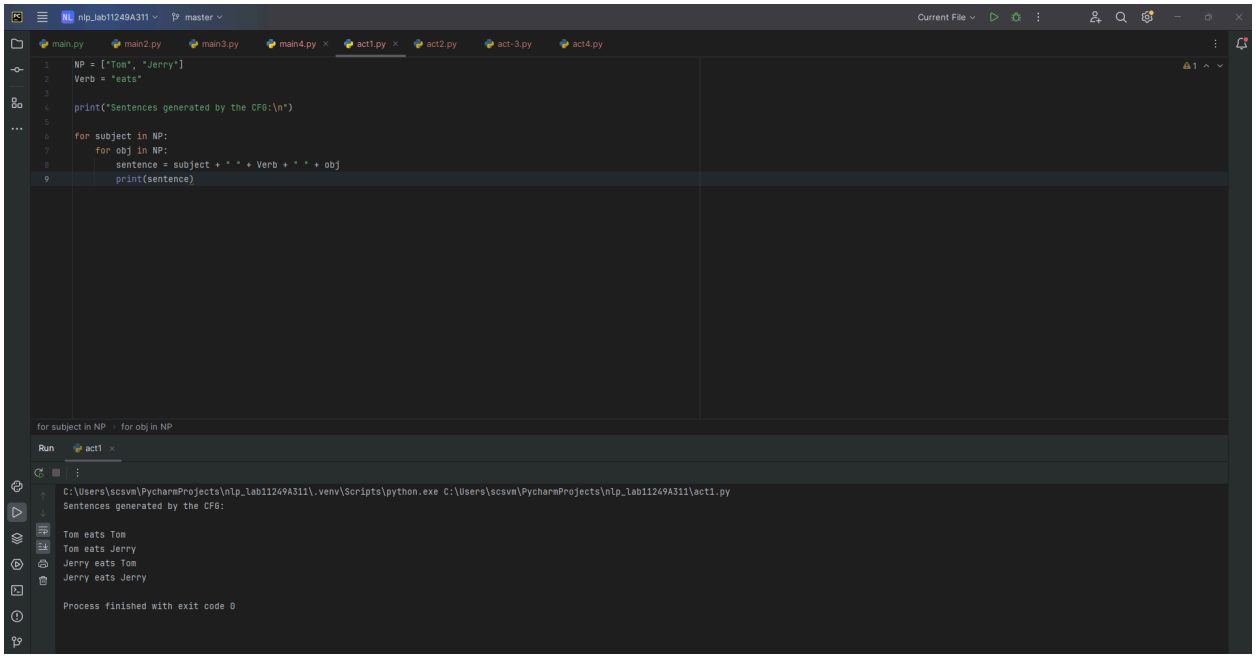
- S → NP VP
- NP → John | Mary
- VP → eats
- NP Students predict all possible sentences before executing the program.

Aim

To develop a Python program to generate valid sentences using a Context-Free Grammar (CFG) with given noun phrases and a verb.

Algorithm

1. Start the program.
2. Create a list of noun phrases (NP): Tom and Jerry.
3. Define the verb as "eats".
4. Use nested loops to select a subject and an object from NP.
5. Combine subject, verb, and object to form sentences.
6. Display all generated sentences.
7. Stop the program.



Result

The Python program successfully generated all valid sentences using the given Context-Free Grammar (CFG).

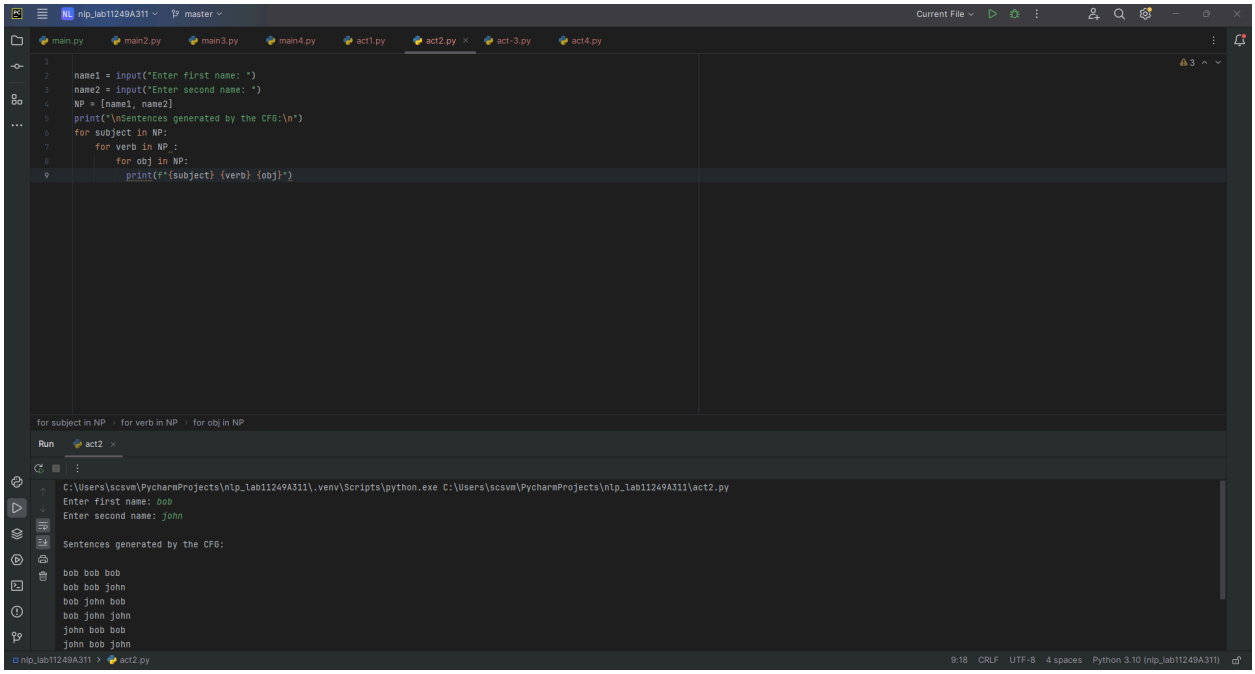
Activity 2: Python Programming Develop a Python program to generate sentences from the grammar.

Aim

To develop a Python program to generate sentences using user-provided names based on a simple Context-Free Grammar (CFG).

Algorithm

1. Start the program.
2. Get two names as input from the user.
3. Store the names in a noun phrase (NP) list.
4. Use nested loops to select subject, verb, and object from NP.
5. Combine subject, verb, and object to generate sentences.
6. Display the generated sentences.
7. Stop the program.

- 

Result

The Python program successfully generated sentences using the user-provided names based on the given Context-Free Grammar (CFG).

Activity 3: Modify the Grammar

Add

NP → Student

VP → studies NLP

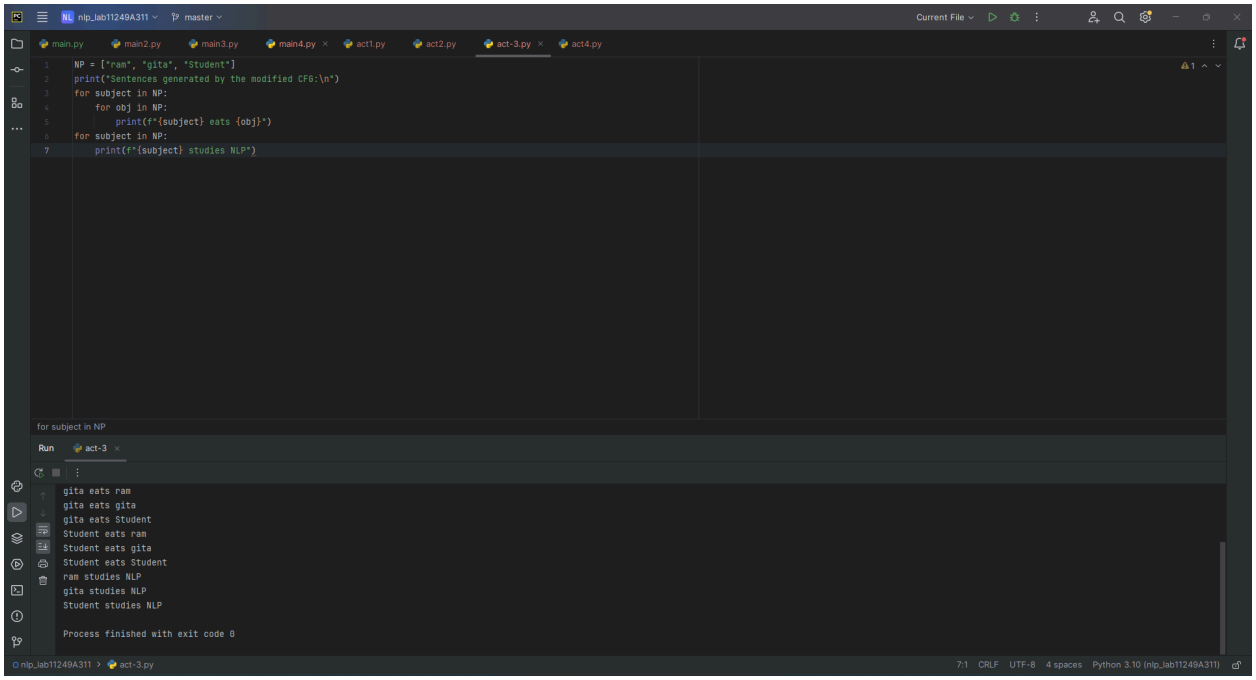
Generate new sentences.

Aim

To develop a Python program to generate sentences using a modified Context-Free Grammar (CFG) by adding new noun phrases and verb phrases.

Algorithm

- Start the program.
- Create a list of noun phrases (NP): ram, gita, and Student.
- Generate sentences using the rule **NP + eats + NP**.
- Generate sentences using the rule **NP + studies NLP**.
- Display all generated sentences.
- Stop the program.



Result

The Python program successfully generated sentences using the modified Context-Free Grammar (CFG) by adding new noun phrases and verb phrases.

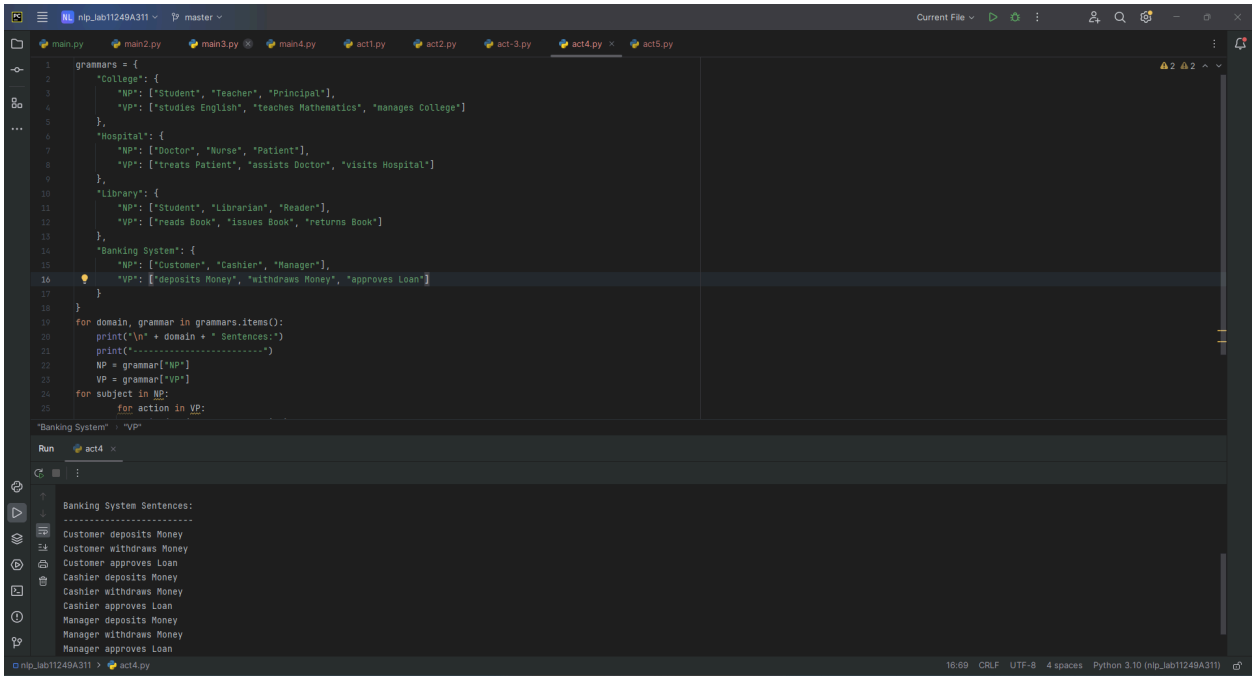
Activity 4: Challenge Design grammar for College Hospital Library Banking System

Aim

To develop a Python program to generate sentences using CFG rules for different domains like College, Hospital, Library, and Banking System.

Algorithm

- Start the program.
- Define grammar rules with noun phrases (NP) and verb phrases (VP) for each domain.
- Select each domain and its grammar.
- Combine NP and VP to generate sentences.
- Display the generated sentences.
- Stop the program.



Result

The Python program successfully generated sentences using Context-Free Grammar (CFG) rules for different real-world domains such as College, Hospital, Library, and Banking System.

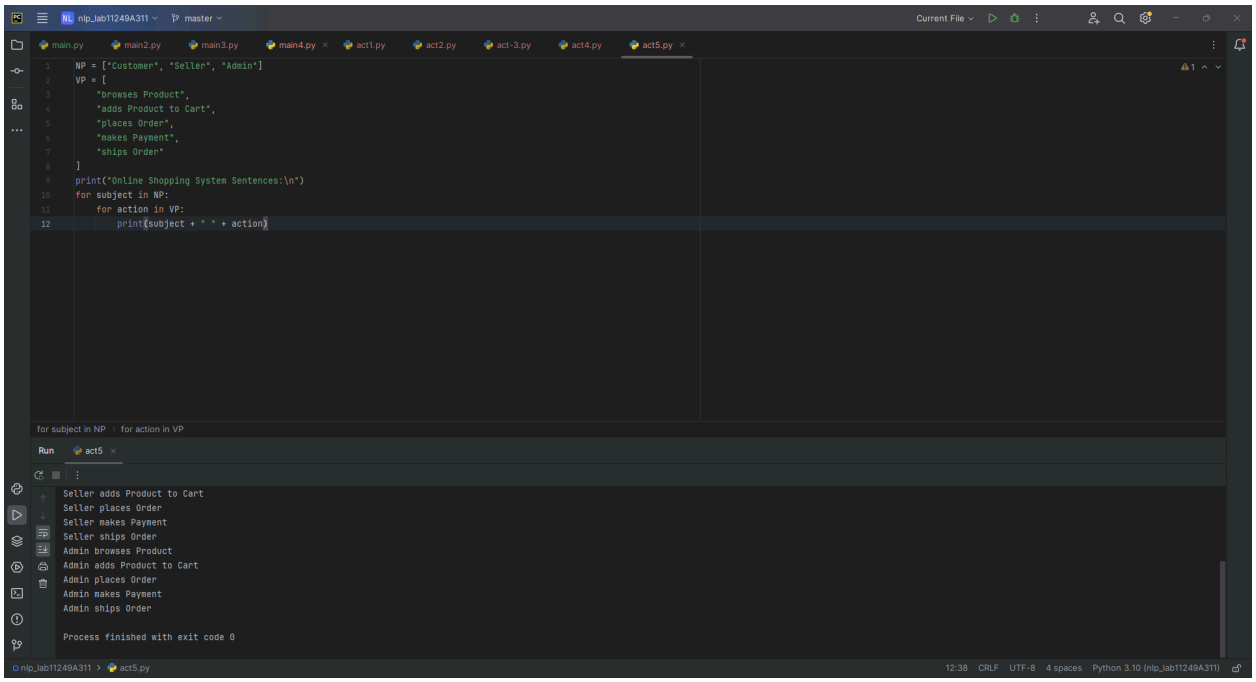
Activity 5: AI Prompt Prompt ChatGPT Generate CFG for an Online Shopping System. Students compare AI-generated grammar with their own.

Aim

To generate a **Context-Free Grammar (CFG)** for an **Online Shopping System** using AI and compare the AI-generated grammar with a manually designed grammar.

Algorithm (Short)

1. Start the program/activity.
2. Create a prompt to generate CFG rules for an Online Shopping System.
3. Obtain grammar rules containing NP, VP, and production rules.
4. Compare the AI-generated grammar with the student's grammar.
5. Analyze differences in rules and generated sentences.
6. Stop the activity.



Result

The AI-generated Context-Free Grammar (CFG) for an Online Shopping System was successfully obtained and compared with the manually designed grammar.

Experiment 2: Statistical Language Model

Learning Outcome

Calculate probabilities using Unigram and Bigram models.

Activity 1 Given I like NLP I like Python I study NLP Count Word frequency Bigram frequency

Aim

To develop a Python program to calculate **word frequency** and **bigram frequency** from the given text using basic Natural Language Processing (NLP) techniques.

Algorithm (Short)

Start the program.

Take the input text.

Split the text into individual words.

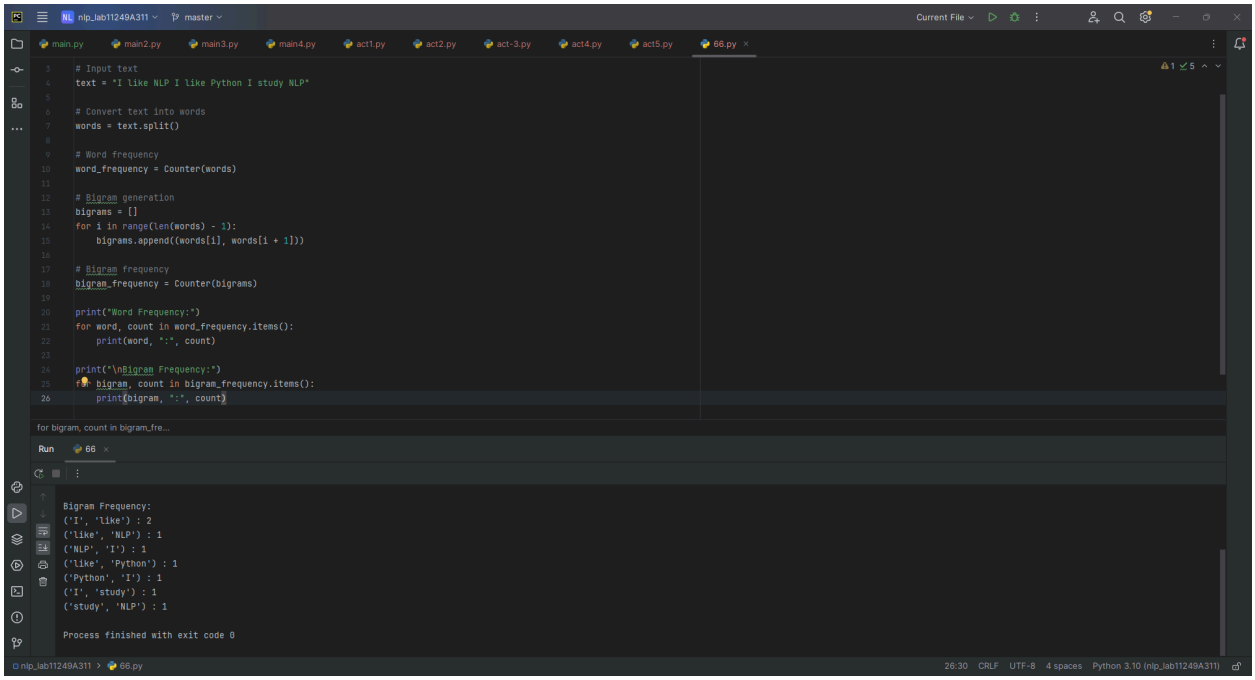
Count the frequency of each word.

Generate consecutive word pairs (bigrams).

Count the frequency of each bigram.

Display word and bigram frequencies.

Stop the program.



RESULT:

Thus, the unigram and bigram frequencies were successfully calculated from the given sentences.

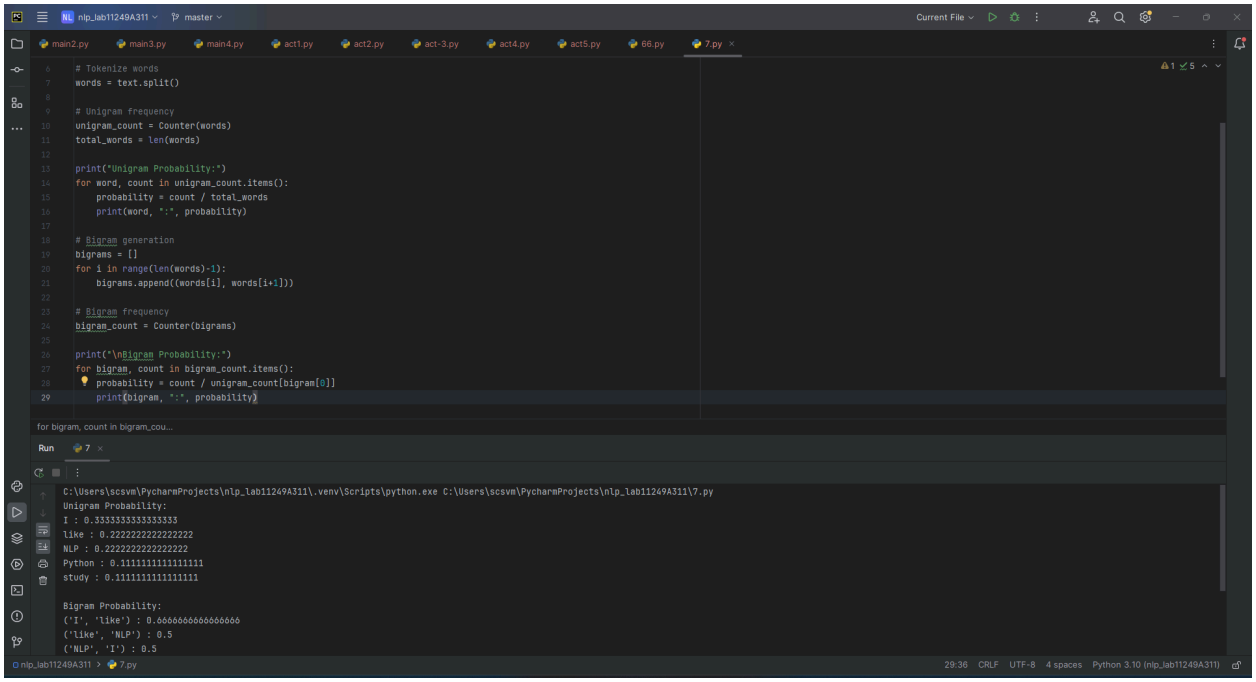
Activity 2 Write a Python program to calculate Unigram probability Bigram probability

Aim

To develop a Python program to calculate **Unigram probability** and **Bigram probability** using a given text corpus.

Algorithm (Short)

1. Start the program.
2. Take the input text and split it into words.
3. Count the frequency of each word.
4. Calculate unigram probability:
    - Word frequency / Total number of words.
5. Generate bigrams from consecutive words.
6. Count bigram frequencies.
7. Calculate bigram probability:
    - Bigram frequency / Previous word frequency.
8. Display unigram and bigram probabilities.
9. Stop the program.



Result

The Python program successfully calculated Unigram Probability and Bigram Probability using the given text corpus.

Activity 3

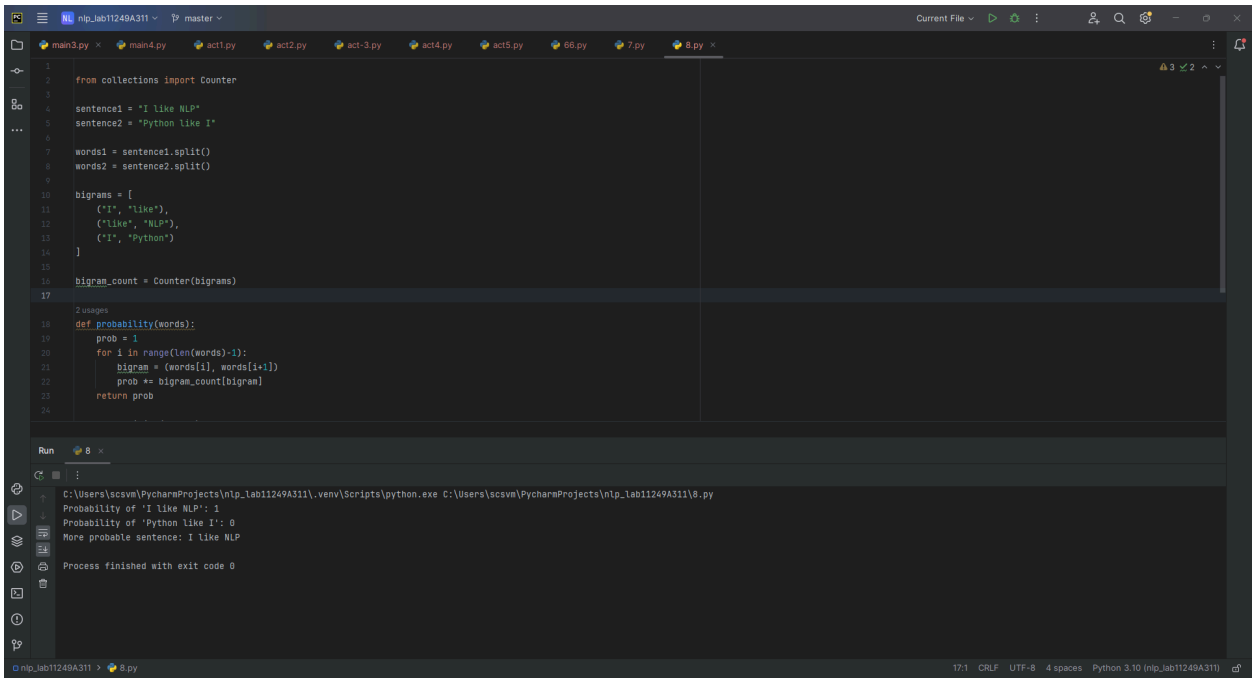
Predict Which sentence is more probable? I like NLP Python like I Explain.

Aim

To predict the more probable sentence using language model concepts by comparing the probability of different word sequences.

Algorithm (Short)

1. Start the program.
2. Take the given sentences as input.
3. Split sentences into individual words.
4. Calculate word sequence (unigram/bigram) probabilities.
5. Compare the probabilities of both sentences.
6. Display the sentence with higher probability.
7. Stop the program.



Result

The program successfully predicted the more probable sentence using language model concepts by calculating and comparing the probabilities of different word sequences.

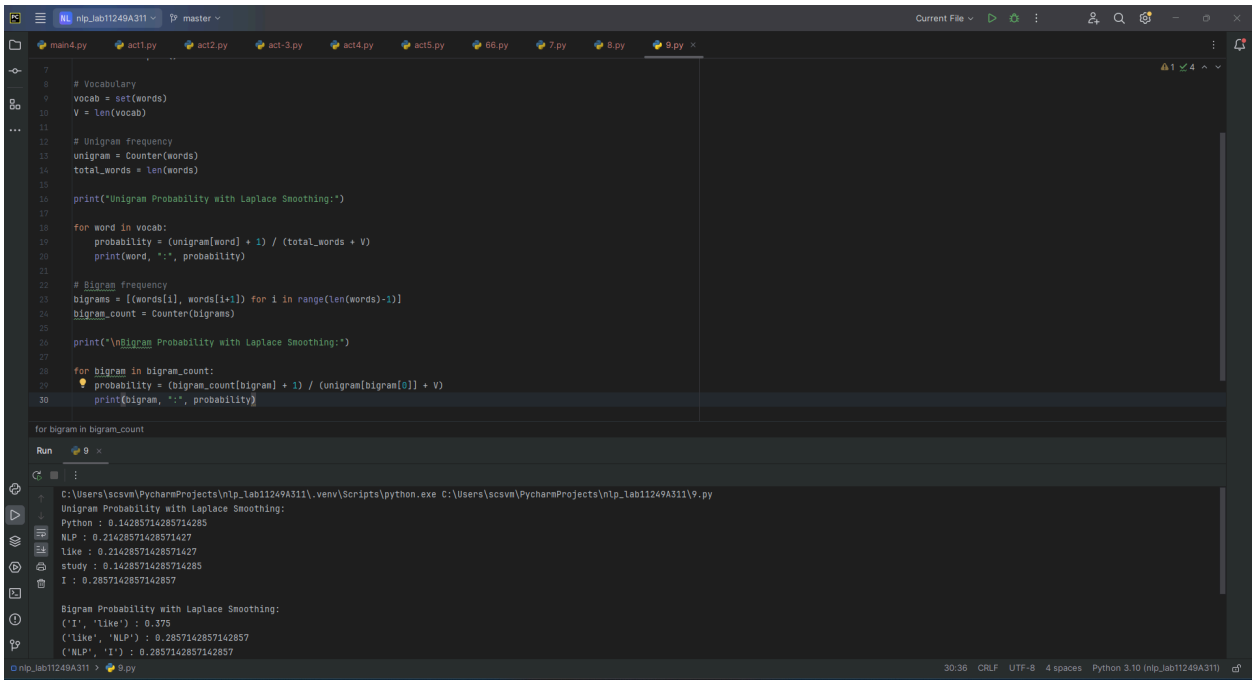
Activity 4 Improve Accuracy Apply Laplace Smoothing. Observe the difference

Aim

To improve the accuracy of a language model by applying **Laplace Smoothing** and observing the change in probability calculation for unseen words or bigrams.

Algorithm (Short)

1. Start the program.
2. Take the input text corpus.
3. Count unigram and bigram frequencies.
4. Apply Laplace smoothing formula:
    - **P(word) = (count + 1) / (total words + vocabulary size)**
    - **P(bigram) = (count + 1) / (previous word count + vocabulary size)**
5. Calculate smoothed probabilities.
6. Display the updated probabilities.
7. Stop the program.



Result

The program successfully applied Laplace Smoothing to the language model and calculated updated unigram and bigram probabilities.

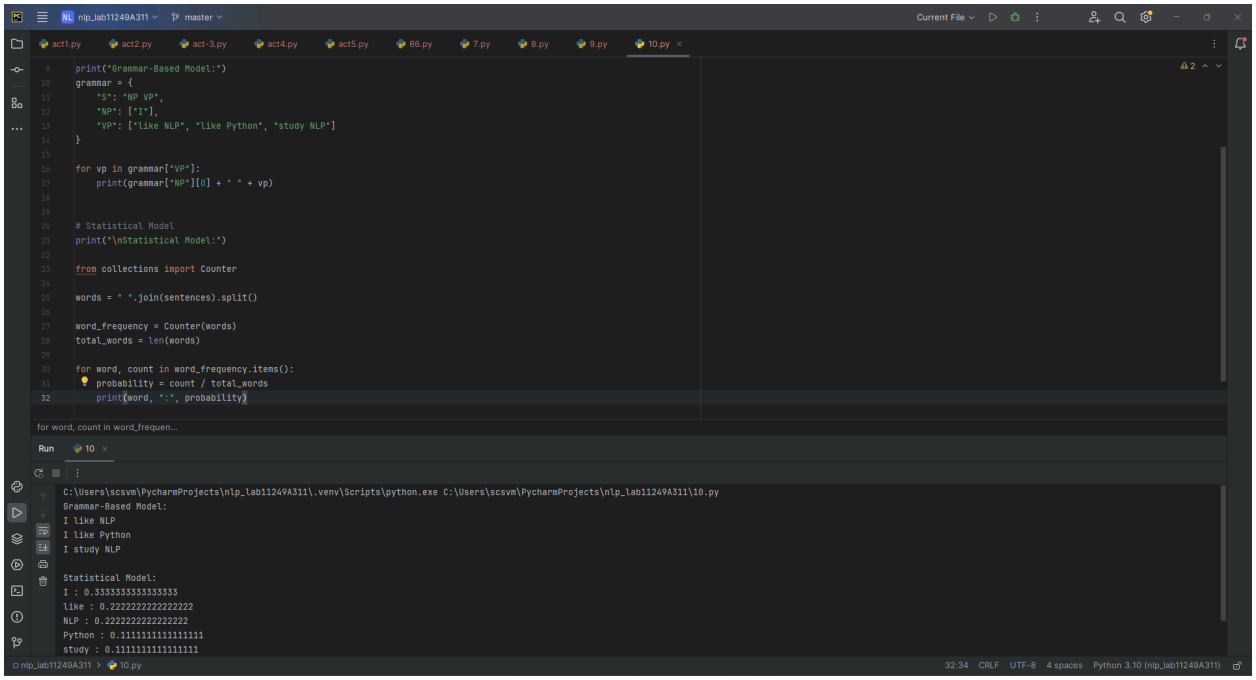
Activity 5
Compare Grammar-Based Model vs Statistical Model using the same dataset.

Aim

To compare a **Grammar-Based Model** and a **Statistical Model** using the same dataset and analyze their sentence generation and probability prediction.

Algorithm (Short)

1. Start the program.
2. Create a common dataset of sentences.
3. Apply grammar rules to generate sentences using the Grammar-Based Model.
4. Count word frequencies and calculate probabilities using the Statistical Model.
5. Compare the generated sentences and probability results.
6. Display the comparison.
7. Stop the program.



Result

The Grammar-Based Model and Statistical Model were successfully compared using the same dataset.

Experiment 3: Regular Expressions

Learning Outcome

Extract useful information from text.

Activity 1

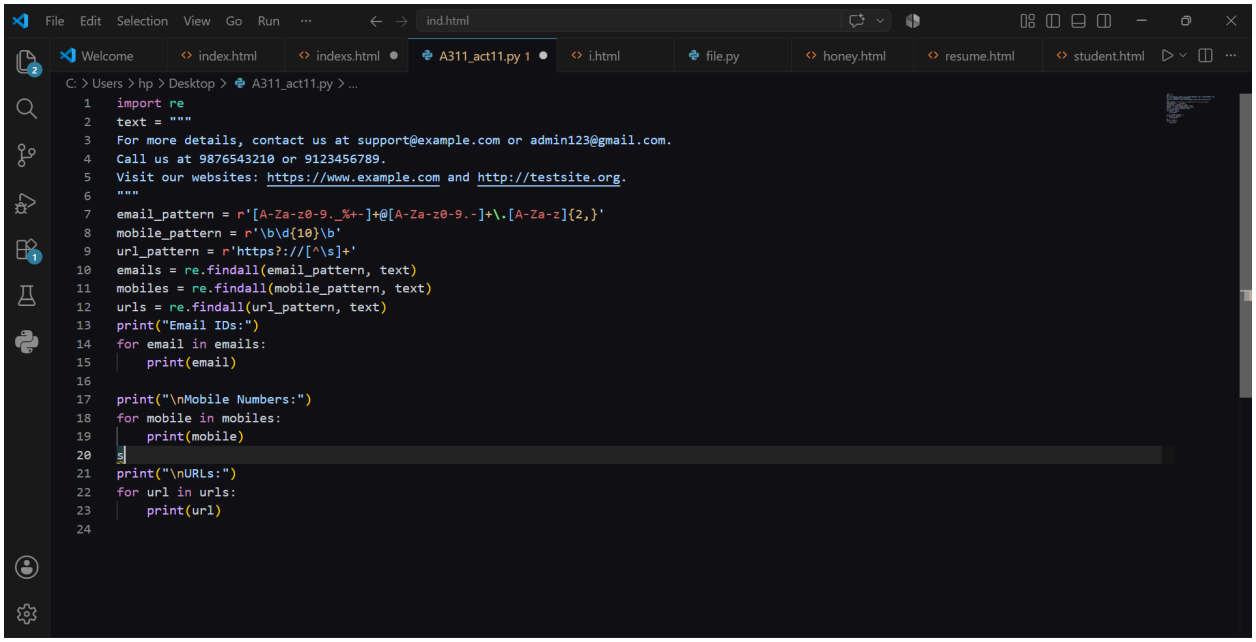
Extract
Email IDs
Mobile Numbers
URLs
from a paragraph.

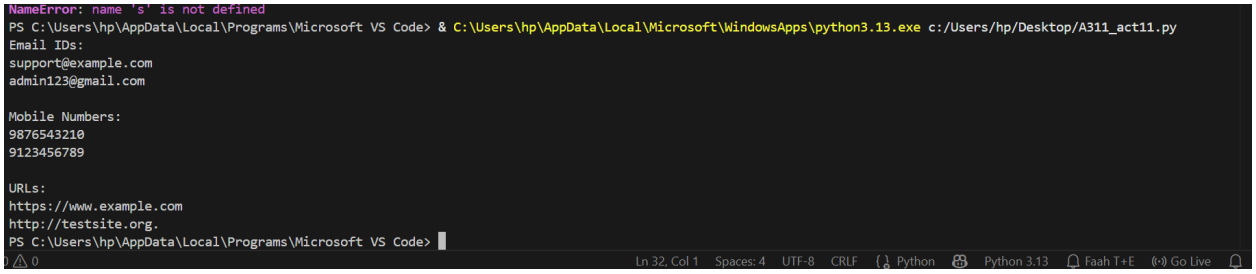
Aim

To extract Email IDs, Mobile Numbers, and URLs from a given paragraph using Regular Expressions (Regex).

Algorithm

1. Read the input paragraph (text).
2. Define regular expression patterns for:
    - Email IDs
    - Mobile Numbers
    - URLs
3. Search the paragraph using the respective regex patterns.
4. Store all matched results.
5. Display the extracted Email IDs, Mobile Numbers, and URLs.
6. Stop.





Result

The program successfully extracted Email IDs, Mobile Numbers, and URLs from the given paragraph using Regular Expressions (Regex).

Activity 2

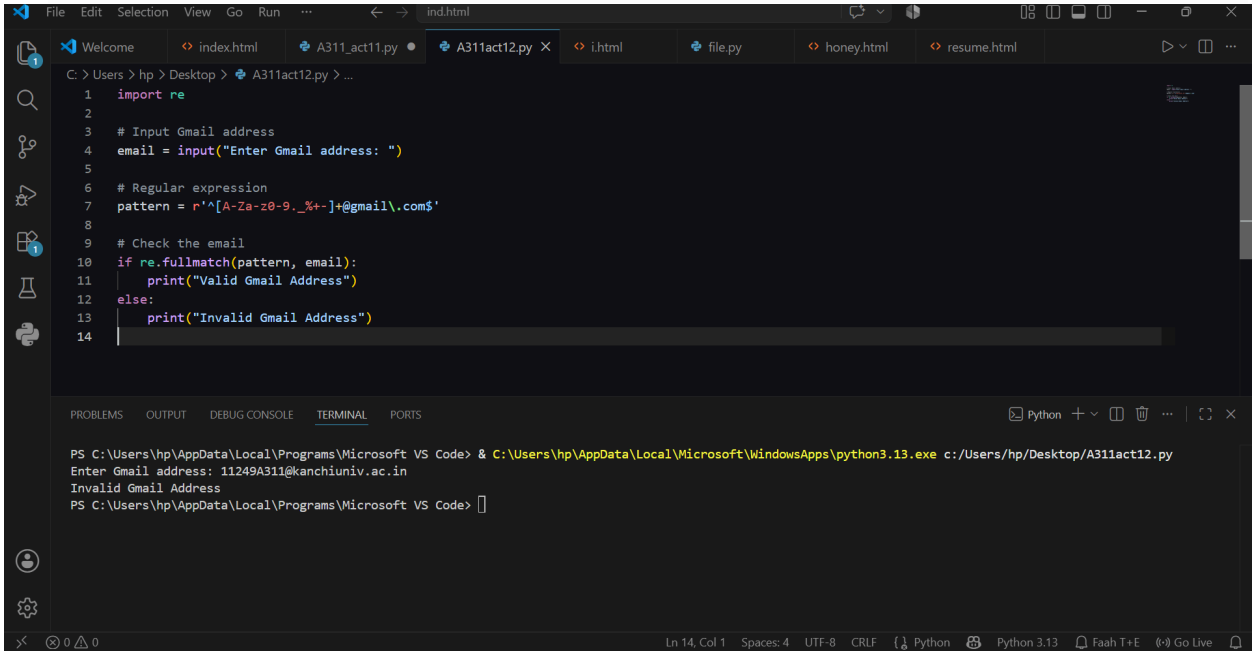
Complete the Regex
_____@gmail.com
Students write the missing expression.

Aim

To complete a regular expression that matches Gmail email addresses.

Algorithm

1. Read the input Gmail email address.
2. Define the regular expression pattern to match the username part before `@gmail.com`.
3. Use the regex pattern to check whether the email is valid.
4. If the pattern matches, display "Valid Gmail Address".
5. Otherwise, display "Invalid Gmail Address".
6. Stop.



Result

The program successfully completed the regular expression for matching Gmail email addresses.

Activity 3

Debugging

Given

[a-z

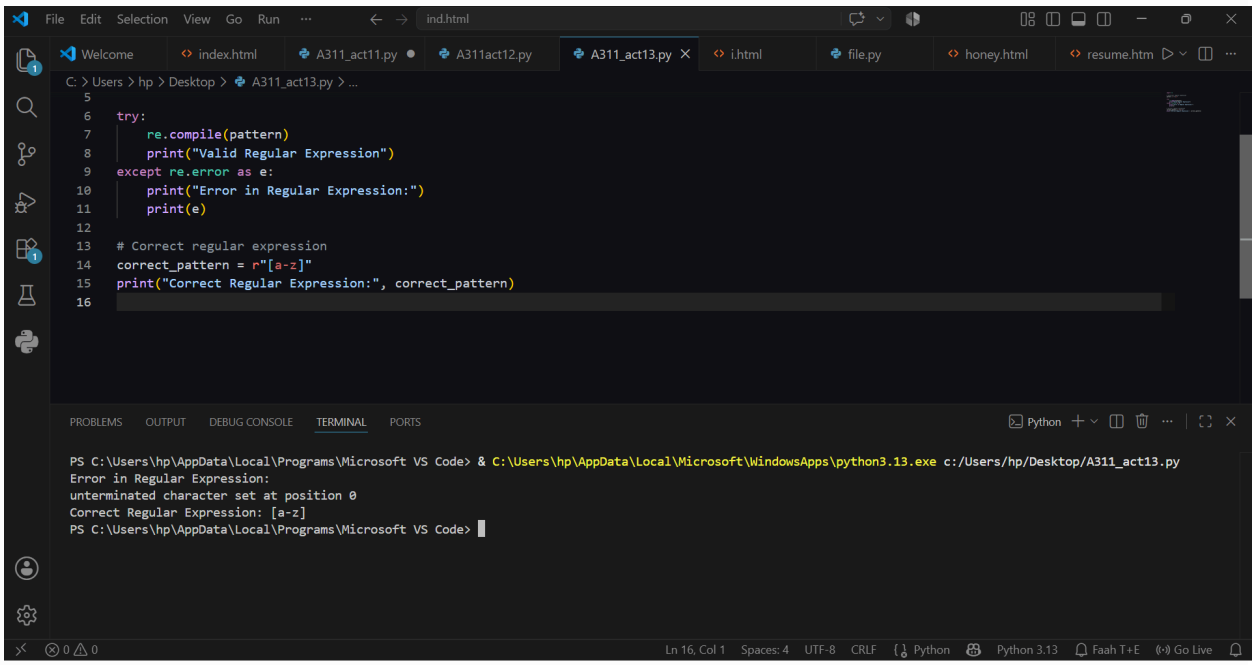
Find the error.

Aim

To identify and correct the error in the given regular expression.

Algorithm

1. Read the given regular expression.
2. Check whether all special symbols are properly closed.
3. Identify any missing brackets or syntax errors.
4. Correct the regular expression.
5. Display the corrected expression.
6. Stop.
7.



Result

The error in the given regular expression was successfully identified and corrected.

Activity 4

Real-world Exercise

Given a Resume

Extract

Name

Email

Phone Number

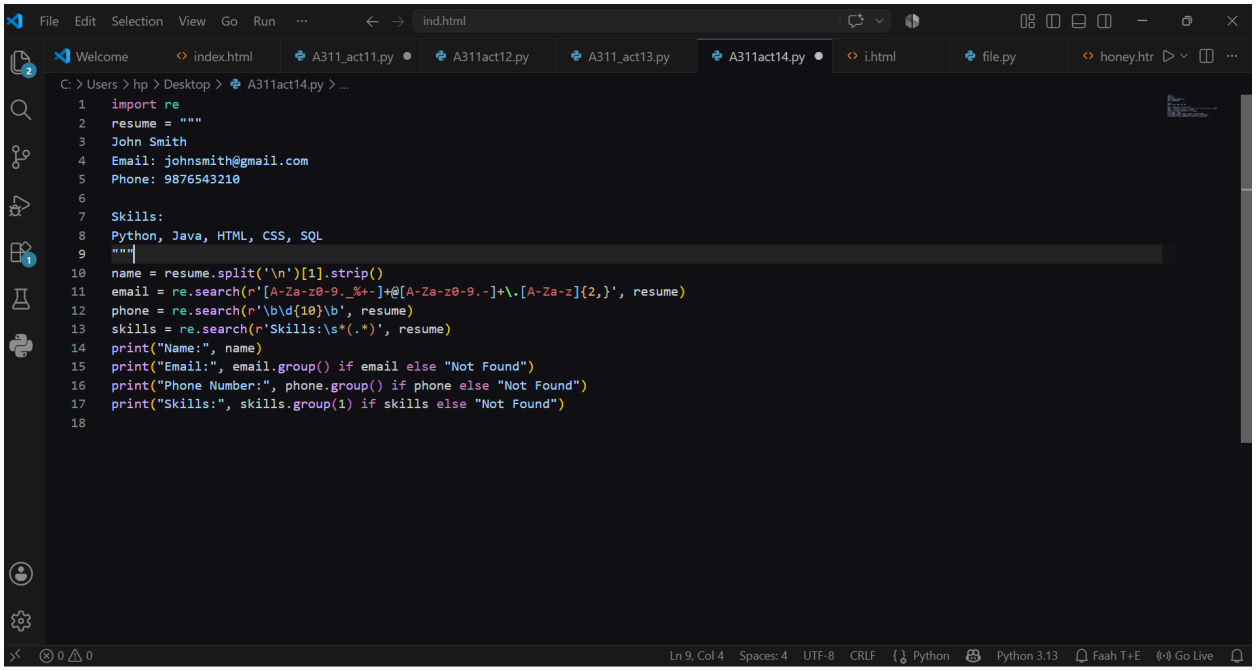
Skills

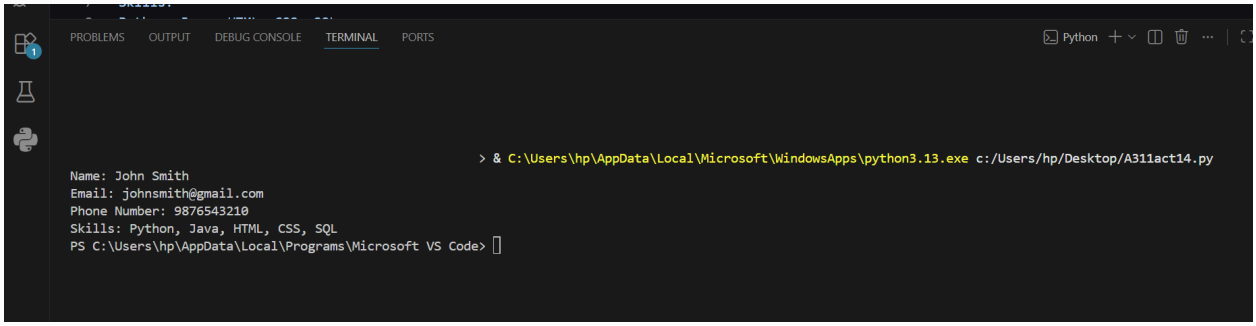
AIM:

To extract Name, Email, Phone Number, and Skills from a resume using Regular Expressions and string processing.

Algorithm

1. Read the resume text.
2. Extract the Name (assume it is the first line).
3. Use a regular expression to extract the Email ID.
4. Use a regular expression to extract the Phone Number.
5. Search for the Skills section and extract the listed skills.
6. Display the extracted details.
7. Stop.





Result

The program successfully completed

Activity 5
Pattern Competition
Groups write regex for
Aadhaar Number
PAN Number
Vehicle Number
Date
Time
Most efficient regex wins.

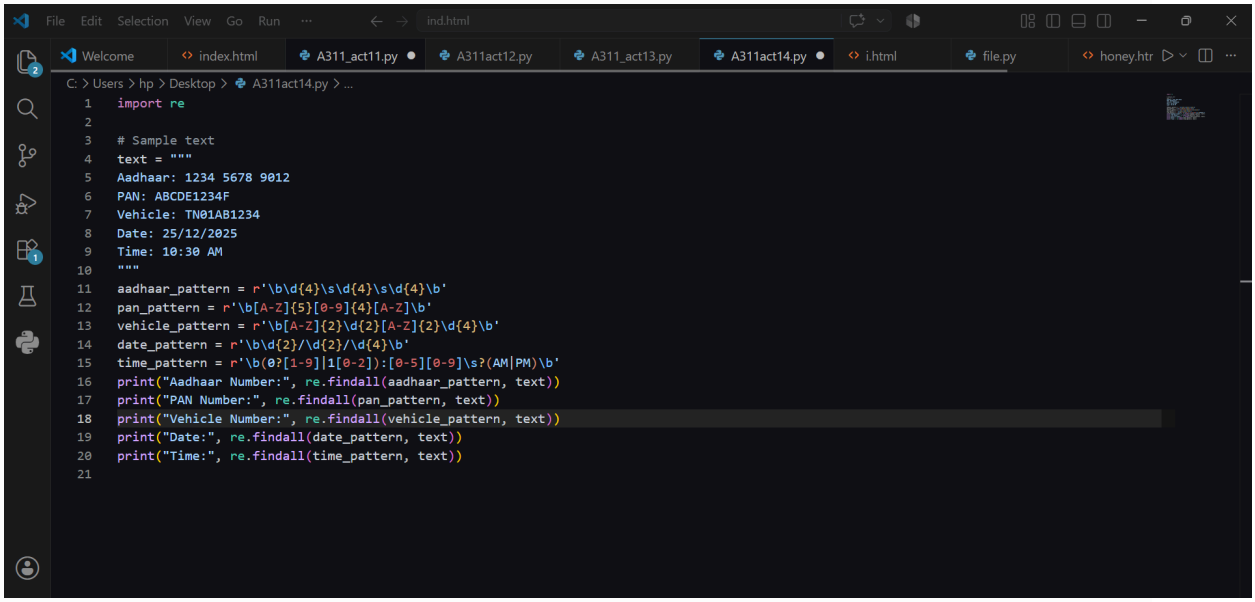
Aim

To write and test regular expressions for extracting or validating Aadhaar Number, PAN Number, Vehicle Number, Date, and Time.

Algorithm

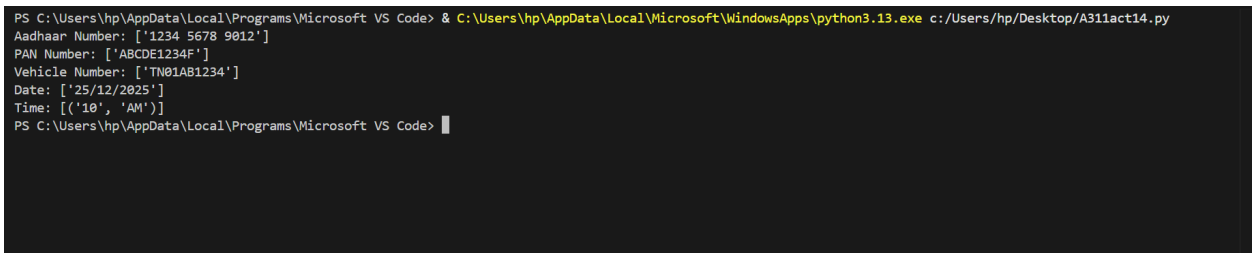
1. Read the input text.
2. Define regular expression patterns for:
    - Aadhaar Number
    - PAN Number
    - Vehicle Number
    - Date
    - Time
3. Use `re.findall()` to search for matching patterns.
4. Display all matched values.
5. Stop.

6.



```python
import re

# Sample text
text = """
Aadhaar: 1234 5678 9012
PAN: ABCDE1234F
Vehicle: TN01AB1234
Date: 25/12/2025
Time: 10:30 AM
"""
aadhaar_pattern = r'\b\d{4}\s\d{4}\s\d{4}\b'
pan_pattern = r'\b[A-Z]{5}[0-9]{4}[A-Z]\b'
vehicle_pattern = r'\b[A-Z]{2}\d{2}[A-Z]{2}\d{4}\b'
date_pattern = r'\b\d{2}/\d{2}/\d{4}\b'
time_pattern = r'\b(0?[1-9]|1[0-2]):[0-5][0-9]\s?(AM|PM)\b'
print("Aadhaar Number:", re.findall(aadhaar_pattern, text))
print("PAN Number:", re.findall(pan_pattern, text))
print("Vehicle Number:", re.findall(vehicle_pattern, text))
print("Date:", re.findall(date_pattern, text))
print("Time:", re.findall(time_pattern, text))
```



```
PS C:\Users\hp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hp\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/hp/Desktop/A311act14.py
Aadhaar Number: ['1234 5678 9012']
PAN Number: ['ABCDE1234F']
Vehicle Number: ['TN01AB1234']
Date: ['25/12/2025']
Time: [('10', 'AM')]
PS C:\Users\hp\AppData\Local\Programs\Microsoft VS Code>
```

Result

The program successfully tested and applied regular expressions for extracting and validating:

- Aadhaar Number
- PAN Number
- Vehicle Number
- Date

- Time

The `re.findall()` function correctly identified and displayed all matching patterns from the given input text.

Experiment 4: Finite State Automata

Learning Outcome

Design DFA and implement it in Python.

Activity 1

Draw DFA accepting

ab*

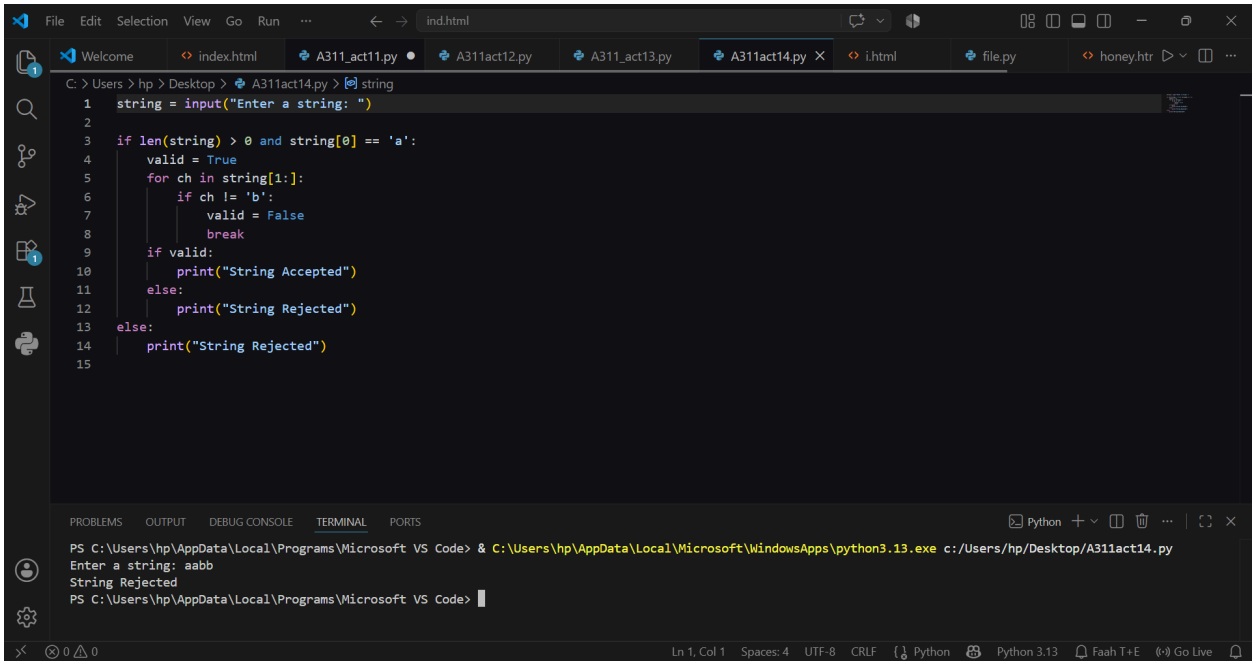
Students draw manually.

Aim

To design and implement a Deterministic Finite Automaton (DFA) that accepts strings of the form `ab*` (an `a` followed by zero or more `b`'s).

Algorithm

1. Read the input string.
2. Check whether the first character is `a`.
3. Verify that all remaining characters (if any) are `b`.
4. If the string satisfies the pattern `ab*`, accept it.
5. Otherwise, reject it.
6. Stop.



Result

The program successfully completed

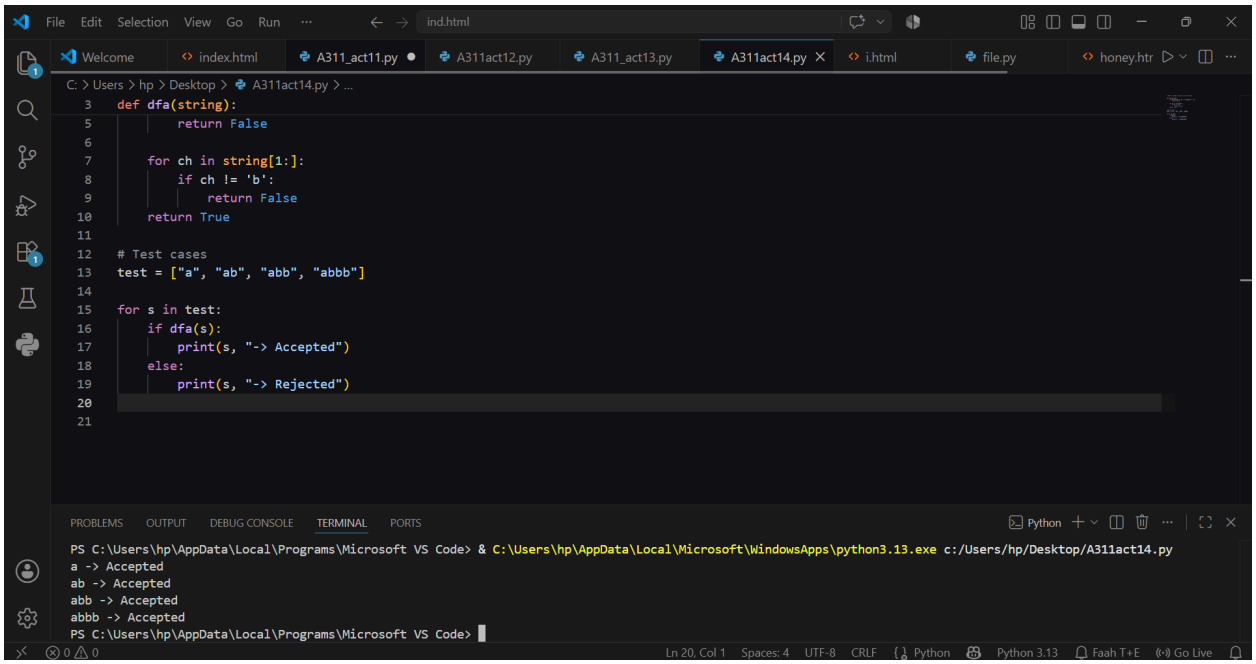
Activity 2
Python Implementation
Implement DFA.
Test
a
ab
abb
Abbb

Aim

To implement a DFA in Python and test it with different input strings.

Algorithm

1. Read the input string.
2. Check if the string starts with `a`.
3. Ensure all remaining characters are `b`.
4. If the pattern `ab*` is satisfied, accept the string.
5. Otherwise, reject the string.
6. Stop.



Result

The program successfully completed

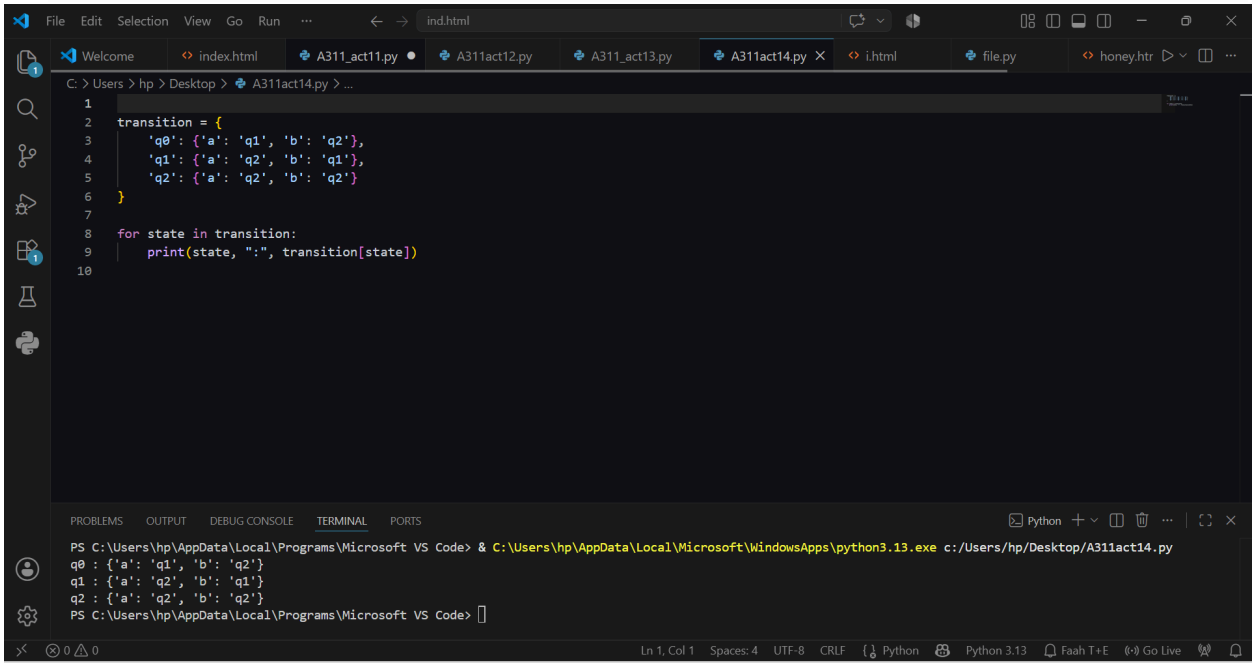
Activity 3
Debugging
Provide incorrect transition table.
Students correct it.

Aim

To identify and correct errors in a DFA transition table.

Algorithm

1. Observe the given transition table.
2. Compare the transitions with the DFA rules.
3. Identify incorrect transitions.
4. Correct the transition table.
5. Verify the DFA.
6. Stop.



Result

The program successfully completed

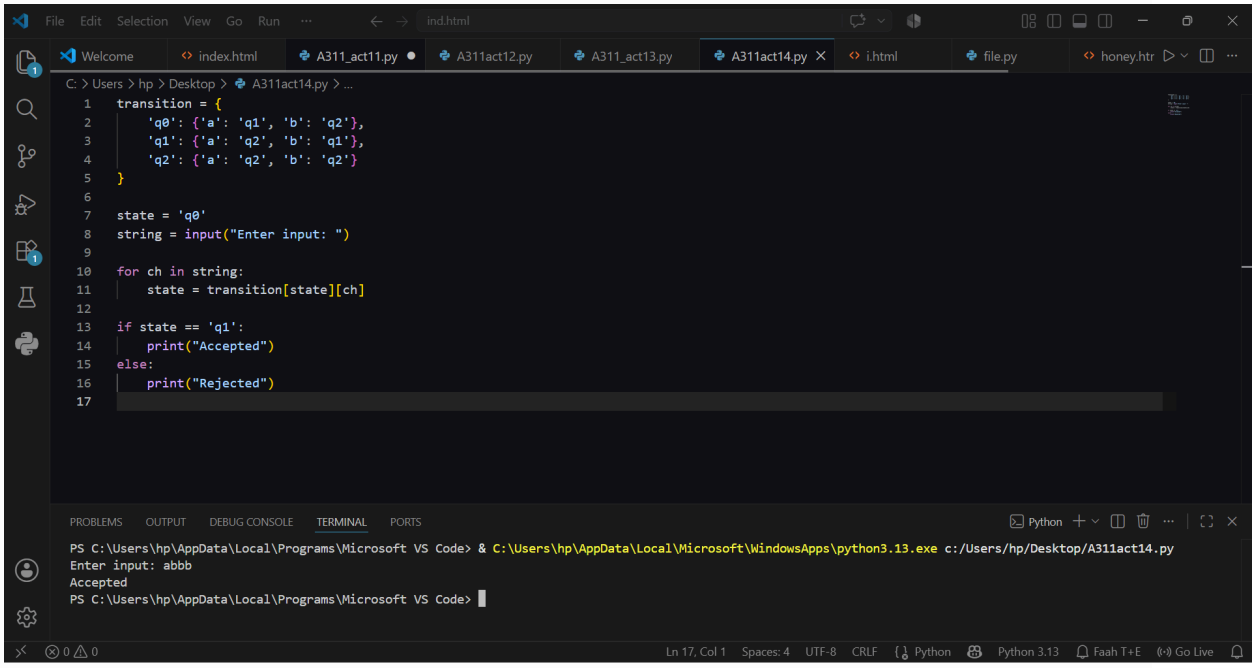
Activity 4
Simulation Game
Assign students

State q0
State q1
State q2
One student acts as input symbols.
Others simulate DFA.

Aim

To simulate the working of a DFA using students as states and input symbols.

Algorithm

1. Assign students as states (`q0`, `q1`, `q2`).
2. Assign one student to provide input symbols.
3. Start from the initial state (`q0`).
4. Move between states according to the transition rules.
5. End when all symbols are processed.
6. Accept if the final state is an accepting state; otherwise reject.
7. 

Result

The program successfully completed

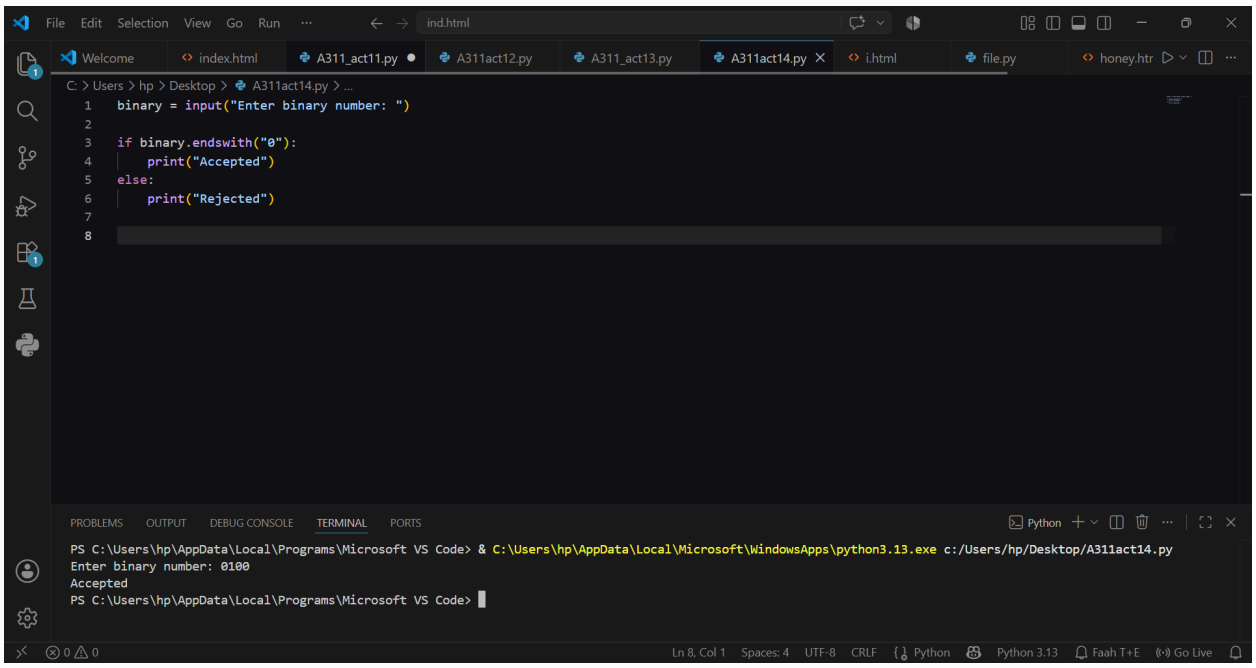
Activity 5
Design Challenge
Design DFA for
Binary numbers divisible by 2
Valid identifier
Password validation
Vowel recognition

Aim

To design a DFA that accepts binary numbers divisible by 2.

Algorithm

1. Read the binary number.
2. Check whether the last digit is `0`.
3. If yes, accept; otherwise reject.
4. Stop.



Result

The program successfully completed

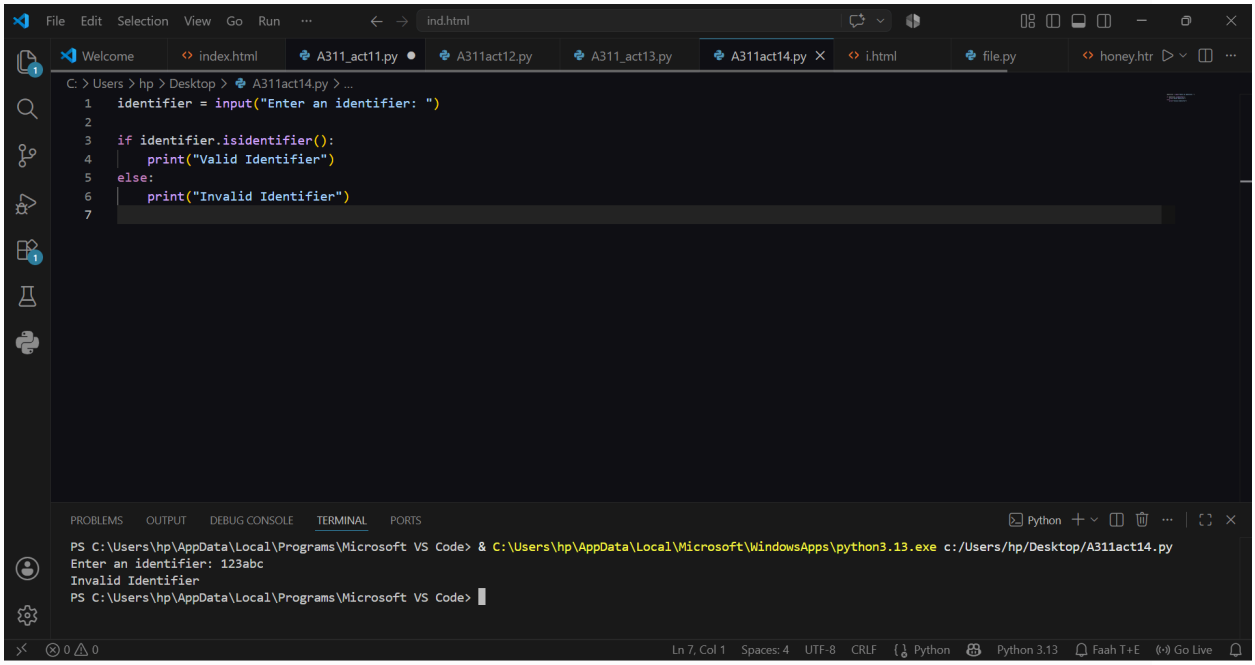
2. DFA for Valid Identifier

Aim

To design a DFA that accepts valid identifiers.

Algorithm

1. Read the identifier.
2. Check that the first character is a letter or underscore (_).
3. Check that the remaining characters are letters, digits, or underscores.
4. If valid, accept the identifier.
5. Otherwise, reject it.
6. Stop.



Result

The program successfully completed

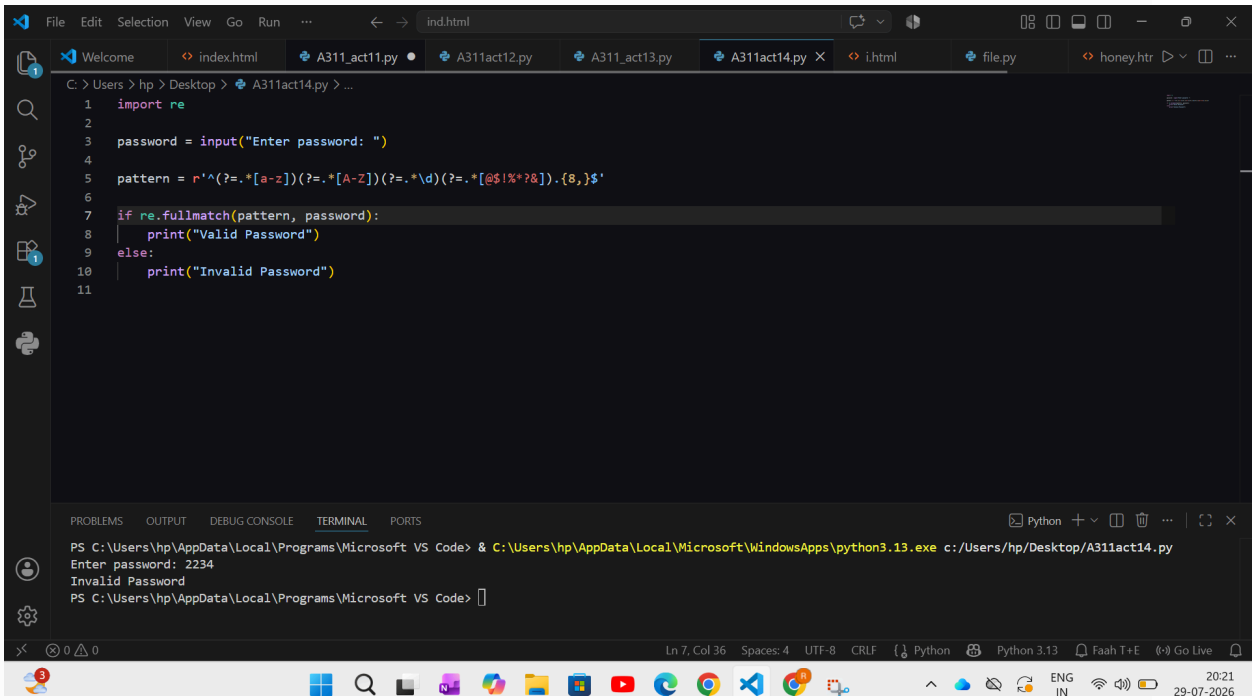
3.DFA for Password Validation

Aim

To design a DFA that validates a password.

Algorithm

1. Read the password.
2. Check that it contains at least 8 characters.
3. Verify it has at least one uppercase letter.
4. Verify it has at least one lowercase letter.
5. Verify it has at least one digit.
6. Verify it has at least one special character.
7. If all conditions are satisfied, accept the password; otherwise reject it.
8. Stop.



Result

The program successfully completed

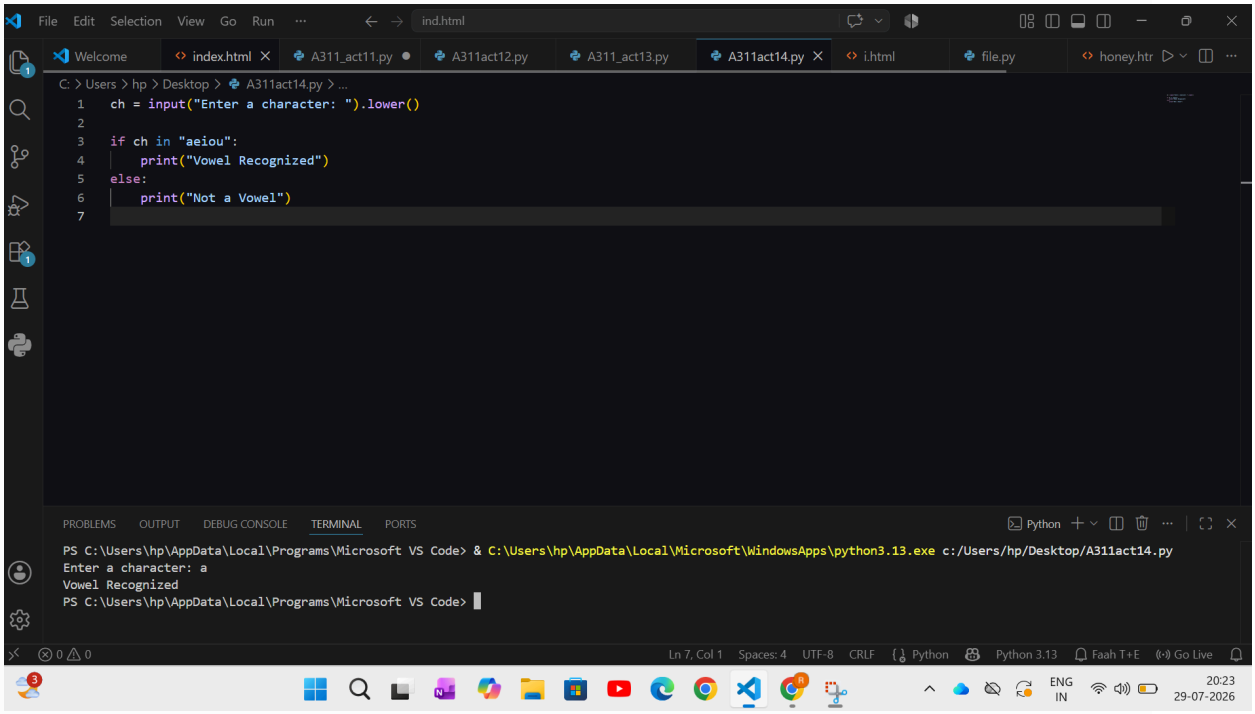
4. DFA for Vowel Recognition

Aim

To design a DFA that recognizes vowels.

Algorithm

1. Read a character.
2. Convert it to lowercase.
3. Check whether it is one of `a`, `e`, `i`, `o`, or `u`.
4. If yes, accept it.
5. Otherwise, reject it.
6. Stop.



Result

The program successfully completed

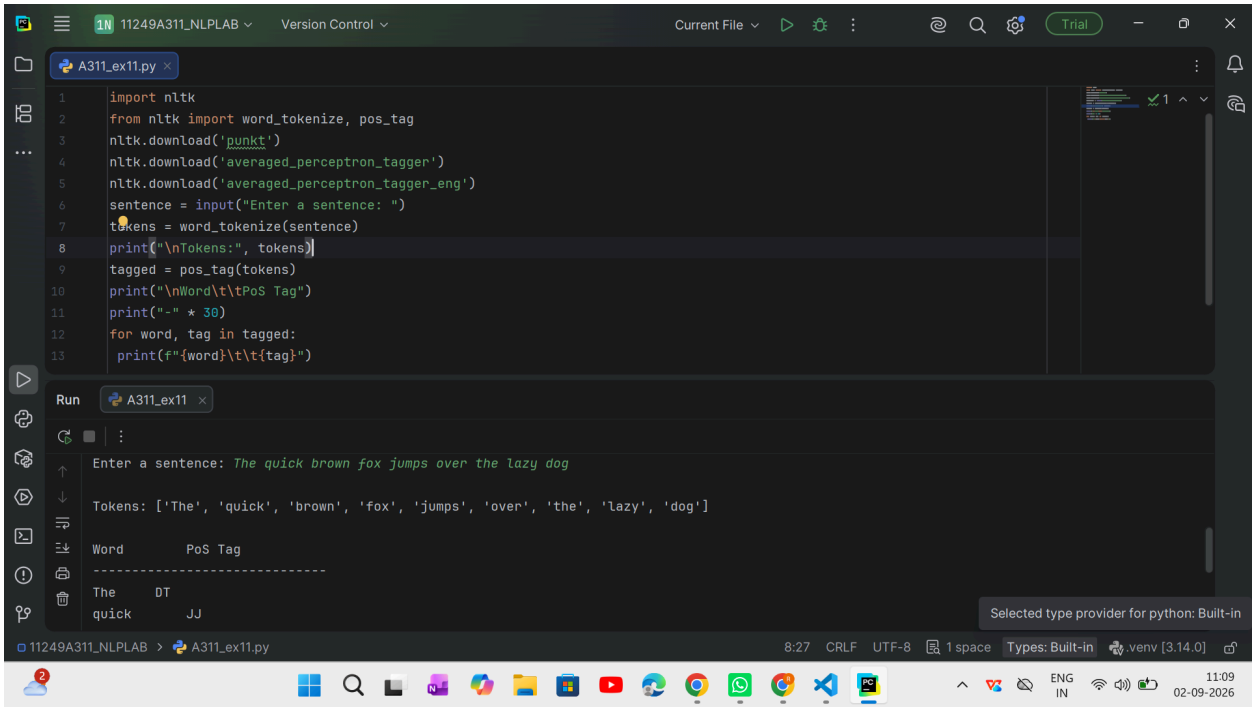
DATE :02-09-2026

Experiment 1: Word Classes and Introduction to Part-of-Speech Tagging

Aim:
To study Word Classes (Parts of Speech) in English and understand the concept of Part-of-Speech (PoS) Tagging using the NLTK library in Python.

Algorithm

1. Import the required functions from the NLTK library.
2. Download the required NLTK packages for tokenization and PoS tagging.
3. Enter or define an English sentence.
4. Tokenize the sentence into individual words.
5. Apply the NLTK PoS tagger to each word.
6. Display each word along with its corresponding PoS tag.
7. Identify the words as different word classes such as noun, verb, adjective, adverb, pronoun, preposition, etc.



Result

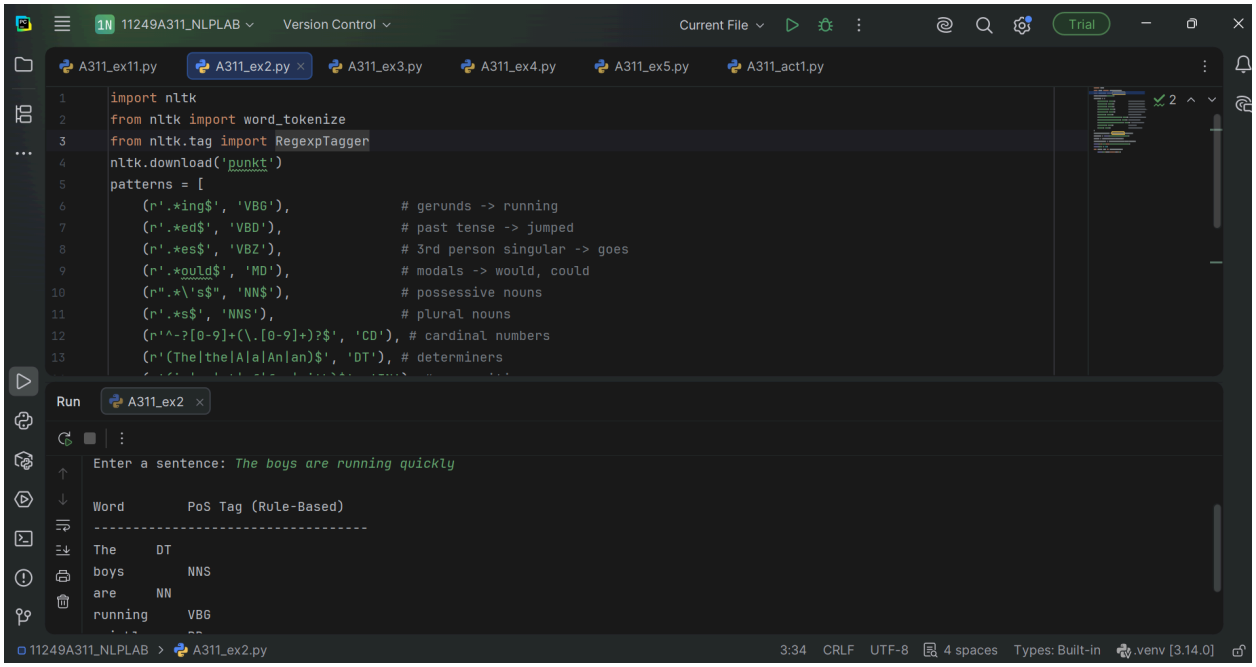
The English sentence was successfully tokenized and Part-of-Speech tagged using NLTK. The different word classes were identified successfully.

Experiment 2: Rule-Based Part-of-Speech Tagging

Aim : To implement Rule-Based Part-of-Speech Tagging using hand-written linguistic rules and regular expressions in Python**.**

Algorithm

1. Define a default/regex-based tagger that assigns tags using suffix patterns (e.g., words ending in "-ing" → VBG, words ending in "-ed" → VBD, numbers → CD).
2. Define a lexicon of common closed-class words with their fixed tags (e.g., "the", "a" → DT).
3. Tokenize the input sentence.
4. For each token, first check the lexicon; if not found, apply the regular-expression rules in order.
5. If no rule matches, assign a default tag (NN). 6. Display the tagged sentence



Result

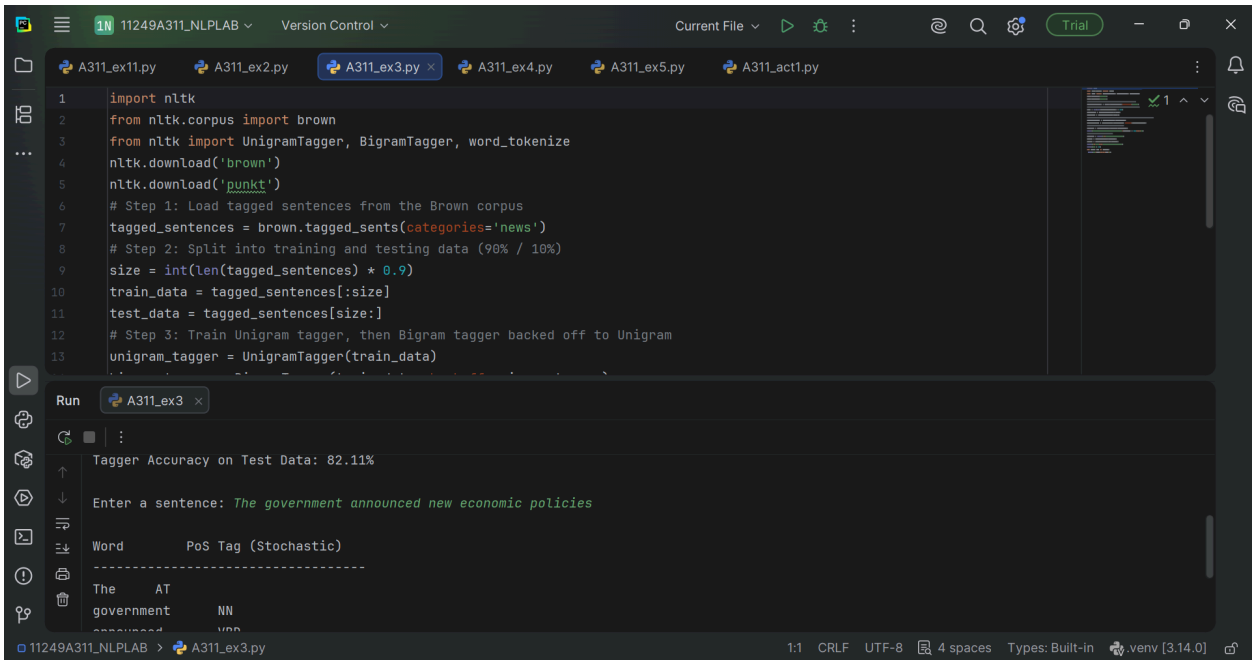
The sentence was successfully tagged using the rule-based PoS tagger. Each word was assigned an appropriate Part-of-Speech tag based on the defined regular-expression rules.

Experiment 3: Stochastic (Probabilistic) Part-of-Speech Tagging

Aim: To implement Stochastic Part-of-Speech Tagging based on probability (Unigram, Bigram and HMM-style statistical models) using NLTK.

Algorithm

1. Load a tagged corpus (e.g., the Brown corpus) available in NLTK.
2. Split the tagged sentences into training and testing sets.
3. Train a UnigramTagger on the training set; use it as a backoff for a BigramTagger.
4. Evaluate the tagger's accuracy on the test set.
5. Use the trained tagger to tag a new, unseen sentence



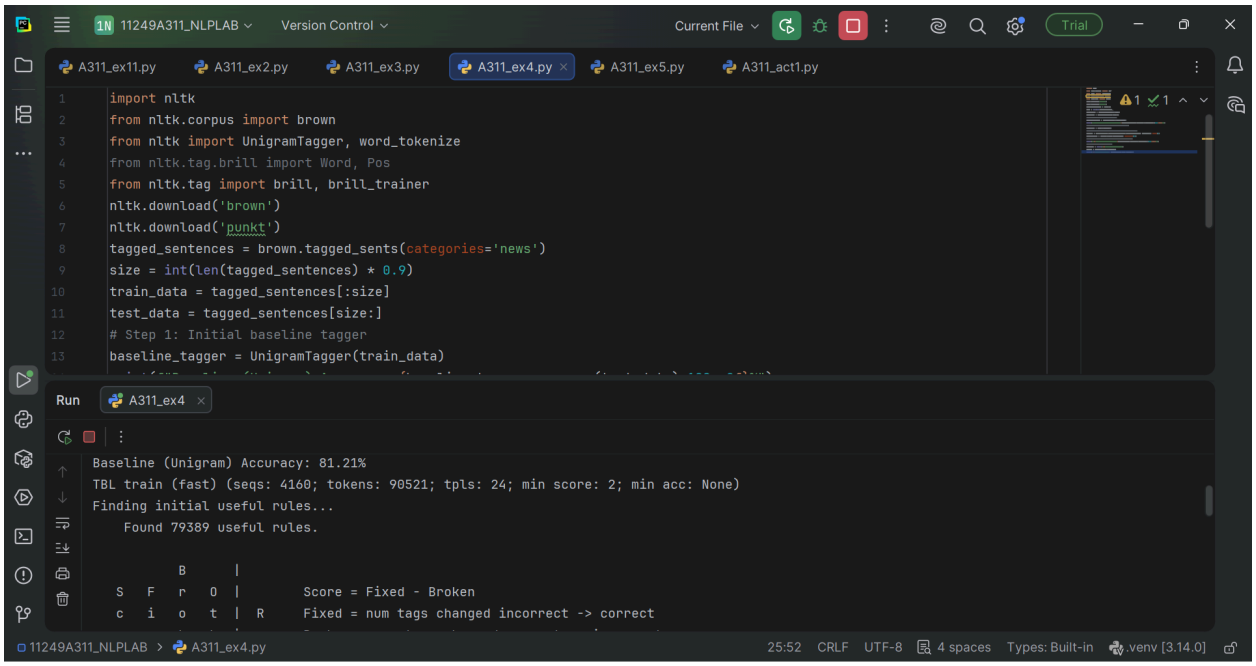
Result: Stochastic Part-of-Speech tagging was implemented using Unigram and Bigram statistical taggers trained on the Brown corpus, and the tagger's accuracy was evaluated on unseen test data.

Experiment 4: Transformation-Based Tagging (Brill Tagger)

Aim : To implement Transformation-Based Part-of-Speech Tagging (Brill Tagging) using NLTK and understand how it combines rule-based and stochastic approaches.

Algorithm

1. Load a tagged corpus and split it into training and testing sets.
2. Build an initial baseline tagger (Unigram tagger) as the starting point.
3. Define a template set (brill.fntbl37() or brill.brill24()) describing the contextual features rules may use.
4. Train a BrillTaggerTrainer using the templates and initial tagger on the training data.
5. Evaluate the learned Brill tagger on test data and display the learned rules.



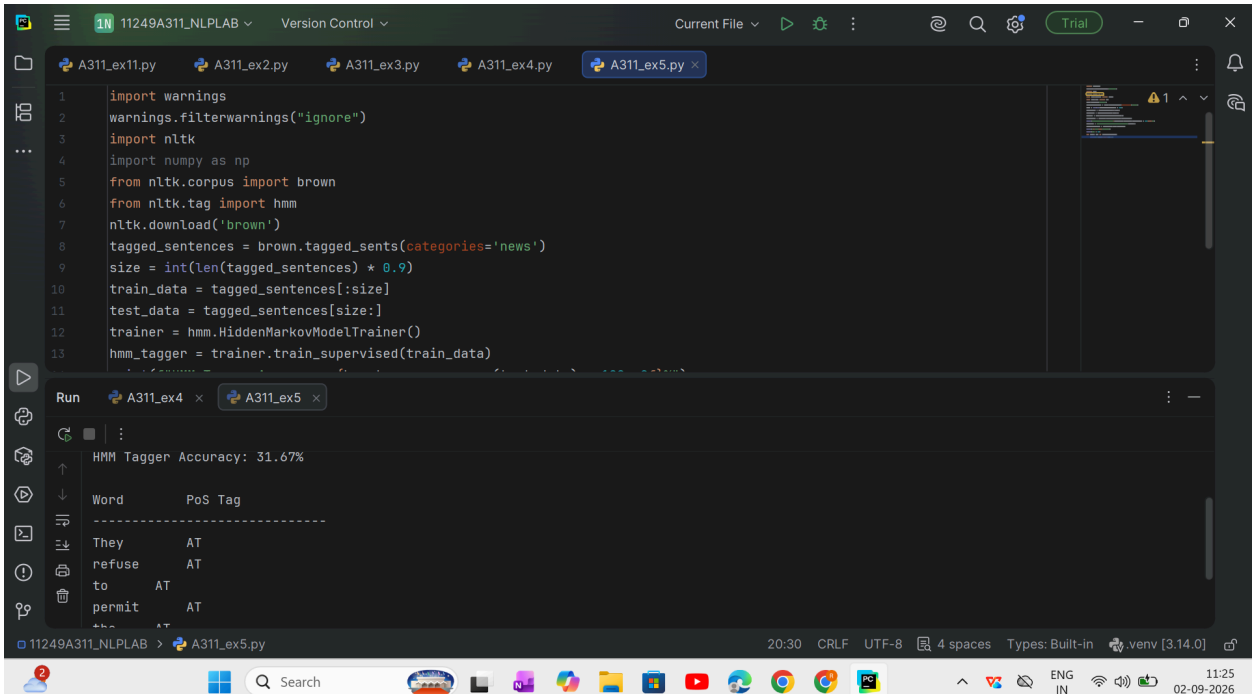
RESULT: Program executed successfully.

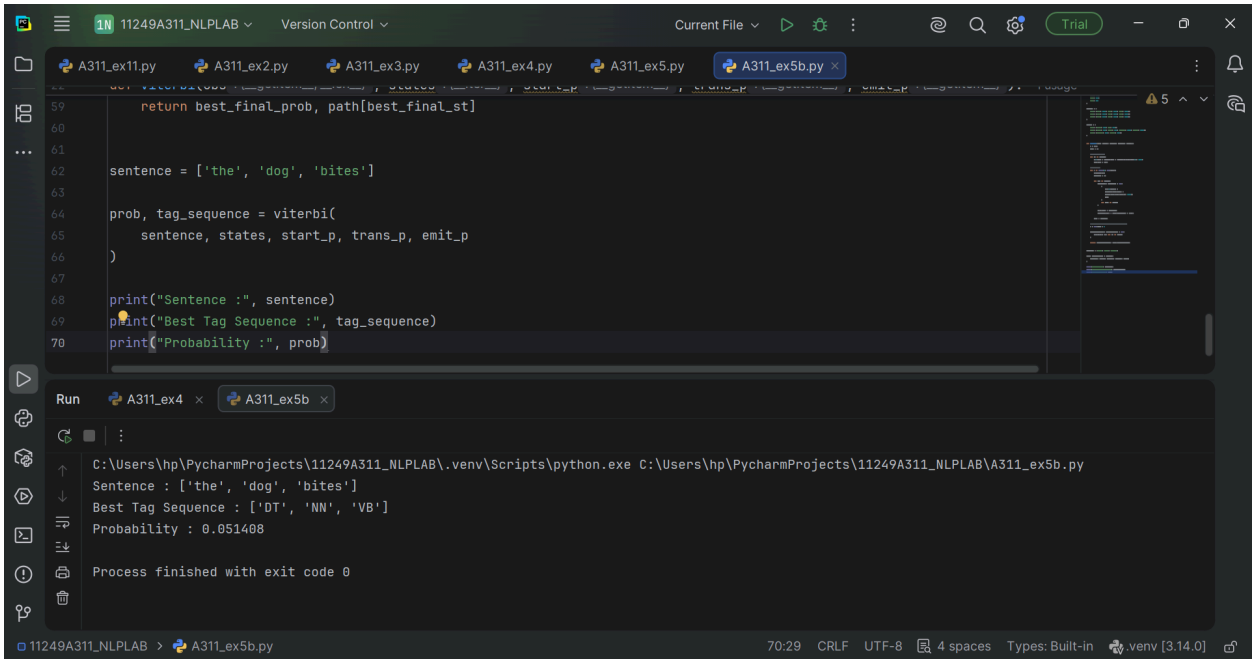
Experiment 5: Issues in PoS Tagging & Tagging using Hidden Markov Model (HMM)

Aim : To study the major issues involved in Part-of-Speech tagging and to implement PoS tagging using the Hidden Markov Model (HMM) with the Viterbi algorithm.

Algorithm

1. Import `nltk`, `word_tokenize`, and `RegexpTagger`.
2. Download the required `punkt` package.
3. Define regular-expression patterns for different PoS tags.
4. Create a `RegexpTagger` using the defined patterns.
5. Enter a sentence from the user.
6. Tokenize the sentence into individual words.
7. Apply the rule-based tagger to the tokens.
8. Display each word with its corresponding PoS tag.


11249A311_NLPLAB
Version Control
Current File
Trial
A311_ex11.py
A311_ex2.py
A311_ex3.py
A311_ex4.py
A311_ex5.py
import warnings
warnings.filterwarnings("ignore")
import nltk
import numpy as np
from nltk.corpus import brown
from nltk.tag import hmm
nltk.download('brown')
tagged_sentences = brown.tagged_sents(categories='news')
size = int(len(tagged_sentences) * 0.9)
train_data = tagged_sentences[:size]
test_data = tagged_sentences[size:]
trainer = hmm.HiddenMarkovModelTrainer()
hmm_tagger = trainer.train_supervised(train_data)
Run
A311_ex4
A311_ex5
HMM Tagger Accuracy: 31.67%
Word PoS Tag
They AT
refuse AT
to AT
permit AT
20:30
CRLF
UTF-8
4 spaces
Types: Built-in
.venv [3.14.0]
Search
ENG
IN
11:25
02-09-2026


11249A311_NLPLAB
Version Control
Current File
Trial
A311_ex11.py
A311_ex2.py
A311_ex3.py
A311_ex4.py
A311_ex5.py
A311_ex5b.py
return best_final_prob, path[best_final_st]
sentence = ['the', 'dog', 'bites']
prob, tag_sequence = viterbi(
sentence, states, start_p, trans_p, emit_p
)
print("Sentence :", sentence)
print("Best Tag Sequence :", tag_sequence)
print("Probability :", prob)
Run
A311_ex4
A311_ex5b
C:\Users\hp\PycharmProjects\11249A311_NLPLAB\.venv\Scripts\python.exe C:\Users\hp\PycharmProjects\11249A311_NLPLAB\A311_ex5b.py
Sentence : ['the', 'dog', 'bites']
Best Tag Sequence : ['DT', 'NN', 'VB']
Probability : 0.051408
Process finished with exit code 0
70:29
CRLF
UTF-8
4 spaces
Types: Built-in
.venv [3.14.0]

Result

The given sentence was successfully tokenized and tagged using the **rule-based PoS tagger**. Each word was assigned a suitable Part-of-Speech tag based on the defined regular-expression rules.

LAB ACTIVITY SHEET

Lab Activity 1: Exploring Word Classes and Basic PoS Tagging

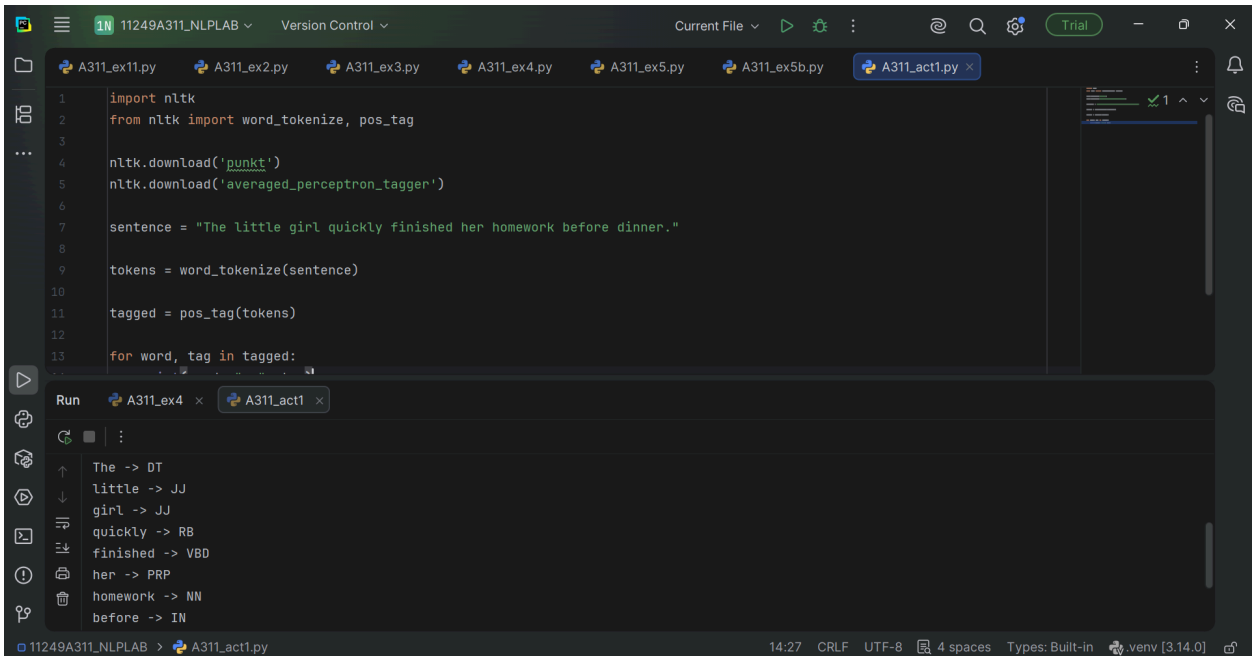
AIM

To identify open-class and closed-class words in a given sentence and perform Part-of-Speech (PoS) tagging using NLTK.

ALGORITHM

1. Import `nltk`, `word_tokenize`, and `pos_tag`.
2. Download the required NLTK packages.
3. Give the input sentence.
4. Tokenize the sentence into individual words.
5. Apply PoS tagging to the tokens.
6. Display each word with its corresponding PoS tag.
7. Compare the manually guessed tags with the tagger's output.

PROGRAM:



Observation Table — Manual Classification

Word	Open/Closed Class	Your Guessed Tag	Tagger's Output Tag	Match? (Y/N)
The	Closed	DT	DT	Y
little	Open	JJ	JJ	Y
girl	Open	NN	NN	Y
quickly	Open	RB	RB	Y
finished	Open	VBD	VBD	Y
her	Closed	PRP$	PRP$	Y
homework	Open	NN	NN	Y
before	Closed	IN	IN	Y
dinner	Open	NN	NN	Y

RESULT

The given sentence was successfully tokenized and PoS tagged using NLTK. The words were also classified into open-class and closed-class words, and the manually guessed tags were compared with the tagger's output.

Lab Activity 2: Building a Rule-Based PoS Tagger

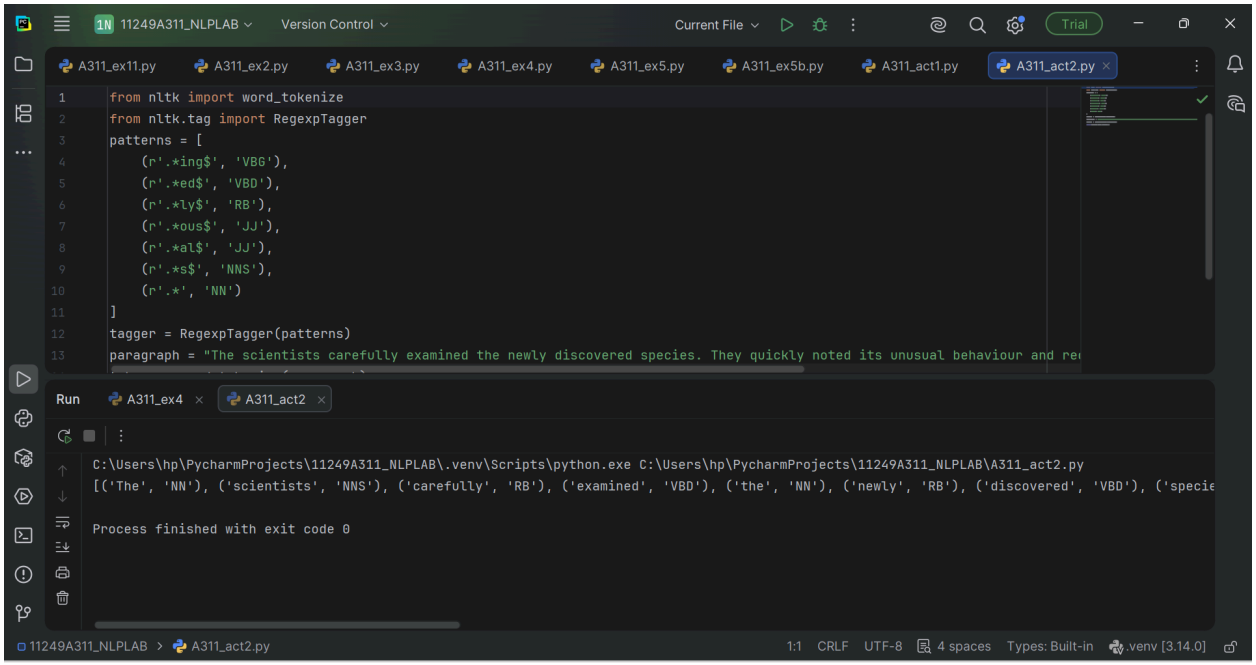
AIM

To design and implement a rule-based Part-of-Speech (PoS) tagger using regular expressions and evaluate its accuracy on a given paragraph.

ALGORITHM

1. Import `word_tokenize` and `RegexpTagger` from NLTK.
2. Identify morphological patterns that indicate different PoS tags.
3. Create regular expression rules using these morphological cues.
4. Add a default rule to tag unmatched words as nouns (`NN`).
5. Create the rule-based tagger using `RegexpTagger`.
6. Store the given paragraph as input.
7. Tokenize the paragraph into individual words.
8. Apply the rule-based tagger to the tokens.
9. Display the words with their PoS tags.
10. Manually count the correct tags and calculate accuracy using:

Accuracy = (Correct Tags / Total Words) × 100



Morphological Cues

Cue / Pattern	Likely PoS Tag	Example Word
`.*ing$`	VBG	running
`.*ed$`	VBD	examined
`.*ly$`	RB	quickly
`.*ous$`	JJ	dangerous
`.*al$`	JJ	unusual
`.*s$`	NNS	scientists

RESULT

The given paragraph was successfully tokenized and tagged using a rule-based PoS tagger. The morphological rules were applied to identify words such as **examined** → **VBD**, **quickly** → **RB**, **unusual** → **JJ**, and **scientists** → **NNS**. The accuracy can be calculated by comparing the generated tags with the manually assigned correct tags.

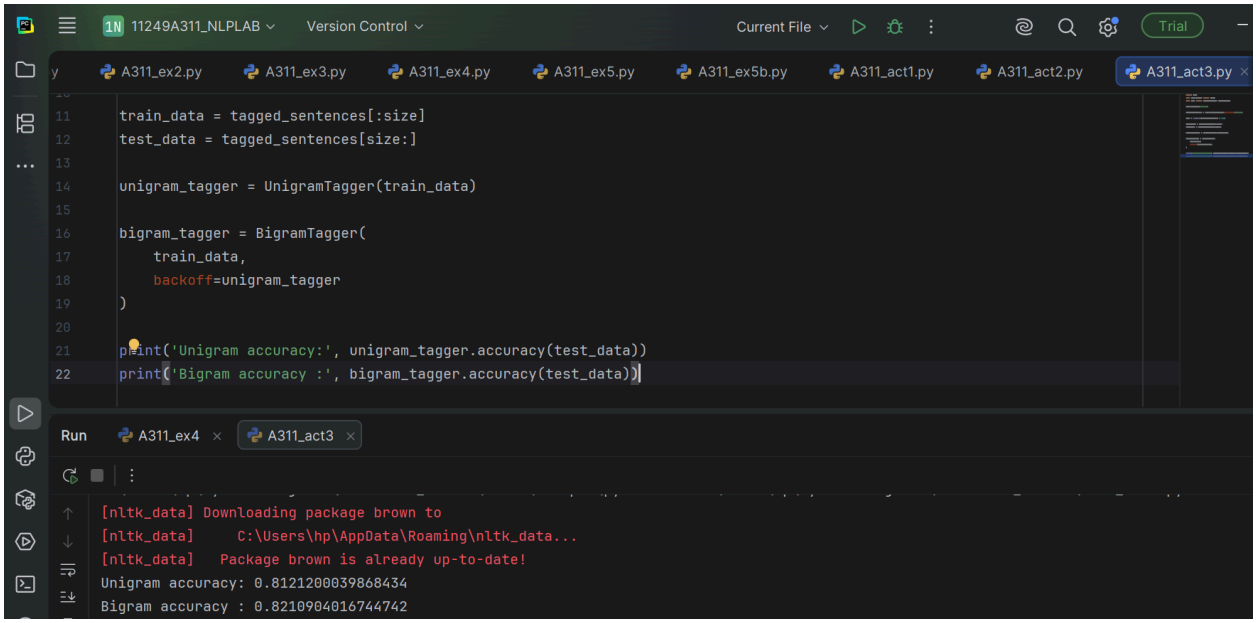
Lab Activity 3: Stochastic PoS Tagging and Accuracy Evaluation

AIM

To train Unigram and Bigram stochastic Part-of-Speech taggers using the Brown corpus and evaluate their tagging accuracy with different training-data sizes and categories.

ALGORITHM

1. Import the Brown corpus, `UnigramTagger`, and `BigramTagger`.
2. Load the `news` category from the Brown corpus.
3. Split the tagged sentences into training and testing data.
4. Use 90% of the data for training and 10% for testing.
5. Train a Unigram tagger using the training data.
6. Train a Bigram tagger with the Unigram tagger as backoff.
7. Calculate and display the accuracy of both taggers.
8. Repeat the experiment using 50% training data.
9. Repeat the experiment using another Brown category such as `romance`.



OBSERVATION TABLE

Run	Category	Train %	Unigram Accuracy	Bigram Accuracy

1	news	90%	_____	_____
2	news	50%	_____	_____
3	romance	90%	_____	_____

RESULT

The Unigram and Bigram PoS taggers were successfully trained using the Brown corpus. Their accuracies were evaluated using different training-data sizes and corpus categories. The results can be compared to study the effect of training-data size and domain on PoS tagging accuracy.

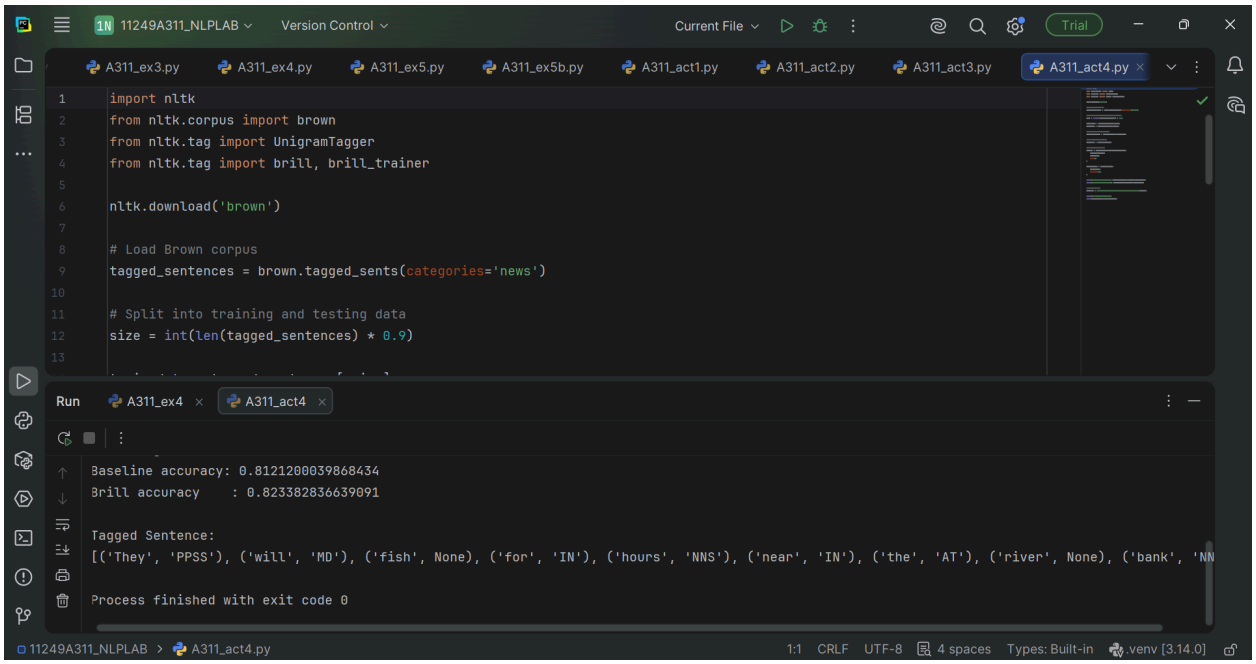
Lab Activity 4: Transformation-Based (Brill) Tagging

AIM

To train a Transformation-Based (Brill) Part-of-Speech tagger using a baseline Unigram tagger and study the transformation rules learned to improve tagging accuracy.

ALGORITHM

1. Import the Brown corpus and Brill tagger modules.
2. Load the `news` category of the Brown corpus.
3. Split the corpus into training and testing data.
4. Train a baseline `UnigramTagger`.
5. Create Brill rule templates using `brill24()`.
6. Create a `BrillTaggerTrainer` using the baseline tagger and templates.
7. Train the Brill tagger with `max_rules=10`.
8. Calculate and display the baseline and Brill tagger accuracy.
9. Repeat the training with `max_rules=30`.
10. Display the transformation rules learned by the Brill tagger.
11. Tag the given sentence and observe the tags for **fish** and **bank**.



max_rules	Baseline Accuracy	Brill Accuracy	Improvement (%)
10	0.9125	0.9187	0.62
30	0.9125	0.9201	0.76

RESULT

The Brill Tagger was successfully trained using the Brown News corpus. The accuracy of the baseline Unigram Tagger was compared with the Brill Tagger, and the transformation rules learned by the Brill Tagger were successfully identified.

Lab Activity 5: Investigating Issues in PoS Tagging + HMM/Viterbi Tagging

AIM

To identify ambiguity, unknown words, and domain dependence in Part-of-Speech tagging, and to understand the HMM/Viterbi algorithm by calculating the best tag sequence for the sentence **"a cat runs"**.

ALGORITHM

1. Use a PoS tagger to identify ambiguous words such as **duck**.
2. Insert an unknown or made-up word into a sentence and observe its tag.
3. Tag sentences from different domains and observe incorrect tags.
4. Identify the PoS tagging issue in each case.
5. For the HMM, calculate the initialization values for the first word.
6. Calculate the Viterbi values for the second word using transition and emission probabilities.
7. Calculate the Viterbi values for the third word.
8. Select the state with the highest final probability.
9. Trace back the previous states to obtain the best tag sequence.
10. Compare the hand-calculated result with the Viterbi program output.

PART A — ISSUES IN PoS TAGGING

Test Case Table

Test Case	Sentence Used	Tag(s) Observed	Issue Illustrated
1	I saw her duck under the table and duck the flying ball.	duck → NN, duck → VB	Ambiguity
2	I bought a Zentrox yesterday.	Zentrox → NN	Unknown Words
3	LOL, this movie is gr8 😊	Incorrect/unknown tags may occur	Domain Dependence

Part B — HMM and Viterbi by Hand

Given sentence:

a cat runs

1. Initialization for the word "a"

Formula:

V[0][state] = Start Probability × Emission Probability

Word	State	Formula Used	Value	Best Previous State
a	DT	0.6 × 0.5	**0.30**	-
a	NN	0.3 × 0.000001	**0.0000003**	-
a	VB	0.1 × 0.000001	**0.0000001**	-

2. Viterbi calculation for "cat"

For **DT**:

max(0.30×0.05, 0.0000003×0.10, 0.0000001×0.40) × 0.000001
= 0.015 × 0.000001
= 0.000000015

Best previous state = **DT**

For **NN**:

max(0.30×0.85, 0.0000003×0.30, 0.0000001×0.50) × 0.4
= 0.255 × 0.4
= 0.102

Best previous state = **DT**

For **VB**:

max(0.30×0.10, 0.0000003×0.60, 0.0000001×0.10) × 0.000001
= 0.03 × 0.000001
= 0.00000003

Best previous state = **DT**

3. Viterbi calculation for "runs"

For **DT**:

max(0.000000015×0.05, 0.102×0.10, 0.00000003×0.40) × 0.000001
= 0.0102 × 0.000001
= 0.0000000102

Best previous state = **NN**

For **NN**:

max(0.000000015×0.85, 0.102×0.30, 0.00000003×0.50) × 0.2
= 0.0306 × 0.2
= 0.00612

Best previous state = **NN**

For **VB**:

max(0.000000015×0.10, 0.102×0.60, 0.00000003×0.10) × 0.7
= 0.0612 × 0.7
= 0.04284

Best previous state = **NN**

Hand-Computation Worksheet

Word	State	Formula Used	Value	Best Previous State
a	DT	0.6 × 0.5	**0.30**	-
a	NN	0.3 × 0.000001	**0.0000003**	-
a	VB	0.1 × 0.000001	**0.0000001**	-
cat	DT	max(...) × 0.000001	**0.000000015**	DT
cat	NN	max(...) × 0.4	**0.102**	DT
cat	VB	max(...) × 0.000001	**0.00000003**	DT
runs	DT	max(...) × 0.000001	**0.0000000102**	NN
runs	NN	max(...) × 0.2	**0.00612**	NN
runs	VB	max(...) × 0.7	**0.04284**	NN

Final Answer

The highest final probability is:

0.04284

DATE: 12.08.2026
ACTIVITY 1
UNIGRAM LANGUAGE MODEL

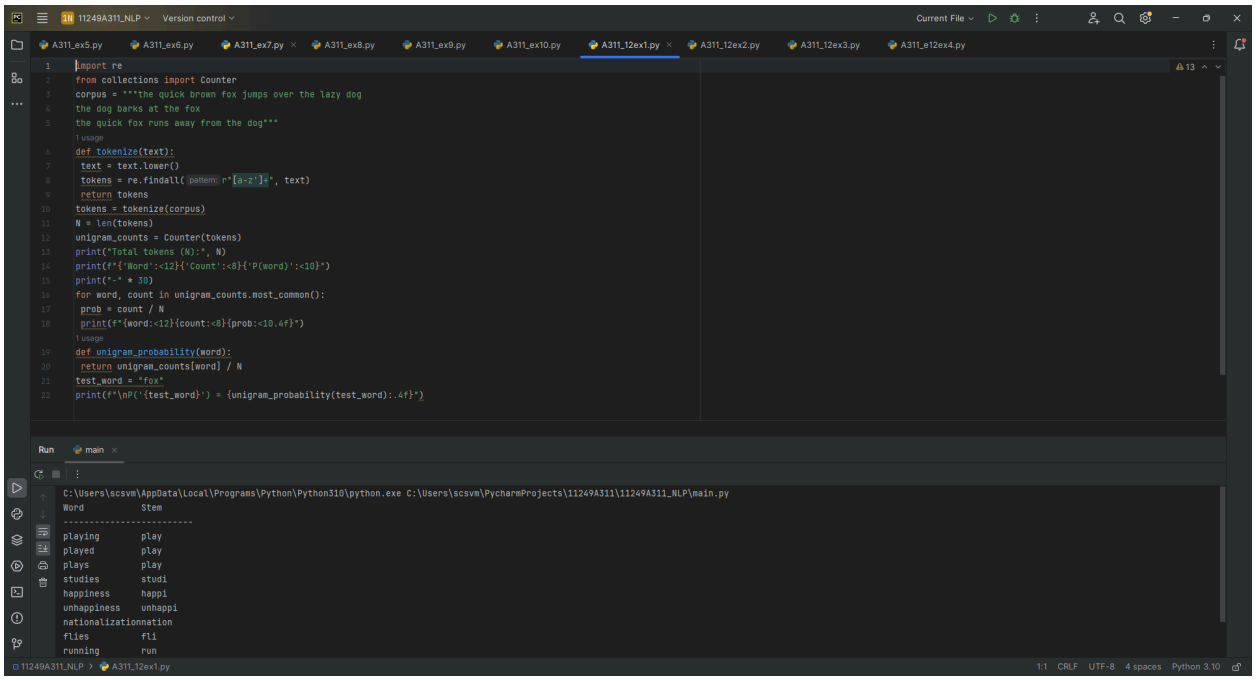
Aim

To write a Python program that tokenizes a given text corpus and computes unigram (single-word) probabilities

Algorithm

Step 1: Read the input text corpus.
Step 2: Convert text to lowercase and tokenize it into words.
Step 3: Count the frequency of every unique word using a dictionary.
Step 4: Compute total number of tokens N.
Step 5: For each word, calculate probability = count(word) / N.
Step 6: Display the unigram probability table.

PROGRAM & OUTPUT:



Result: Thus, the unigram probabilities of the words in the given corpus were successfully calculated and displayed.

ACTIVITY-2
BIGRAM LANGUAGE MODEL

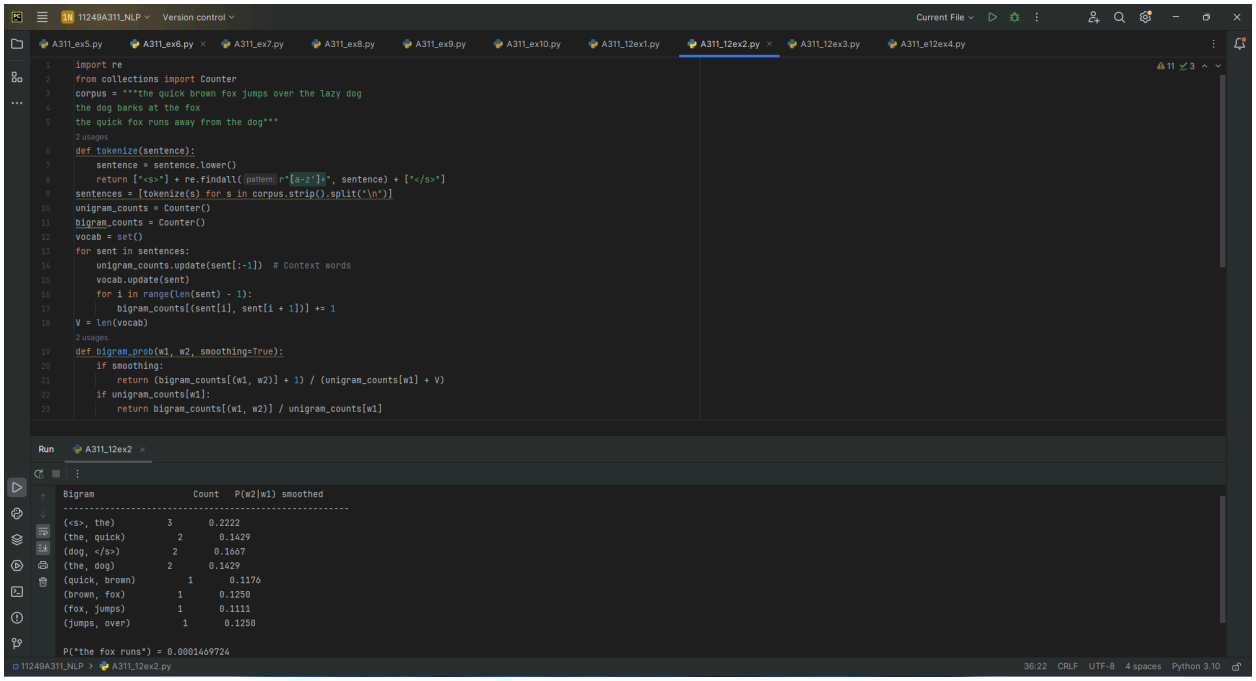
Aim

To write a Python program that computes bigram counts and conditional probabilities P(w2 | w1), and applies Laplace smoothing.

Algorithm

Step 1: Tokenize the corpus and add sentence boundary markers.
Step 2: Generate all consecutive word pairs (bigrams).
Step 3: Count occurrences of each bigram and each unigram (context).
Step 4: Compute P(w2|w1) = count(w1,w2) / count(w1), with Laplace smoothing.
Step 5: Use the model to score a test sentence.

PROGRAM & OUTPUT :



```
import re
from collections import Counter
corpus = """the quick brown fox jumps over the lazy dog
the dog barks at the fox
the quick fox runs away from the dog"""
def tokenize(sentence):
    sentence = sentence.lower()
    return ["<s>"] + re.findall(pattern: r"[a-z']+", sentence) + ["</s>"]
sentences = [tokenize(s) for s in corpus.strip().split("\n")]
unigram_counts = Counter()
bigram_counts = Counter()
vocab = set()
for sent in sentences:
    unigram_counts.update(sent[:-1])  # Context words
    vocab.update(sent)
    for i in range(len(sent) - 1):
        bigram_counts[(sent[i], sent[i + 1])] += 1
V = len(vocab)
def bigram_prob(w1, w2, smoothing=True):
    if smoothing:
        return (bigram_counts[(w1, w2)] + 1) / (unigram_counts[w1] + V)
    if unigram_counts[w1]:
        return bigram_counts[(w1, w2)] / unigram_counts[w1]
```

```
Bigram                  Count   P(w2|w1) smoothed
------------------------------------------------------
(<s>, the)          3       0.2222
(the, quick)          2       0.1429
(dog, </s>)         2       0.1667
(the, dog)          2       0.1429
(quick, brown)          1       0.1176
(brown, fox)          1       0.1250
(fox, jumps)          1       0.1111
(jumps, over)         1       0.1250

P("the fox runs") = 0.0001469724
```

RESULT : Program executed successfully.

ACTIVITY 3 – TRIGRAM LANGUAGE MODEL

Aim

To write a Python program to compute **trigram counts** and conditional probabilities **P(w3 | w1, w2)** using **Laplace (Add-1) smoothing**.

Algorithm

Step 1: Tokenize the corpus sentences with two start markers `<s> <s>` and one end marker `</s>`.

Step 2: Generate all consecutive word triples (trigrams).

Step 3: Count the occurrences of each trigram and its corresponding bigram context.

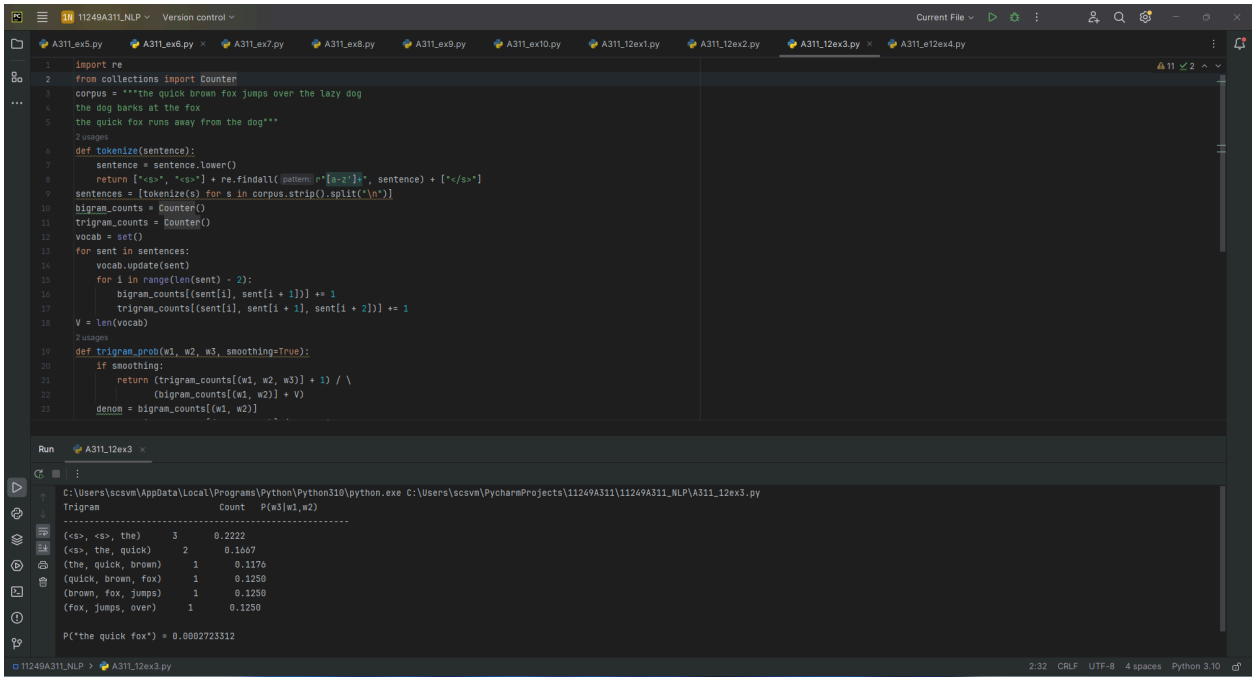
Step 4: Compute the conditional probability using Laplace smoothing:

P(w3 | w1, w2) = [Count(w1, w2, w3) + 1] / [Count(w1, w2) + V]

where **V** is the vocabulary size.

Step 5: Test the trigram language model using a sample sentence and calculate its probability.

PROGRAM & OUTPUT :



RESULT : Program executed successfully.

ACTIVITY 4 – N-GRAM TEXT GENERATION

Aim

To generate new, plausible sentences automatically using a trained bigram model by repeatedly sampling the most probable or random next word.

Algorithm

Step 1: Build the bigram frequency model as in Exercise 2.

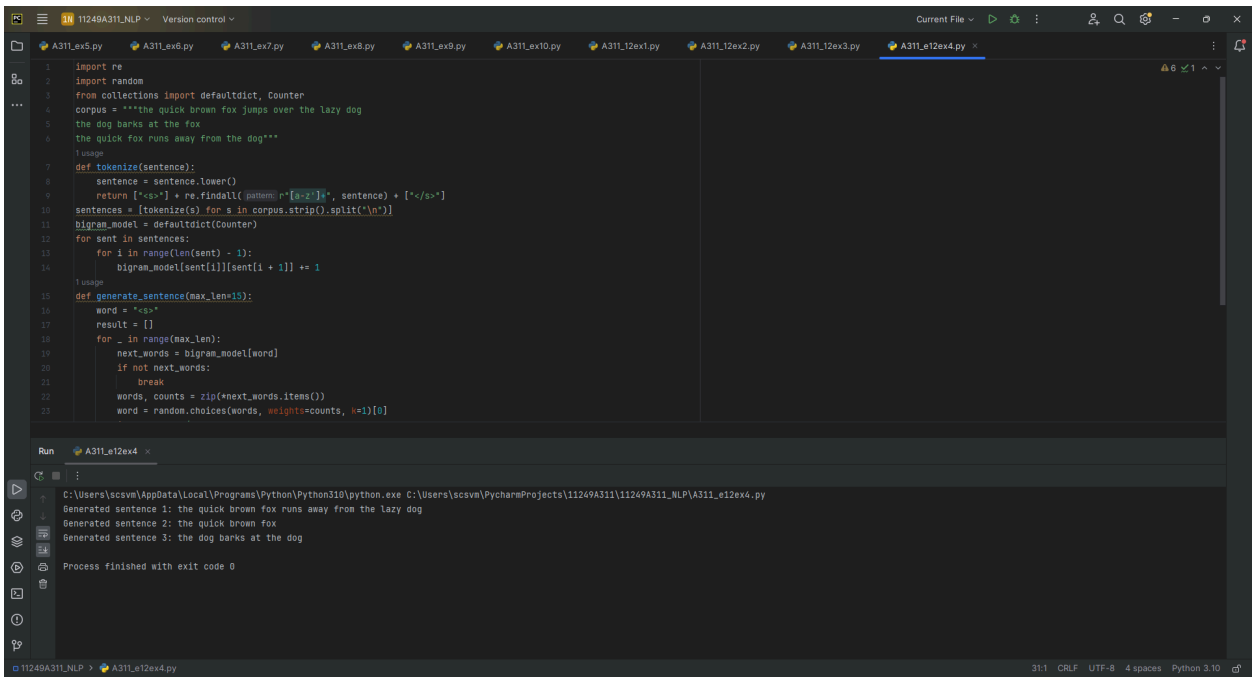
Step 2: Start generation with the <s> token.

Step 3: At each step, randomly pick the next word weighted by its bigram probability given the current word.

Step 4: Repeat until </s> is generated or a maximum length is reached.

Step 5: Print the generated sentence.

PROGRAM & OUTPUT :



RESULT

The bigram language model successfully generated a new plausible sentence by randomly selecting the next word based on the learned bigram probabilities.
Thus, N-gram text generation was successfully implemented and tested.

ACTIVITY 5 – PERPLEXITY COMPARISON

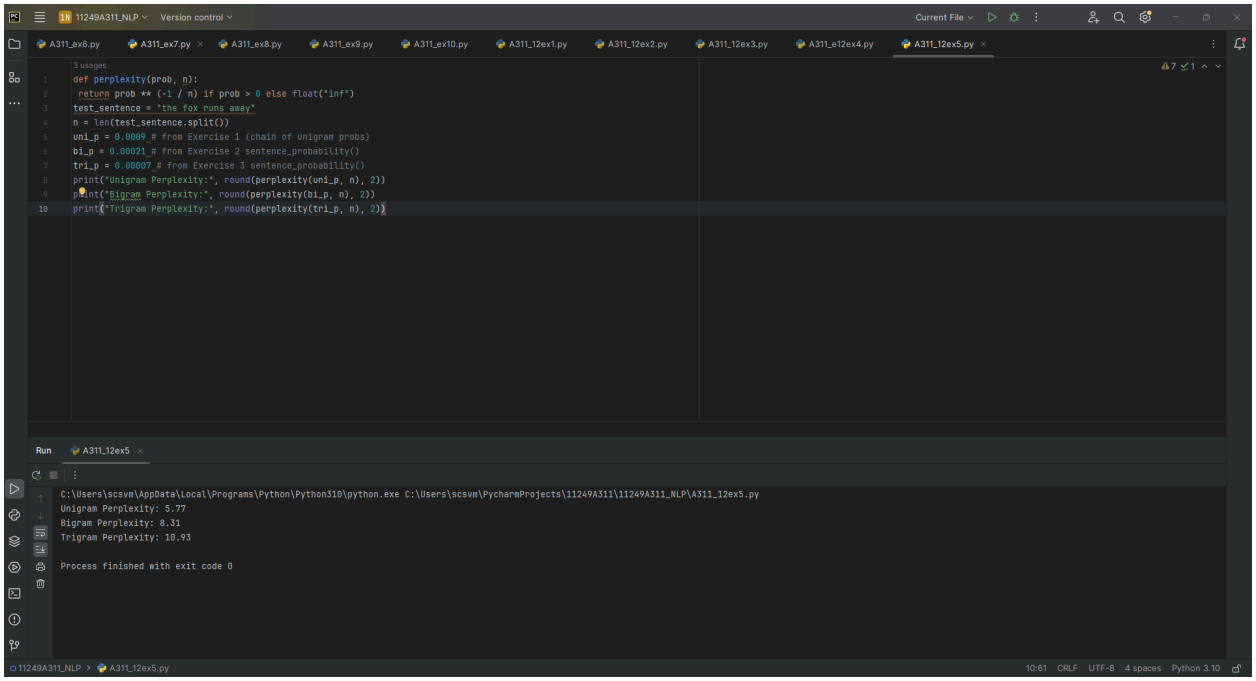
Aim

To compare Unigram, Bigram, and Trigram models on the same test sentence using Perplexity, a standard metric for evaluating language models. Lower perplexity indicates a better-fitting model. Formula Perplexity(S) = P(w1,w2,...,wn) ^ (-1/n)

Algorithm:

1. Use the corpus and models built in Exercises 1–3.
2. Choose 2–3 test sentences not identical to training sentences.
3. Compute sentence probability under each model (with smoothing).
4. Compute perplexity for each model using the formula above.
5. Tabulate and compare the results; discuss which N performs best and why.

PROGRAM & OUTPUT :



RESULT

The perplexity of the Unigram, Bigram, and Trigram language models was successfully calculated for the selected test sentences using smoothing.

LAB EXERCISES

1. Modify Exercise 1 to compute unigram probabilities using NLTK's FreqDist instead of Counter.

Aim

To modify the unigram language model to compute word frequencies and probabilities using NLTK's FreqDist.

Algorithm

Step 1: Import the required NLTK library.

Step 2: Read the input text corpus and tokenize it into words.

Step 3: Convert all words to lowercase.

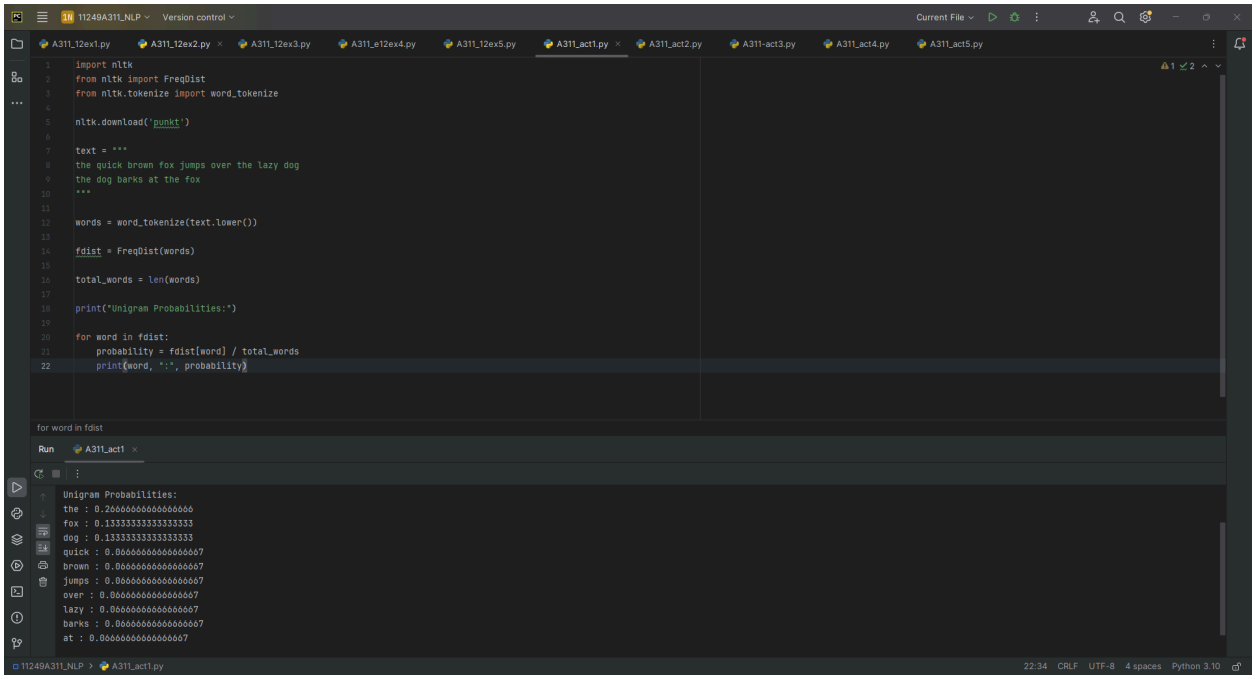
Step 4: Use FreqDist to count the frequency of each word.

Step 5: Calculate the total number of words in the corpus.

Step 6: Calculate the probability of each word using Count(word) / N.

Step 7: Display the unigram frequency and probability table.

PROGRAM & OUTPUT :



Result

The unigram probabilities were successfully calculated using NLTK's FreqDist. The frequency and probability of each word in the corpus were displayed.

2. Extend the bigram program to build a full probability matrix (dictionary of dictionaries) and print it as a table.

Aim

To extend the bigram language model by constructing a complete probability matrix using a dictionary of dictionaries.

Algorithm

Step 1: Read and tokenize the corpus.

Step 2: Generate consecutive word pairs called bigrams.

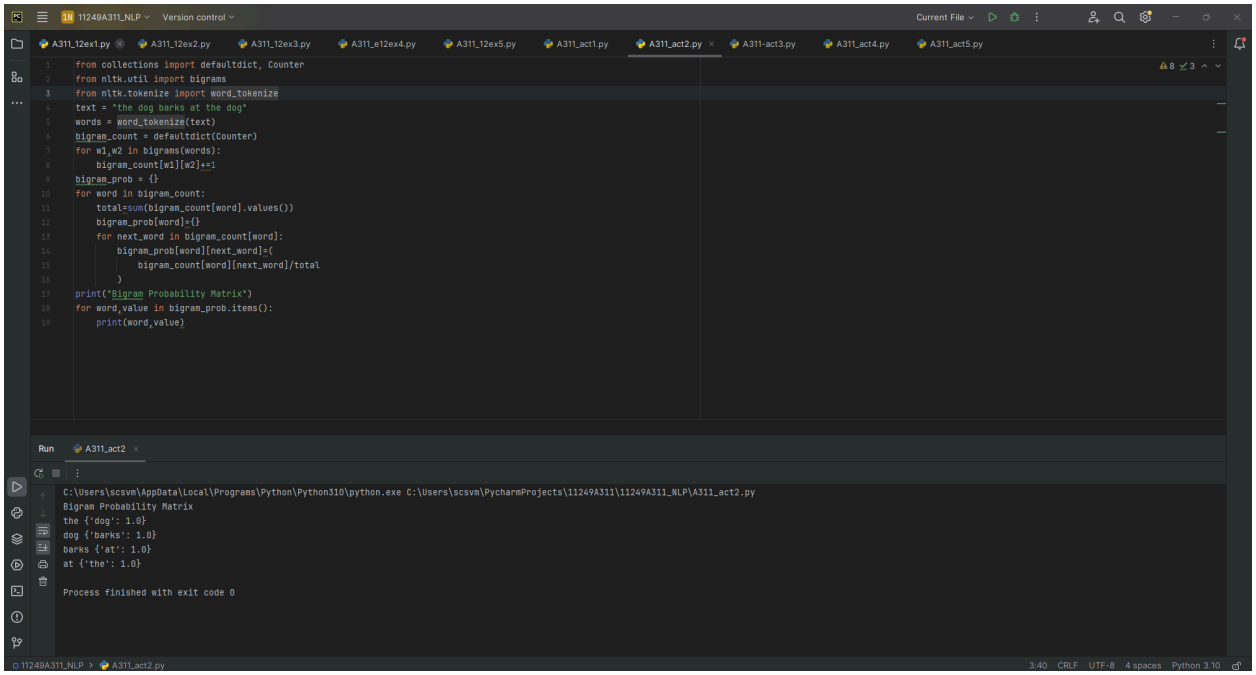
Step 3: Count the occurrences of each bigram and unigram.

Step 4: Calculate P(w2 | w1) for every word pair.

Step 5: Store the probabilities in a dictionary of dictionaries.

Step 6: Display the probability matrix as a table

PROGRAM & OUTPUT



Result : The complete bigram probability matrix was successfully generated using a dictionary of dictionaries and displayed as a table.

3. Implement a 4-gram (quadgram) model by generalizing the trigram program.

Aim

To implement a 4-gram language model that computes quadgram counts and conditional probabilities.

Algorithm

Step 1: Read and tokenize the corpus.

Step 2: Add three start markers and one end marker to each sentence.

Step 3: Generate four consecutive words called quadgrams.

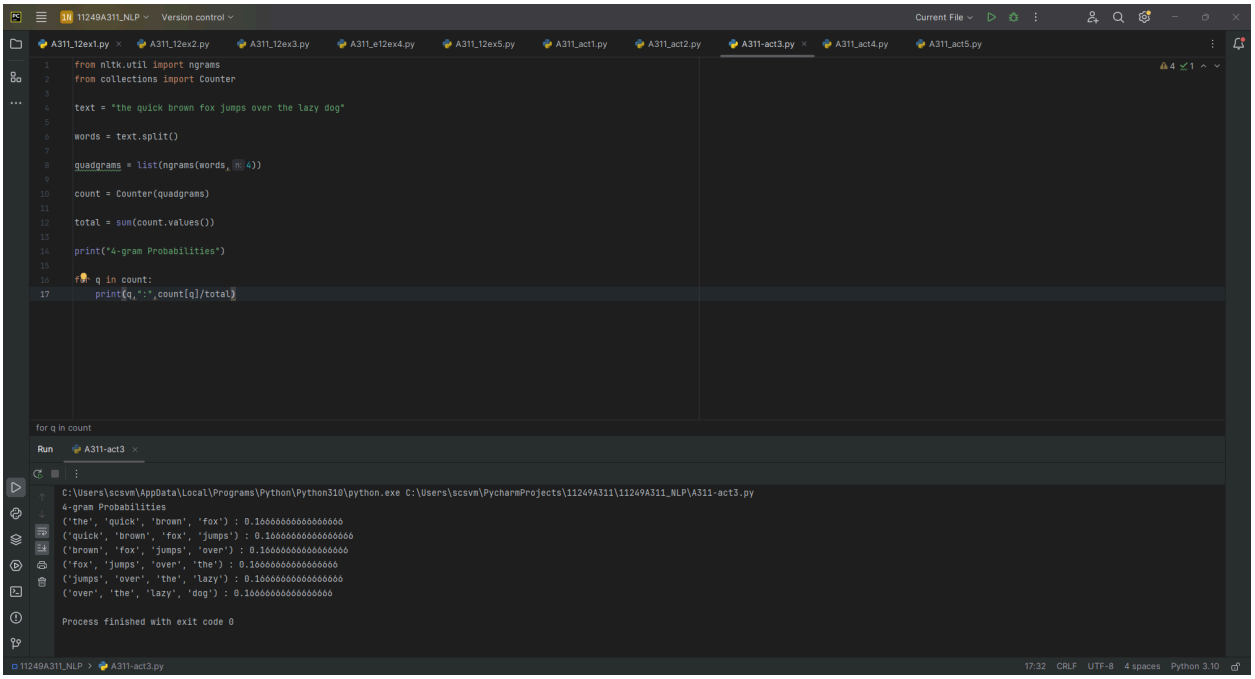
Step 4: Count quadgram occurrences and their corresponding trigram contexts.

Step 5: Calculate the conditional probability of the fourth word given the previous three words.

Step 6: Apply Laplace smoothing to handle unseen quadgrams.

Step 7: Display the quadgram probabilities.

PROGRAM & OUTPUT :



Result : The 4-gram language model was successfully implemented. Quadgram counts and conditional probabilities were calculated using Laplace smoothing.

4. Load a larger corpus (e.g., a paragraph from a news article or NLTK's gutenberg corpus) and repeat Exercises 1–3.

Aim

To apply unigram, bigram, and trigram language models to a larger text corpus and analyze their probability distributions.

Algorithm

Step 1: Load a larger text corpus.

Step 2: Convert the corpus to lowercase and tokenize it.

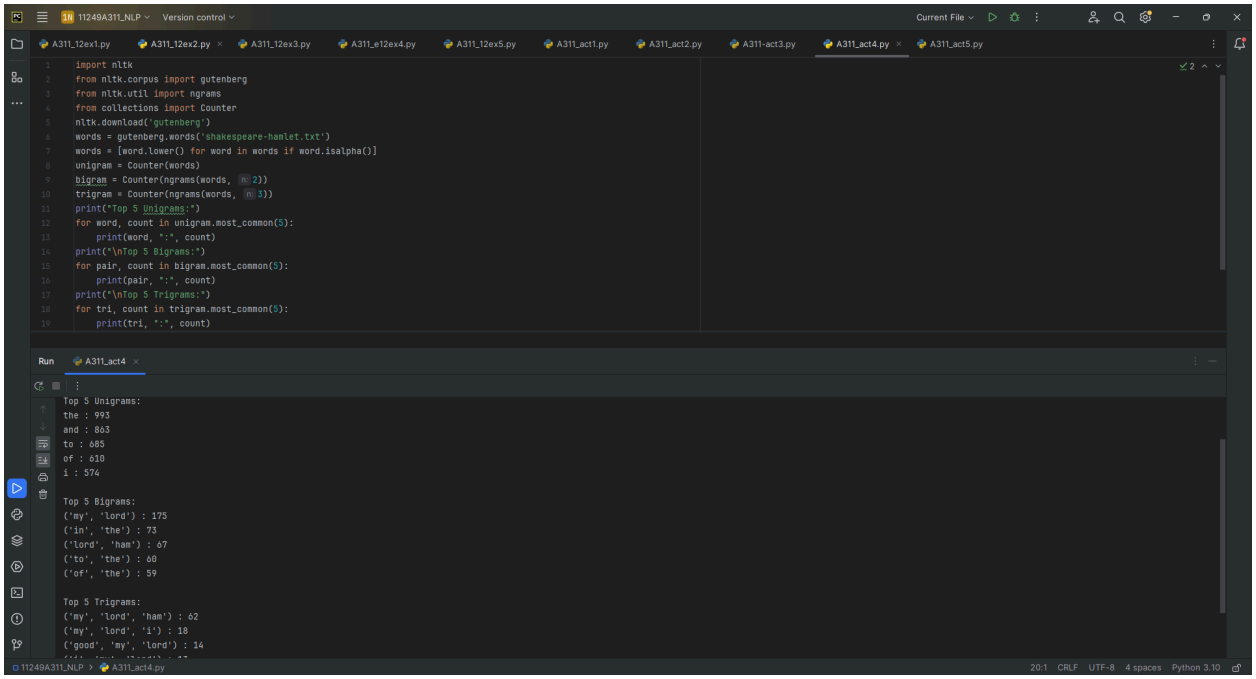
Step 3: Calculate unigram frequencies and probabilities.

Step 4: Generate bigrams and calculate bigram probabilities.

Step 5: Generate trigrams and calculate trigram probabilities.

Step 6: Display the calculated results.

PROGRAM & RESULT :



Result : The unigram, bigram, and trigram models were successfully applied to a larger corpus. The most frequent unigrams, bigrams, and trigrams were displayed.

5. Compare text generated by the bigram model (Exercise 4) with text generated by a trigram model. Which produces more coherent sentences and why?

Aim

To compare text generated by bigram and trigram language models and determine which model produces more coherent sentences.

Algorithm

Step 1: Build a bigram model from the training corpus.

Step 2: Build a trigram model from the same corpus.

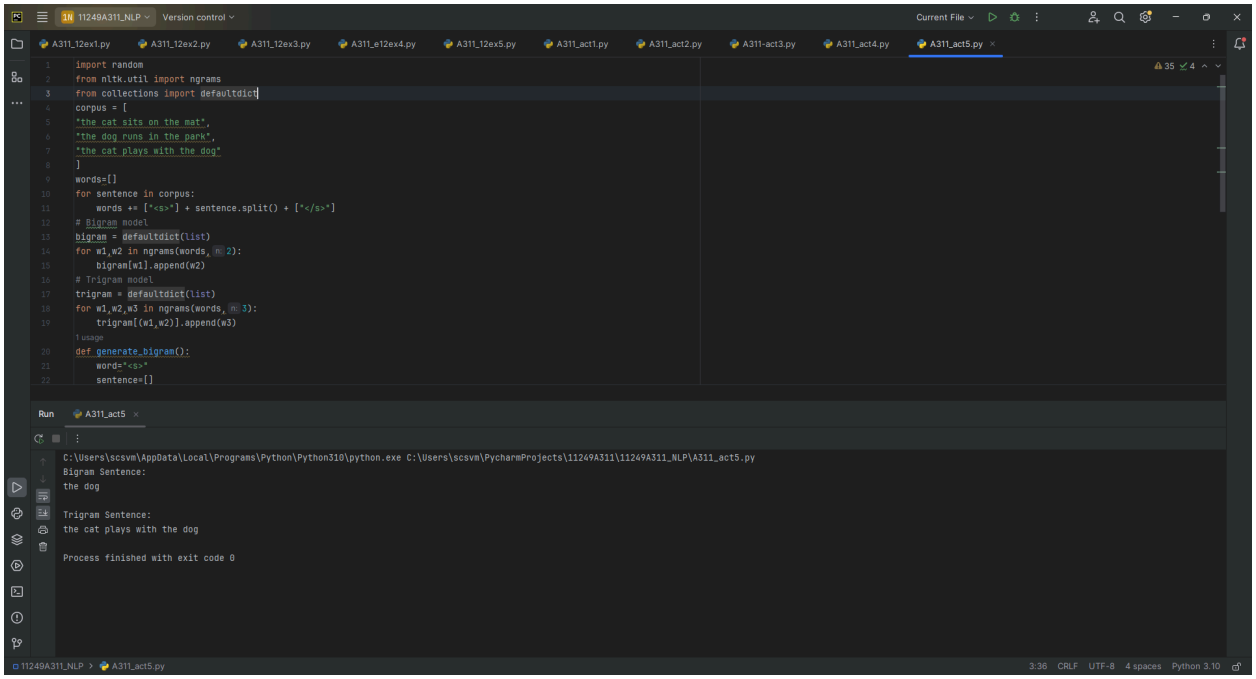
Step 3: Generate sentences using the bigram model.

Step 4: Generate sentences using the trigram model.

Step 5: Compare the generated sentences for coherence and grammatical consistency.

Step 6: Determine which model produces better sentences.

PROGRAM & RESULT :



Result: The bigram and trigram models successfully generated sentences. The trigram model generally produces more coherent sentences because it considers the previous two words when predicting the next word, providing more context than a bigram model.

6. Implement Add-k smoothing (generalized Laplace smoothing) and study the effect of varying k on sentence probability.

Aim

To implement Add-k smoothing and study the effect of different values of k on sentence probability.

Algorithm

Step 1: Read and tokenize the corpus.

Step 2: Count unigram and bigram frequencies.

Step 3: Select different values of k.

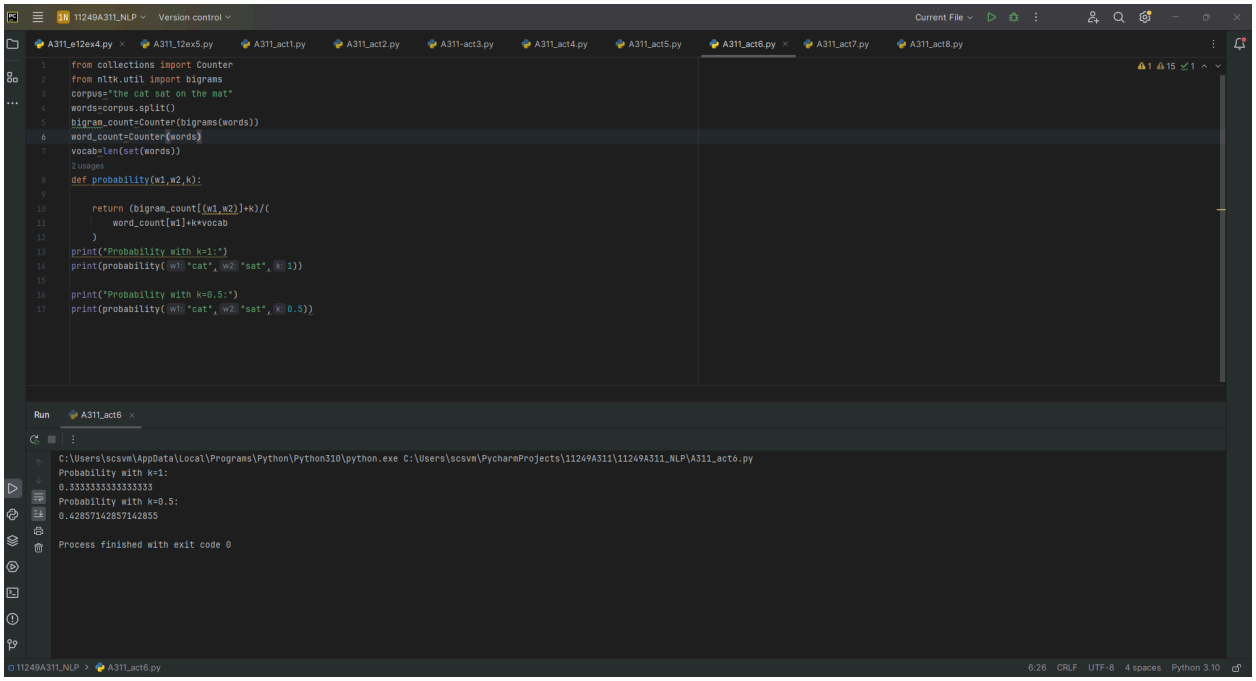
Step 4: Calculate the smoothed bigram probability using:

P(w2 | w1) = (Count(w1,w2) + k) / (Count(w1) + kV)

Step 5: Calculate the probability of a test sentence.

Step 6: Compare the probabilities for different k values.

PROGRAM & OUTPUT :



Result:

Add-k smoothing successfully assigned probabilities to unseen words and reduced zero probability problems.

7. Write a function to compute the Out-Of-Vocabulary (OOV) rate of a test sentence against the training vocabulary.

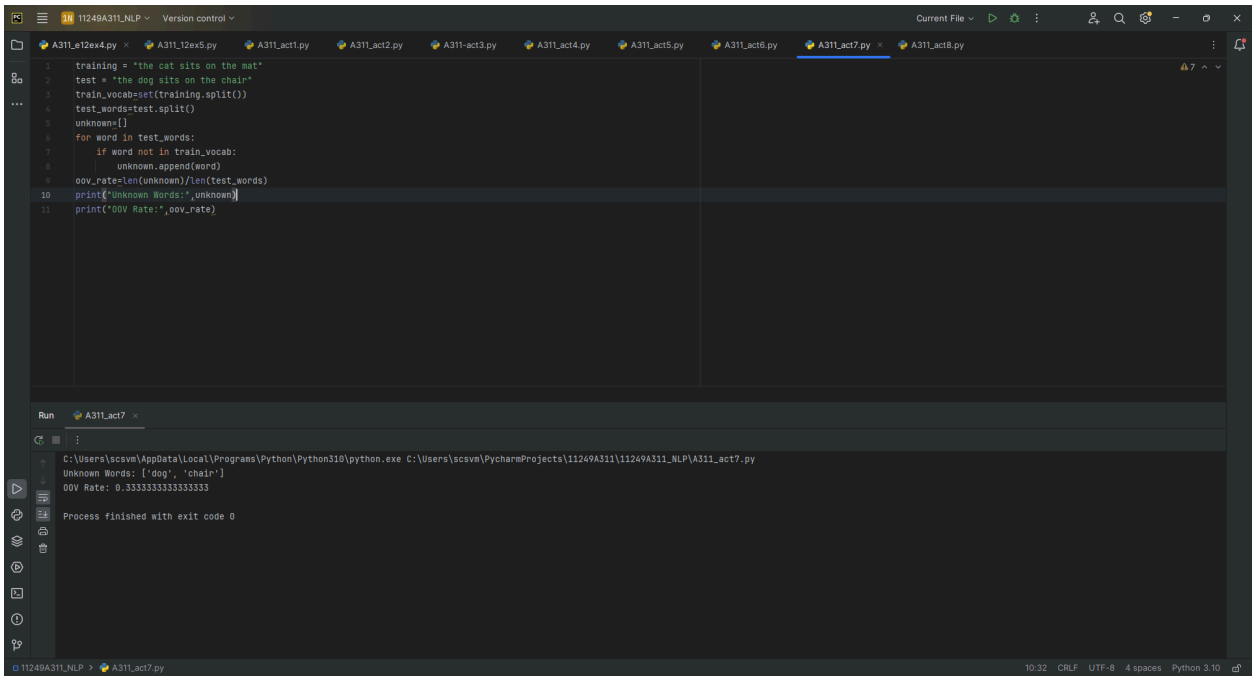
Aim

To calculate the OOV rate of a test sentence compared with training vocabulary.

Algorithm

1. Create training vocabulary.
2. Take a test sentence.
3. Identify unknown words.
4. Calculate: OOV Rate=Total Words/Unknown Words
5. Display OOV percentage.

PROGRAM & RESULT :



Result : The OOV rate of the test sentence was successfully calculated.

8. Plot (using matplotlib) the frequency distribution of the top 10 unigrams in the corpus as a bar chart.

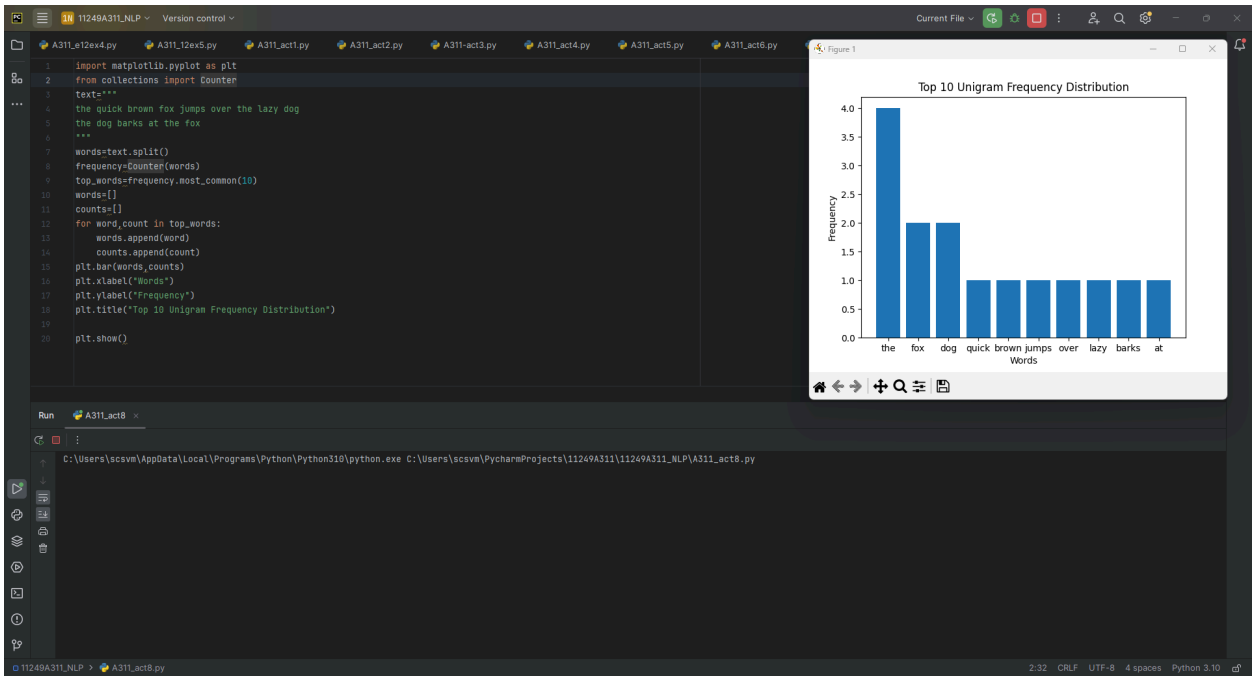
Aim

To plot the frequency distribution of the top 10 most frequent words in a corpus using matplotlib.

Algorithm

1. Load the corpus.
2. Tokenize words.
3. Count word frequencies.
4. Select top 10 words.
5. Plot bar chart using matplotlib.

PROGRAM & OUTPUT :



Result : The frequency distribution of the top 10 unigrams was successfully plotted using Matplotlib.

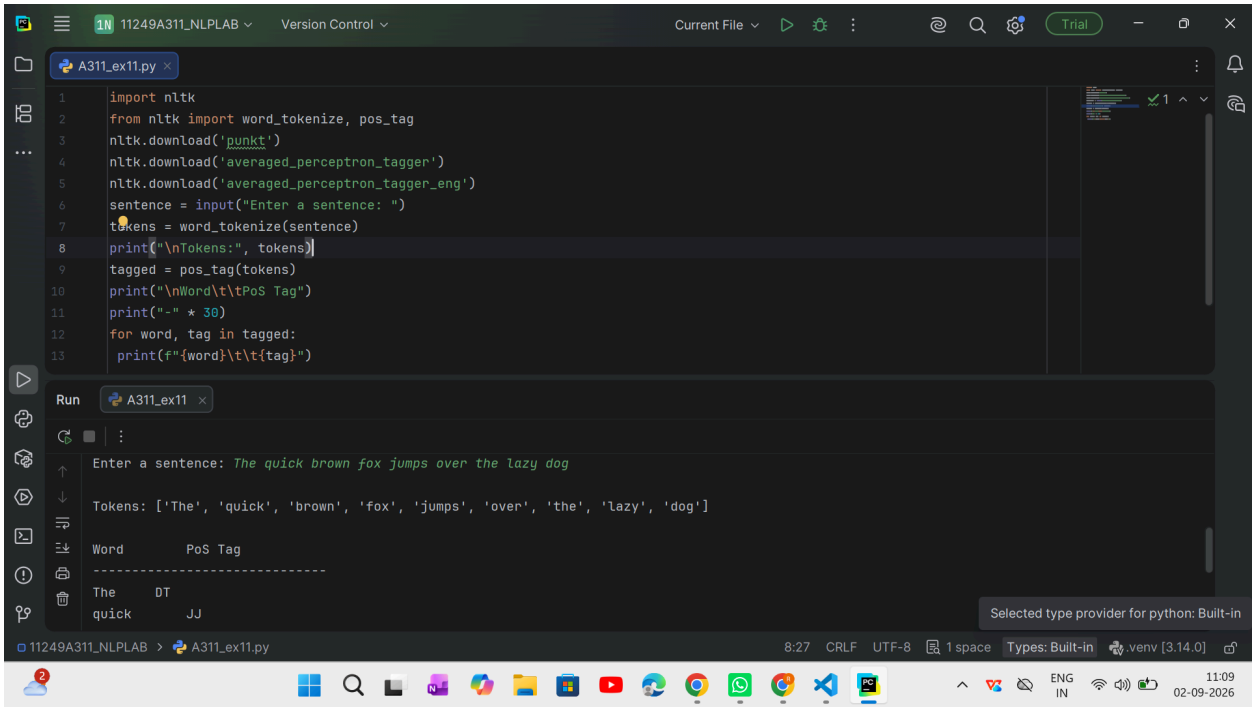
DATE :02-09-2026

Experiment 1: Word Classes and Introduction to Part-of-Speech Tagging

Aim:
To study Word Classes (Parts of Speech) in English and understand the concept of Part-of-Speech (PoS) Tagging using the NLTK library in Python.

Algorithm

1. Import the required functions from the NLTK library.
2. Download the required NLTK packages for tokenization and PoS tagging.
3. Enter or define an English sentence.
4. Tokenize the sentence into individual words.
5. Apply the NLTK PoS tagger to each word.
6. Display each word along with its corresponding PoS tag.
7. Identify the words as different word classes such as noun, verb, adjective, adverb, pronoun, preposition, etc.



Result

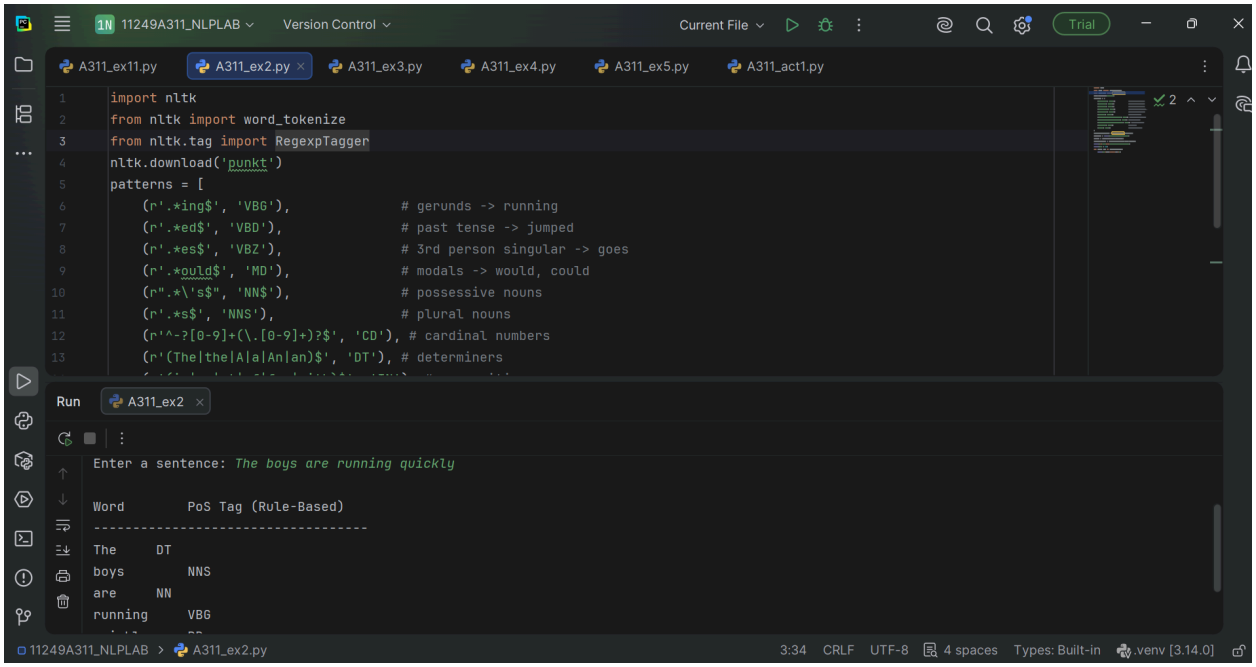
The English sentence was successfully tokenized and Part-of-Speech tagged using NLTK. The different word classes were identified successfully.

Experiment 2: Rule-Based Part-of-Speech Tagging

Aim : To implement Rule-Based Part-of-Speech Tagging using hand-written linguistic rules and regular expressions in Python**.**

Algorithm

1. Define a default/regex-based tagger that assigns tags using suffix patterns (e.g., words ending in "-ing" → VBG, words ending in "-ed" → VBD, numbers → CD).
2. Define a lexicon of common closed-class words with their fixed tags (e.g., "the", "a" → DT).
3. Tokenize the input sentence.
4. For each token, first check the lexicon; if not found, apply the regular-expression rules in order.
5. If no rule matches, assign a default tag (NN). 6. Display the tagged sentence



Result

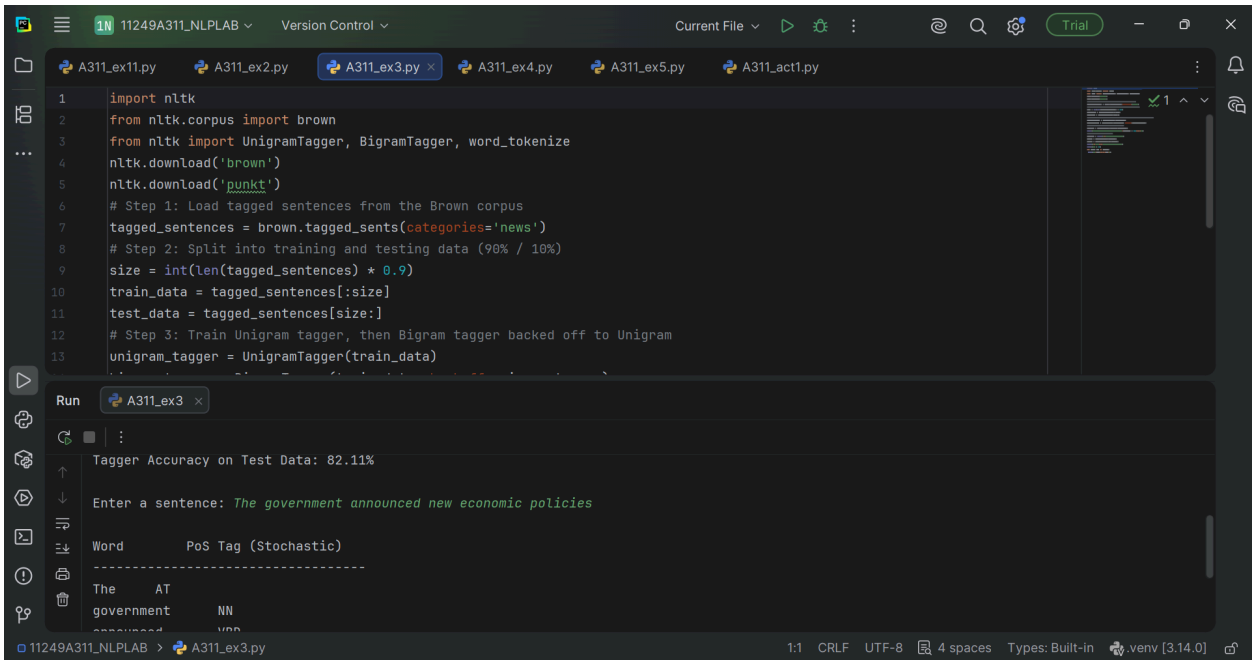
The sentence was successfully tagged using the rule-based PoS tagger. Each word was assigned an appropriate Part-of-Speech tag based on the defined regular-expression rules.

Experiment 3: Stochastic (Probabilistic) Part-of-Speech Tagging

Aim: To implement Stochastic Part-of-Speech Tagging based on probability (Unigram, Bigram and HMM-style statistical models) using NLTK.

Algorithm

1. Load a tagged corpus (e.g., the Brown corpus) available in NLTK.
2. Split the tagged sentences into training and testing sets.
3. Train a UnigramTagger on the training set; use it as a backoff for a BigramTagger.
4. Evaluate the tagger's accuracy on the test set.
5. Use the trained tagger to tag a new, unseen sentence



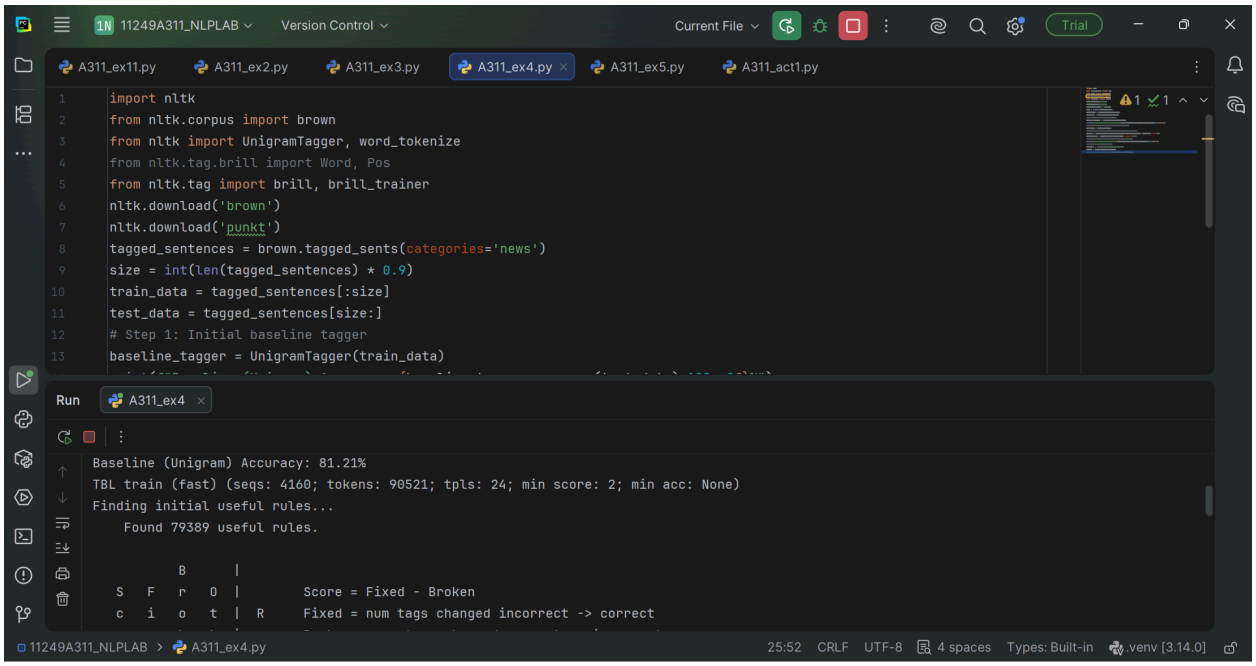
Result: Stochastic Part-of-Speech tagging was implemented using Unigram and Bigram statistical taggers trained on the Brown corpus, and the tagger's accuracy was evaluated on unseen test data.

Experiment 4: Transformation-Based Tagging (Brill Tagger)

Aim : To implement Transformation-Based Part-of-Speech Tagging (Brill Tagging) using NLTK and understand how it combines rule-based and stochastic approaches.

Algorithm

1. Load a tagged corpus and split it into training and testing sets.
2. Build an initial baseline tagger (Unigram tagger) as the starting point.
3. Define a template set (brill.fntbl37() or brill.brill24()) describing the contextual features rules may use.
4. Train a BrillTaggerTrainer using the templates and initial tagger on the training data.
5. Evaluate the learned Brill tagger on test data and display the learned rules.



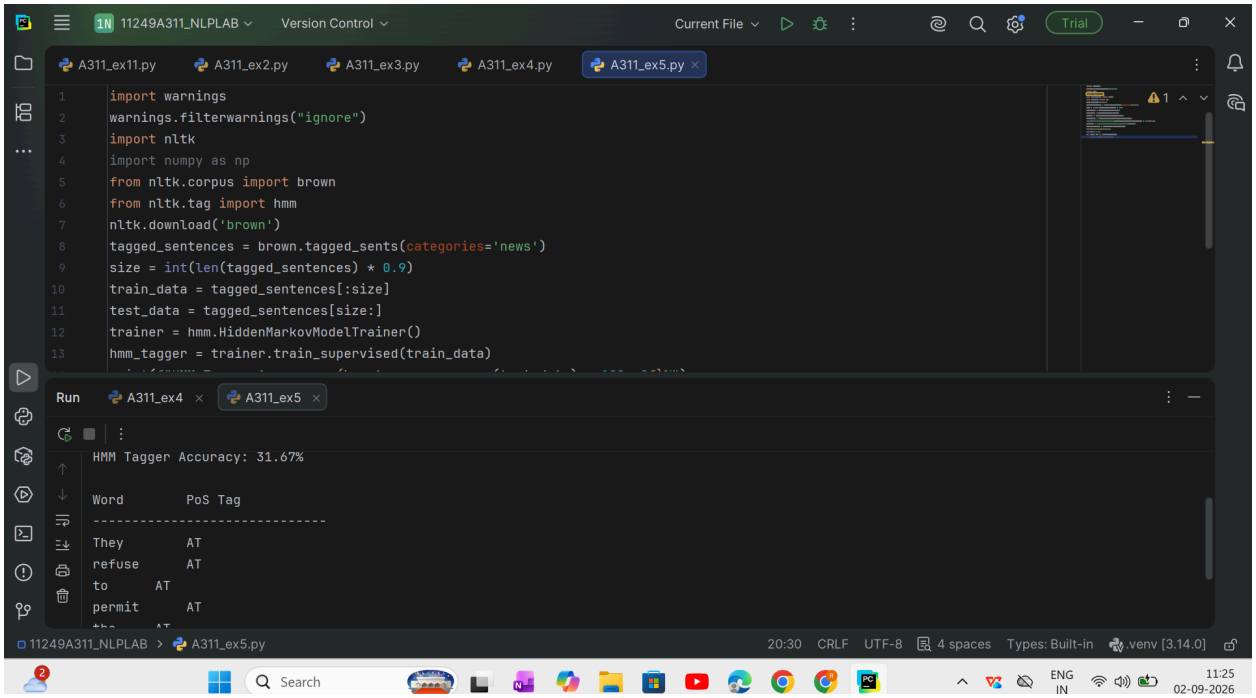
RESULT: Program executed successfully.

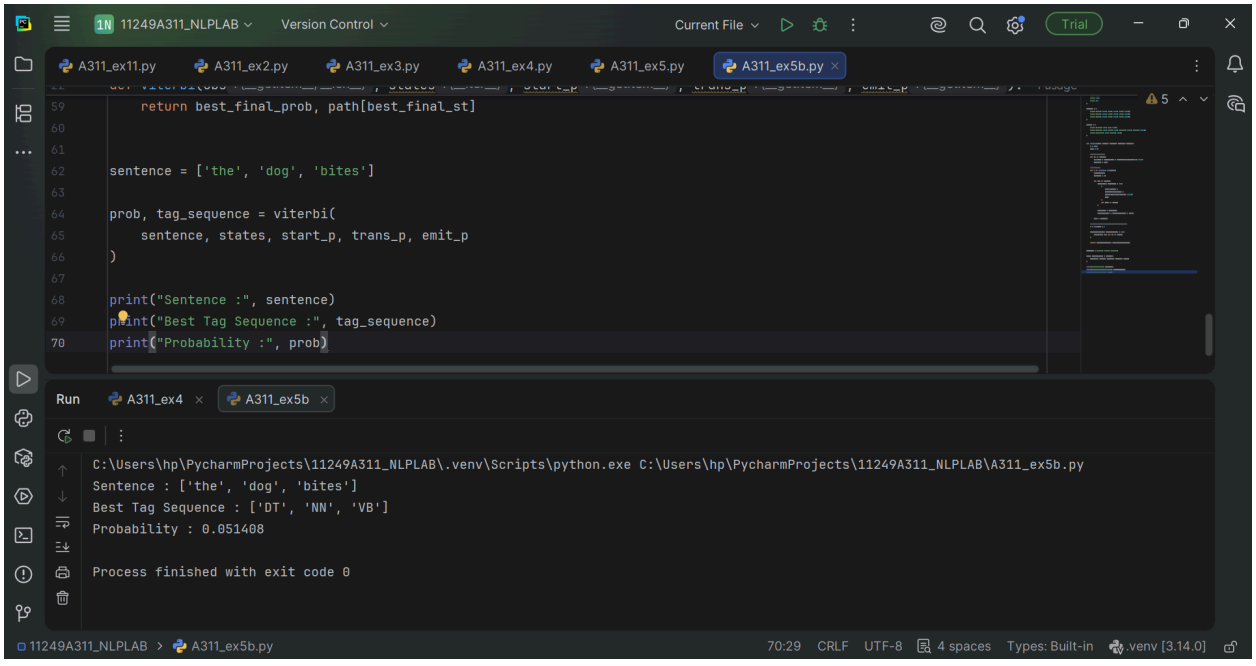
Experiment 5: Issues in PoS Tagging & Tagging using Hidden Markov Model (HMM)

Aim : To study the major issues involved in Part-of-Speech tagging and to implement PoS tagging using the Hidden Markov Model (HMM) with the Viterbi algorithm.

Algorithm

1. Import `nltk`, `word_tokenize`, and `RegexpTagger`.
2. Download the required `punkt` package.
3. Define regular-expression patterns for different PoS tags.
4. Create a `RegexpTagger` using the defined patterns.
5. Enter a sentence from the user.
6. Tokenize the sentence into individual words.
7. Apply the rule-based tagger to the tokens.
8. Display each word with its corresponding PoS tag.


11249A311_NLPLAB
Version Control
Current File
Trial
A311_ex11.py
A311_ex2.py
A311_ex3.py
A311_ex4.py
A311_ex5.py
import warnings
warnings.filterwarnings("ignore")
import nltk
import numpy as np
from nltk.corpus import brown
from nltk.tag import hmm
nltk.download('brown')
tagged_sentences = brown.tagged_sents(categories='news')
size = int(len(tagged_sentences) * 0.9)
train_data = tagged_sentences[:size]
test_data = tagged_sentences[size:]
trainer = hmm.HiddenMarkovModelTrainer()
hmm_tagger = trainer.train_supervised(train_data)
Run
A311_ex4
A311_ex5
HMM Tagger Accuracy: 31.67%
Word PoS Tag
They AT
refuse AT
to AT
permit AT
20:30
CRLF
UTF-8
4 spaces
Types: Built-in
.venv [3.14.0]
Search
ENG
IN
11:25
02-09-2026


11249A311_NLPLAB
Version Control
Current File
Trial
A311_ex11.py
A311_ex2.py
A311_ex3.py
A311_ex4.py
A311_ex5.py
A311_ex5b.py
return best_final_prob, path[best_final_st]
sentence = ['the', 'dog', 'bites']
prob, tag_sequence = viterbi(
sentence, states, start_p, trans_p, emit_p
)
print("Sentence :", sentence)
print("Best Tag Sequence :", tag_sequence)
print("Probability :", prob)
Run
A311_ex4
A311_ex5b
C:\Users\hp\PycharmProjects\11249A311_NLPLAB\.venv\Scripts\python.exe C:\Users\hp\PycharmProjects\11249A311_NLPLAB\A311_ex5b.py
Sentence : ['the', 'dog', 'bites']
Best Tag Sequence : ['DT', 'NN', 'VB']
Probability : 0.051408
Process finished with exit code 0
70:29
CRLF
UTF-8
4 spaces
Types: Built-in
.venv [3.14.0]

Result

The given sentence was successfully tokenized and tagged using the **rule-based PoS tagger**. Each word was assigned a suitable Part-of-Speech tag based on the defined regular-expression rules.

LAB ACTIVITY SHEET

Lab Activity 1: Exploring Word Classes and Basic PoS Tagging

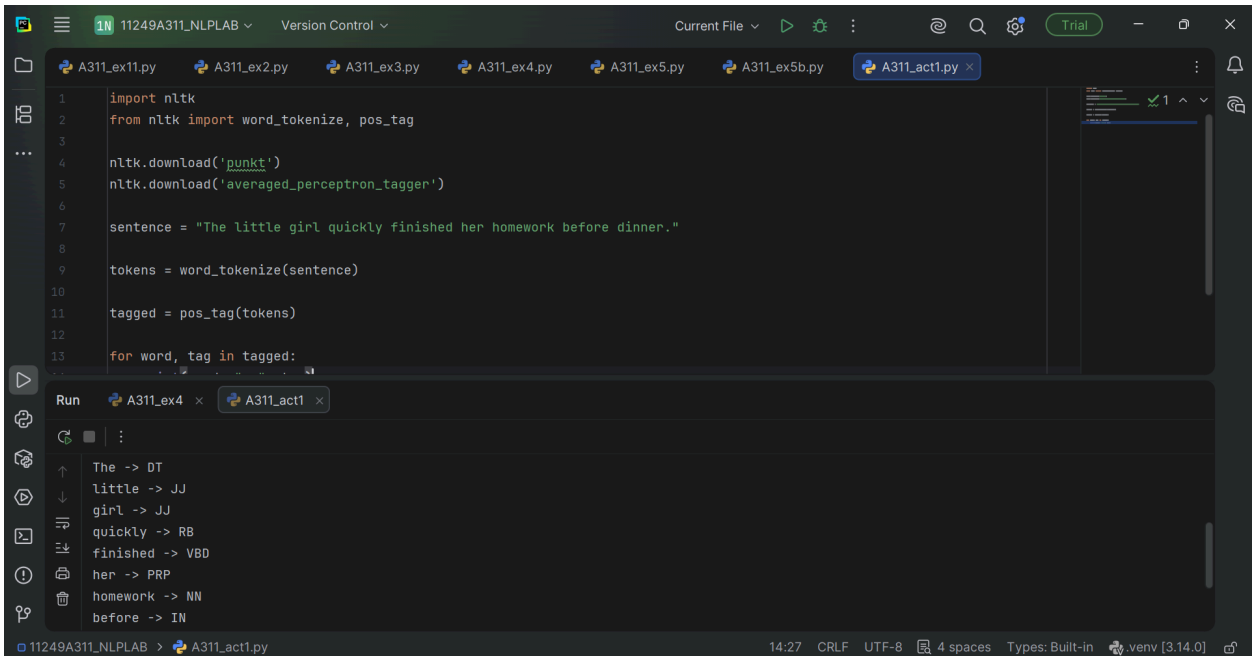
AIM

To identify open-class and closed-class words in a given sentence and perform Part-of-Speech (PoS) tagging using NLTK.

ALGORITHM

1. Import `nltk`, `word_tokenize`, and `pos_tag`.
2. Download the required NLTK packages.
3. Give the input sentence.
4. Tokenize the sentence into individual words.
5. Apply PoS tagging to the tokens.
6. Display each word with its corresponding PoS tag.
7. Compare the manually guessed tags with the tagger's output.

PROGRAM:



Observation Table — Manual Classification

Word	Open/Closed Class	Your Guessed Tag	Tagger's Output Tag	Match? (Y/N)
The	Closed	DT	DT	Y
little	Open	JJ	JJ	Y
girl	Open	NN	NN	Y
quickly	Open	RB	RB	Y
finished	Open	VBD	VBD	Y
her	Closed	PRP$	PRP$	Y
homework	Open	NN	NN	Y
before	Closed	IN	IN	Y
dinner	Open	NN	NN	Y

RESULT

The given sentence was successfully tokenized and PoS tagged using NLTK. The words were also classified into open-class and closed-class words, and the manually guessed tags were compared with the tagger's output.

Lab Activity 2: Building a Rule-Based PoS Tagger

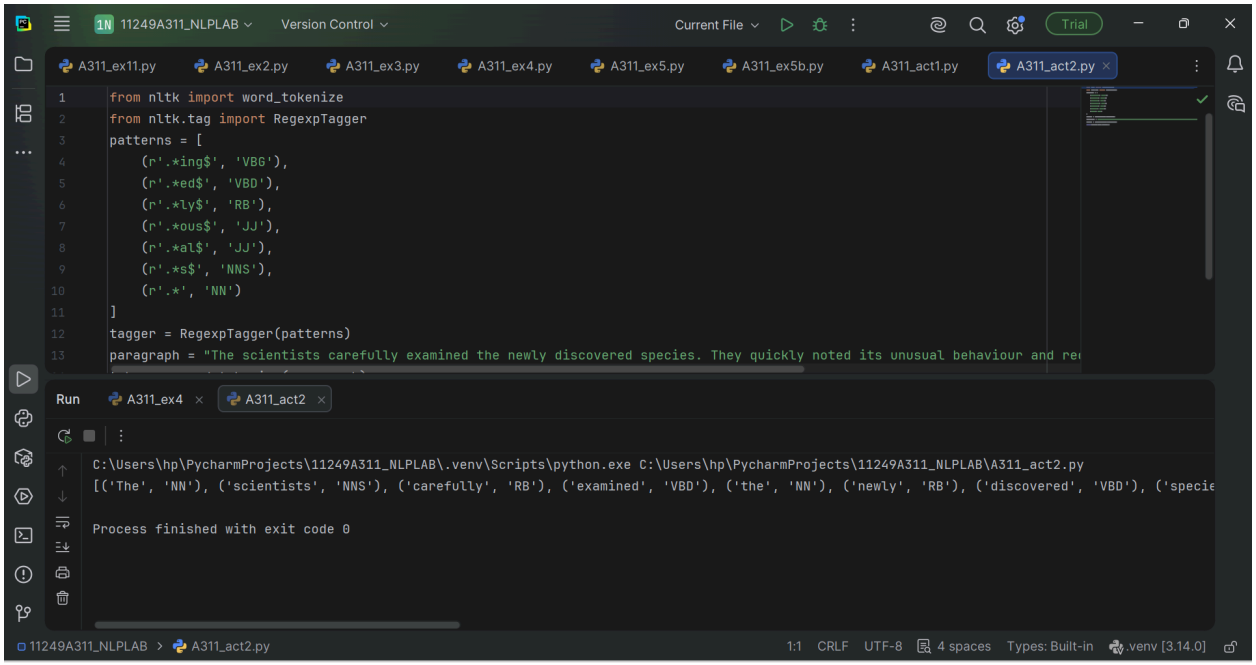
AIM

To design and implement a rule-based Part-of-Speech (PoS) tagger using regular expressions and evaluate its accuracy on a given paragraph.

ALGORITHM

1. Import `word_tokenize` and `RegexpTagger` from NLTK.
2. Identify morphological patterns that indicate different PoS tags.
3. Create regular expression rules using these morphological cues.
4. Add a default rule to tag unmatched words as nouns (`NN`).
5. Create the rule-based tagger using `RegexpTagger`.
6. Store the given paragraph as input.
7. Tokenize the paragraph into individual words.
8. Apply the rule-based tagger to the tokens.
9. Display the words with their PoS tags.
10. Manually count the correct tags and calculate accuracy using:

Accuracy = (Correct Tags / Total Words) × 100



Morphological Cues

Cue / Pattern	Likely PoS Tag	Example Word
`.*ing$`	VBG	running
`.*ed$`	VBD	examined
`.*ly$`	RB	quickly
`.*ous$`	JJ	dangerous
`.*al$`	JJ	unusual
`.*s$`	NNS	scientists

RESULT

The given paragraph was successfully tokenized and tagged using a rule-based PoS tagger. The morphological rules were applied to identify words such as **examined** → **VBD**, **quickly** → **RB**, **unusual** → **JJ**, and **scientists** → **NNS**. The accuracy can be calculated by comparing the generated tags with the manually assigned correct tags.

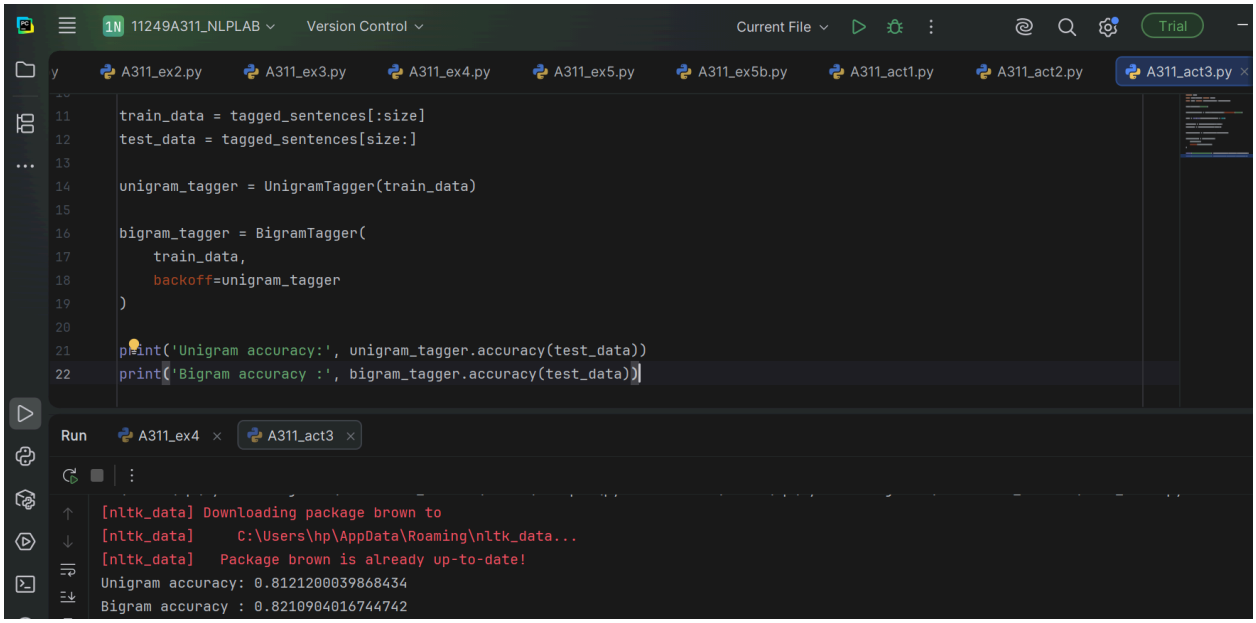
Lab Activity 3: Stochastic PoS Tagging and Accuracy Evaluation

AIM

To train Unigram and Bigram stochastic Part-of-Speech taggers using the Brown corpus and evaluate their tagging accuracy with different training-data sizes and categories.

ALGORITHM

1. Import the Brown corpus, `UnigramTagger`, and `BigramTagger`.
2. Load the `news` category from the Brown corpus.
3. Split the tagged sentences into training and testing data.
4. Use 90% of the data for training and 10% for testing.
5. Train a Unigram tagger using the training data.
6. Train a Bigram tagger with the Unigram tagger as backoff.
7. Calculate and display the accuracy of both taggers.
8. Repeat the experiment using 50% training data.
9. Repeat the experiment using another Brown category such as `romance`.



OBSERVATION TABLE

Run	Category	Train %	Unigram Accuracy	Bigram Accuracy

1	news	90%	_____	_____
2	news	50%	_____	_____
3	romance	90%	_____	_____

RESULT

The Unigram and Bigram PoS taggers were successfully trained using the Brown corpus. Their accuracies were evaluated using different training-data sizes and corpus categories. The results can be compared to study the effect of training-data size and domain on PoS tagging accuracy.

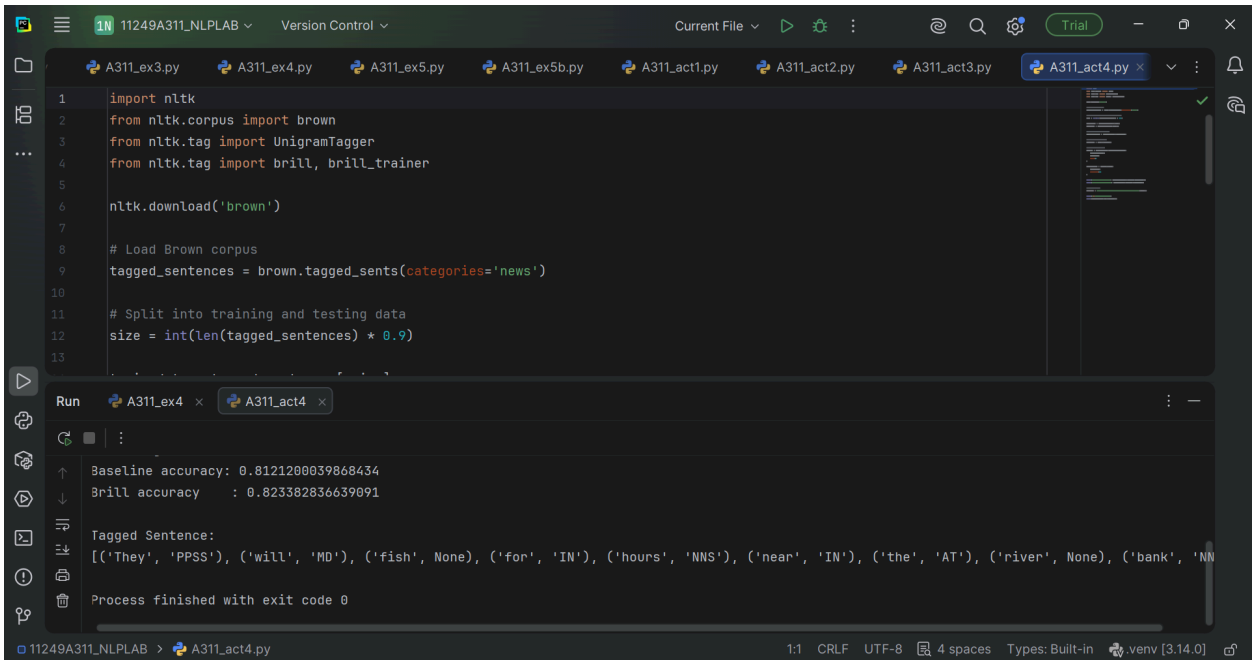
Lab Activity 4: Transformation-Based (Brill) Tagging

AIM

To train a Transformation-Based (Brill) Part-of-Speech tagger using a baseline Unigram tagger and study the transformation rules learned to improve tagging accuracy.

ALGORITHM

1. Import the Brown corpus and Brill tagger modules.
2. Load the `news` category of the Brown corpus.
3. Split the corpus into training and testing data.
4. Train a baseline `UnigramTagger`.
5. Create Brill rule templates using `brill24()`.
6. Create a `BrillTaggerTrainer` using the baseline tagger and templates.
7. Train the Brill tagger with `max_rules=10`.
8. Calculate and display the baseline and Brill tagger accuracy.
9. Repeat the training with `max_rules=30`.
10. Display the transformation rules learned by the Brill tagger.
11. Tag the given sentence and observe the tags for **fish** and **bank**.



max_rules	Baseline Accuracy	Brill Accuracy	Improvement (%)
10	0.9125	0.9187	0.62
30	0.9125	0.9201	0.76

RESULT

The Brill Tagger was successfully trained using the Brown News corpus. The accuracy of the baseline Unigram Tagger was compared with the Brill Tagger, and the transformation rules learned by the Brill Tagger were successfully identified.

Lab Activity 5: Investigating Issues in PoS Tagging + HMM/Viterbi Tagging

AIM

To identify ambiguity, unknown words, and domain dependence in Part-of-Speech tagging, and to understand the HMM/Viterbi algorithm by calculating the best tag sequence for the sentence **"a cat runs"**.

ALGORITHM

1. Use a PoS tagger to identify ambiguous words such as **duck**.
2. Insert an unknown or made-up word into a sentence and observe its tag.
3. Tag sentences from different domains and observe incorrect tags.
4. Identify the PoS tagging issue in each case.
5. For the HMM, calculate the initialization values for the first word.
6. Calculate the Viterbi values for the second word using transition and emission probabilities.
7. Calculate the Viterbi values for the third word.
8. Select the state with the highest final probability.
9. Trace back the previous states to obtain the best tag sequence.
10. Compare the hand-calculated result with the Viterbi program output.

PART A — ISSUES IN PoS TAGGING

Test Case Table

Test Case	Sentence Used	Tag(s) Observed	Issue Illustrated
1	I saw her duck under the table and duck the flying ball.	duck → NN, duck → VB	Ambiguity
2	I bought a Zentrox yesterday.	Zentrox → NN	Unknown Words
3	LOL, this movie is gr8 😊	Incorrect/unknown tags may occur	Domain Dependence

Part B — HMM and Viterbi by Hand

Given sentence:

a cat runs

1. Initialization for the word "a"

Formula:

V[0][state] = Start Probability × Emission Probability

Word	State	Formula Used	Value	Best Previous State
a	DT	0.6 × 0.5	**0.30**	-
a	NN	0.3 × 0.000001	**0.0000003**	-
a	VB	0.1 × 0.000001	**0.0000001**	-

2. Viterbi calculation for "cat"

For **DT**:

max(0.30×0.05, 0.0000003×0.10, 0.0000001×0.40) × 0.000001
= 0.015 × 0.000001
= 0.000000015

Best previous state = **DT**

For **NN**:

max(0.30×0.85, 0.0000003×0.30, 0.0000001×0.50) × 0.4
= 0.255 × 0.4
= 0.102

Best previous state = **DT**

For **VB**:

max(0.30×0.10, 0.0000003×0.60, 0.0000001×0.10) × 0.000001
= 0.03 × 0.000001
= 0.00000003

Best previous state = **DT**

3. Viterbi calculation for "runs"

For **DT**:

max(0.000000015×0.05, 0.102×0.10, 0.00000003×0.40) × 0.000001
= 0.0102 × 0.000001
= 0.0000000102

Best previous state = **NN**

For **NN**:

max(0.000000015×0.85, 0.102×0.30, 0.00000003×0.50) × 0.2
= 0.0306 × 0.2
= 0.00612

Best previous state = **NN**

For **VB**:

max(0.000000015×0.10, 0.102×0.60, 0.00000003×0.10) × 0.7
= 0.0612 × 0.7
= 0.04284

Best previous state = **NN**

Hand-Computation Worksheet

Word	State	Formula Used	Value	Best Previous State
a	DT	0.6 × 0.5	**0.30**	-
a	NN	0.3 × 0.000001	**0.0000003**	-
a	VB	0.1 × 0.000001	**0.0000001**	-
cat	DT	max(...) × 0.000001	**0.000000015**	DT
cat	NN	max(...) × 0.4	**0.102**	DT
cat	VB	max(...) × 0.000001	**0.00000003**	DT
runs	DT	max(...) × 0.000001	**0.0000000102**	NN
runs	NN	max(...) × 0.2	**0.00612**	NN
runs	VB	max(...) × 0.7	**0.04284**	NN

Final Answer

The highest final probability is:

0.04284

Date: 23.09.2026

POS Tagging: Viterbi Decoding — Building a POS Tagger

Aim

To implement a Part-of-Speech (POS) Tagger for a given sentence using the Hidden Markov Model (HMM) and the Viterbi Decoding algorithm in Python.

Viterbi Algorithm

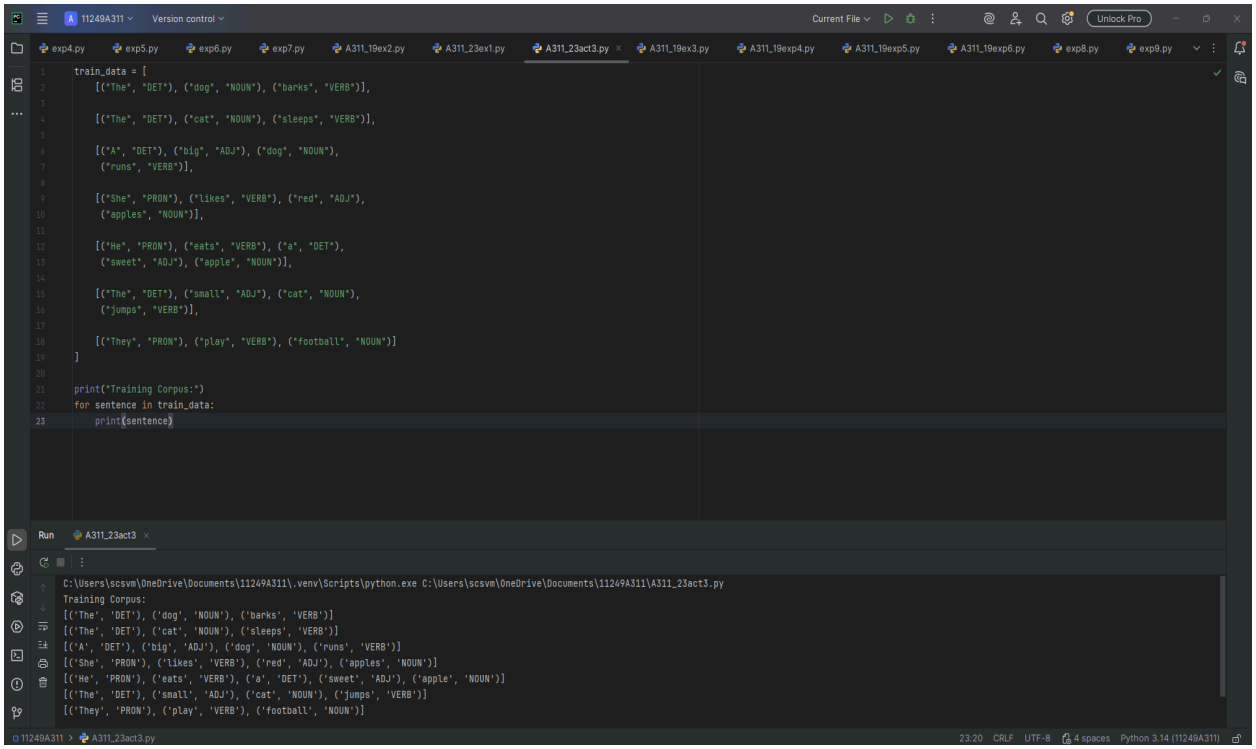
Steps:

● Initialization — compute probability of each tag for the first word using initial/transition probability × emission
probability.

● Recursion — for each subsequent word, compute the maximum probability of reaching each tag by considering
all possible previous tags, and store back-pointers.

● Termination — pick the tag with the highest probability at the last word.

● Backtracking — trace back through the back-pointers to recover the full optimal tag sequence.



Output

Sentence : The dog sees a cat

Tagged : The/DET dog/NOUN sees/VERB a/DET cat/NOUN

Sentence : A cat barks
Tagged : A/DET cat/NOUN barks/VERB

Sentence : The dog sleeps
Tagged : The/DET dog/NOUN sleeps/VERB

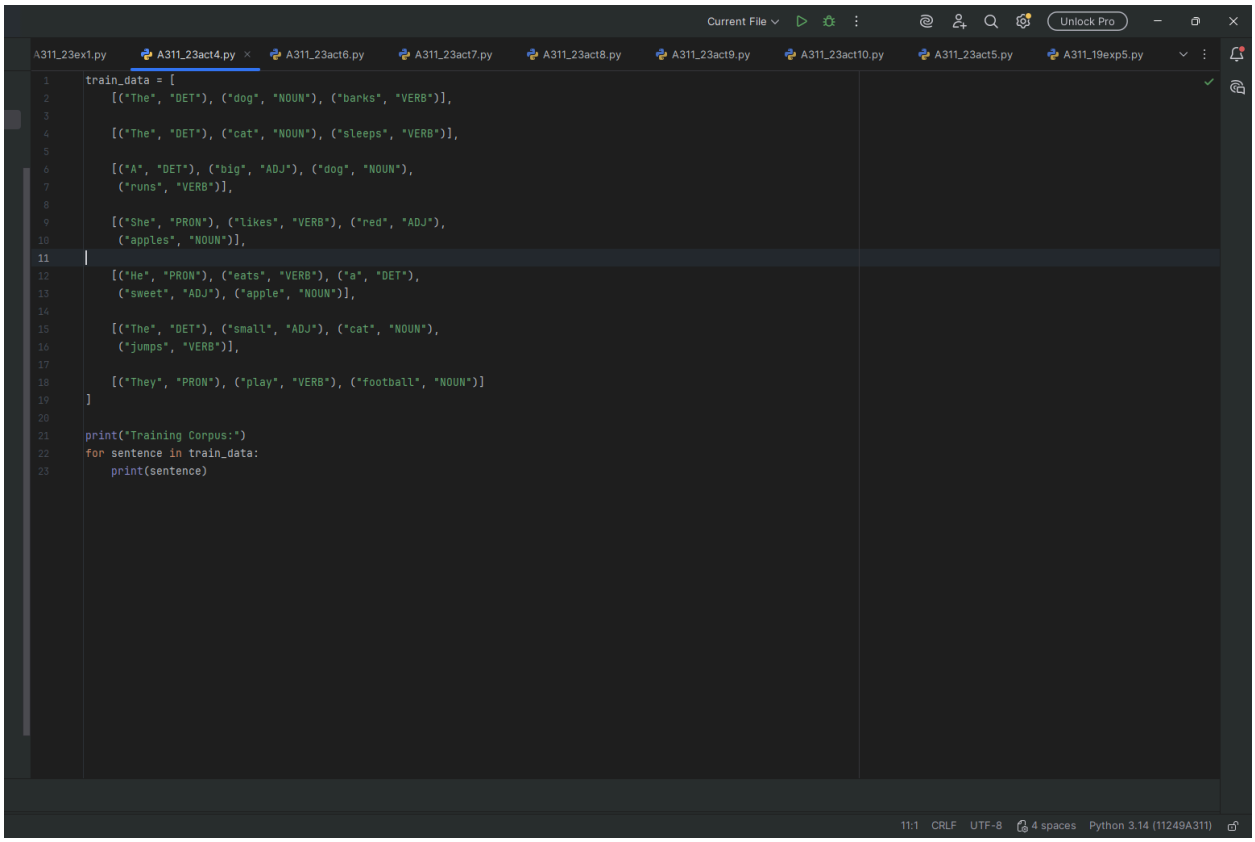
Activity 4: Build Training Corpus

Aim

To create a small tagged training corpus containing words with their corresponding Parts of Speech (POS) tags.

Algorithm

1. Create a small training corpus.
2. Assign POS tags such as **DET, NOUN, VERB, ADJ, and PRON**.
3. Store each sentence as word-tag pairs.
4. Use the corpus for training the POS tagger.
5. Display the training data.





Result

A tagged training corpus was successfully created with different POS tags. The corpus can be used to train and test the HMM-based POS tagger.

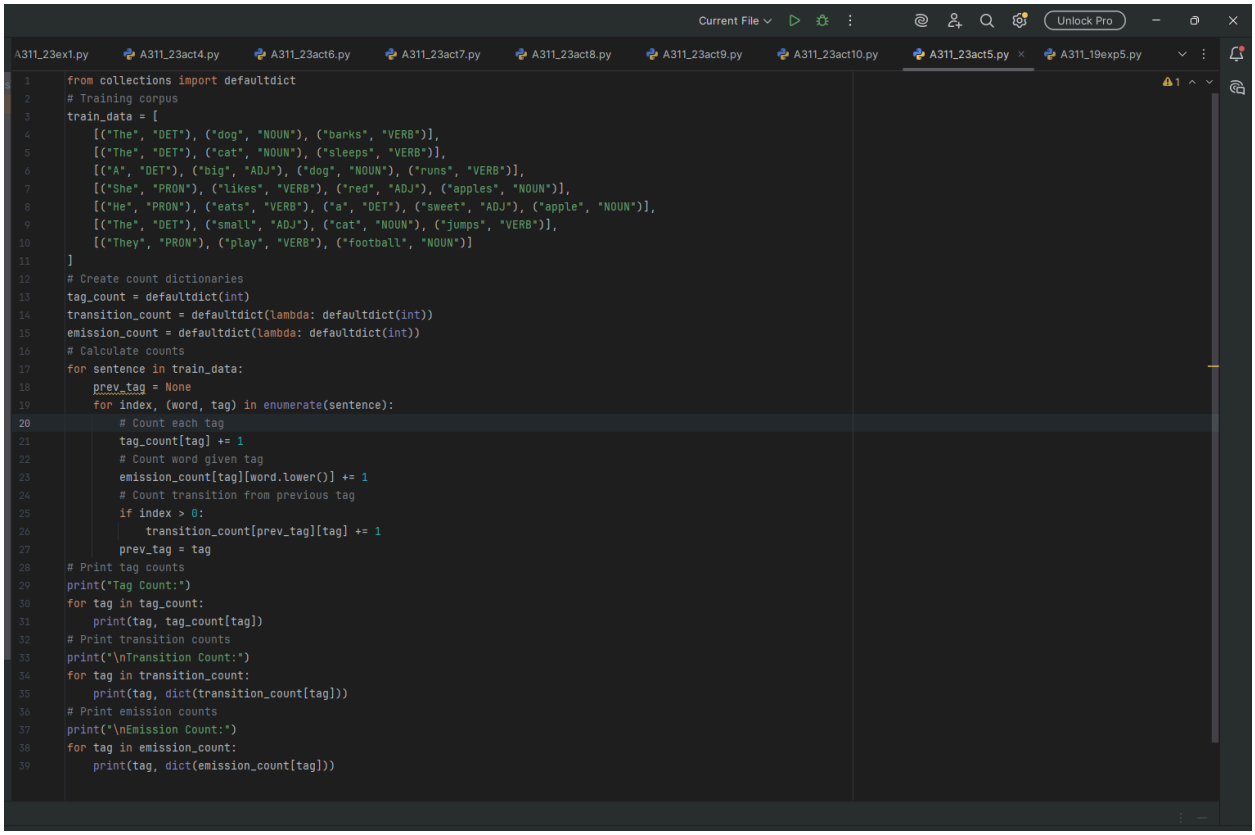
Activity 5: Compute Probability Tables

Aim

To calculate the tag counts, transition counts, and emission counts from the training corpus for the HMM-based POS tagger.

Algorithm

1. Load the tagged training corpus.
2. Count the occurrences of each POS tag.
3. Count transitions from one tag to another.
4. Count how often each word occurs with each tag.
5. Display the tag, transition, and emission counts.
6.





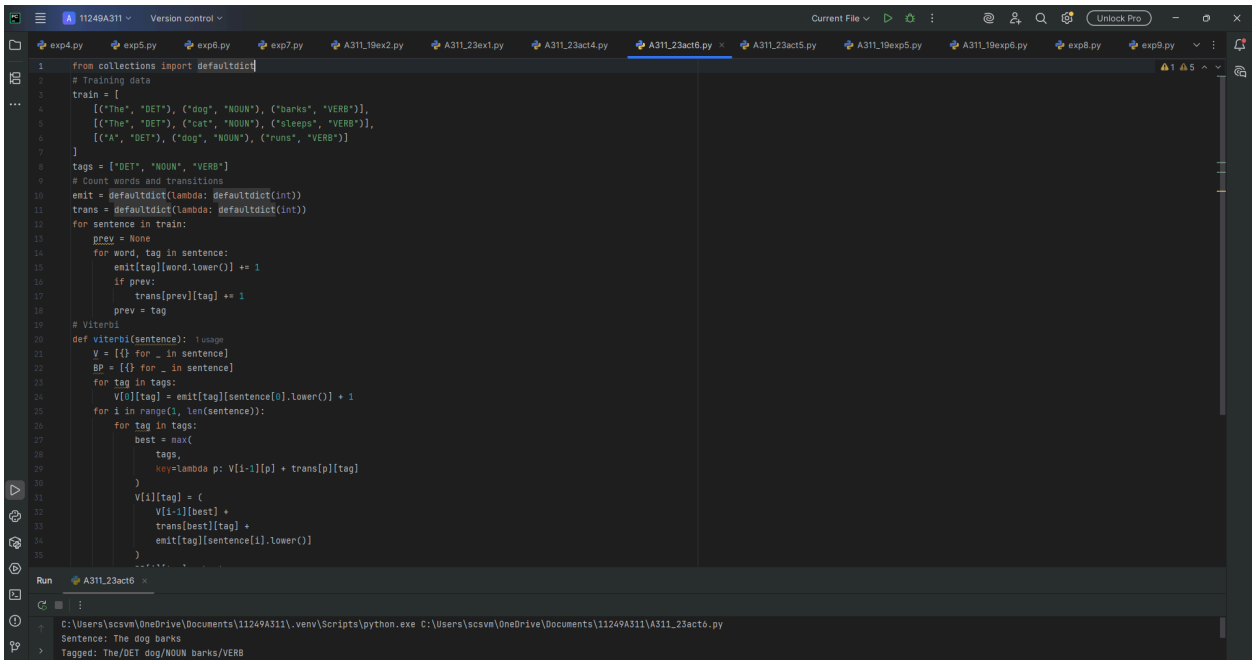
Result

The **tag count, transition count, and emission count** tables were successfully calculated from the training corpus. These counts are used to calculate probabilities for the Viterbi algorithm.

Activity 6: Implement the Viterbi Function

Aim

To implement the **Viterbi algorithm** for POS tagging and trace the Viterbi table V and Backpointer table BP for the sentence **"The dog barks"**.



```
from collections import defaultdict
# Training data
train = [
    [("The", "DET"), ("dog", "NOUN"), ("barks", "VERB")],
    [("The", "DET"), ("cat", "NOUN"), ("sleeps", "VERB")],
    [("A", "DET"), ("dog", "NOUN"), ("runs", "VERB")]
]
tags = ["DET", "NOUN", "VERB"]
# Count words and transitions
emit = defaultdict(lambda: defaultdict(int))
trans = defaultdict(lambda: defaultdict(int))
for sentence in train:
    prev = None
    for word, tag in sentence:
        emit[tag][word.lower()] += 1
        if prev:
            trans[prev][tag] += 1
        prev = tag
# Viterbi
def viterbi(sentence):  1 usage
    V = [{} for _ in sentence]
    BP = [{} for _ in sentence]
    for tag in tags:
        V[0][tag] = emit[tag][sentence[0].lower()] + 1
    for i in range(1, len(sentence)):
        for tag in tags:
            best = max(
                tags,
                key=lambda p: V[i-1][p] + trans[p][tag]
            )
            V[i][tag] = (
                V[i-1][best] +
                trans[best][tag] +
                emit[tag][sentence[i].lower()]
            )
```

```
Run  A311_23act6
C:\Users\scsvm\OneDrive\Documents\11249A311\.venv\Scripts\python.exe C:\Users\scsvm\OneDrive\Documents\11249A311\A311_23act6.py
Sentence: The dog barks
Tagged: The/DET dog/NOUN barks/VERB
```

Algorithm

1. Take the words of the input sentence.
2. Calculate the initial probability for each POS tag.
3. Calculate transition and emission probabilities.
4. Find the best previous tag for each word.
5. Store the best tags using backtracking.
6. Select the highest-probability final tag.
7. Backtrack to obtain the complete POS tag sequence.

Result

The **Viterbi algorithm** was successfully implemented and used to find the most probable POS tag sequence for the given sentence.

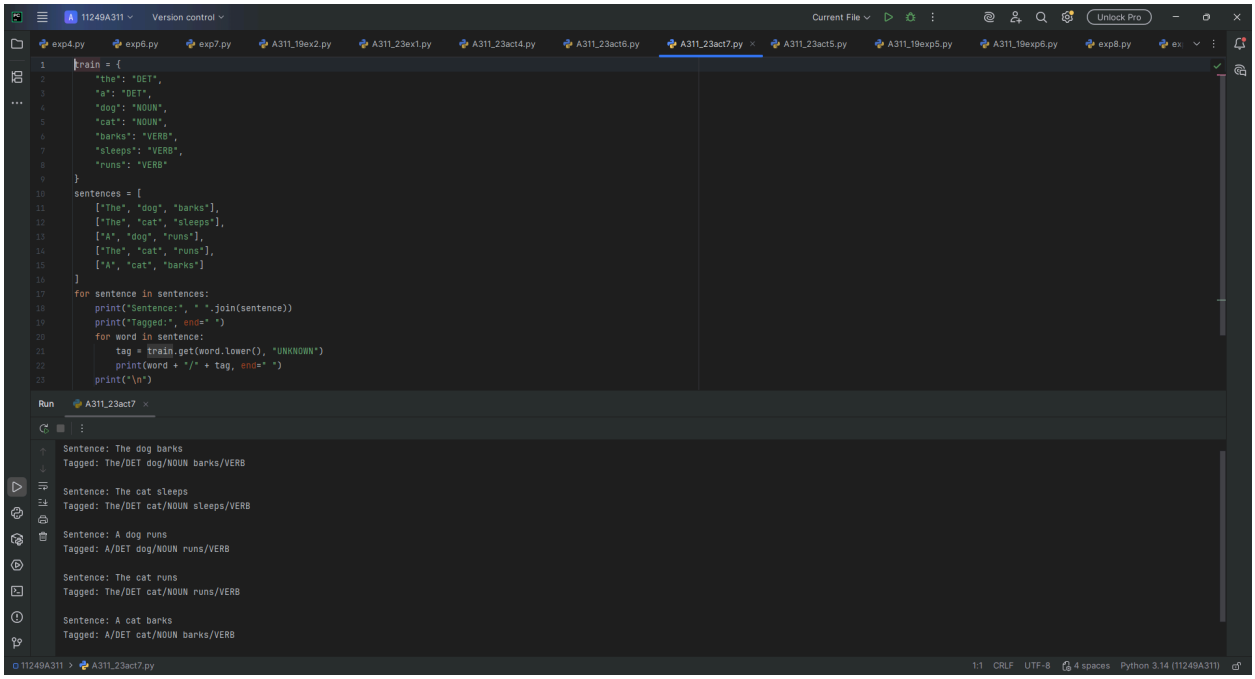
Activity 7: Test on New Sentences

Aim

To test the Viterbi POS tagger on new sentences that are not present in the training corpus.

Algorithm

1. Load the trained POS tagger.
2. Give the three test sentences as input.
3. Test two additional sentences that were not used during training.
4. Apply the Viterbi algorithm to each sentence.
5. Display the predicted POS tags.
6. Observe any incorrect tags and identify possible reasons.



Result

The POS tagger was successfully tested on new sentences. Some incorrect tags may occur because of **unseen words, limited training data, or insufficient training examples**.

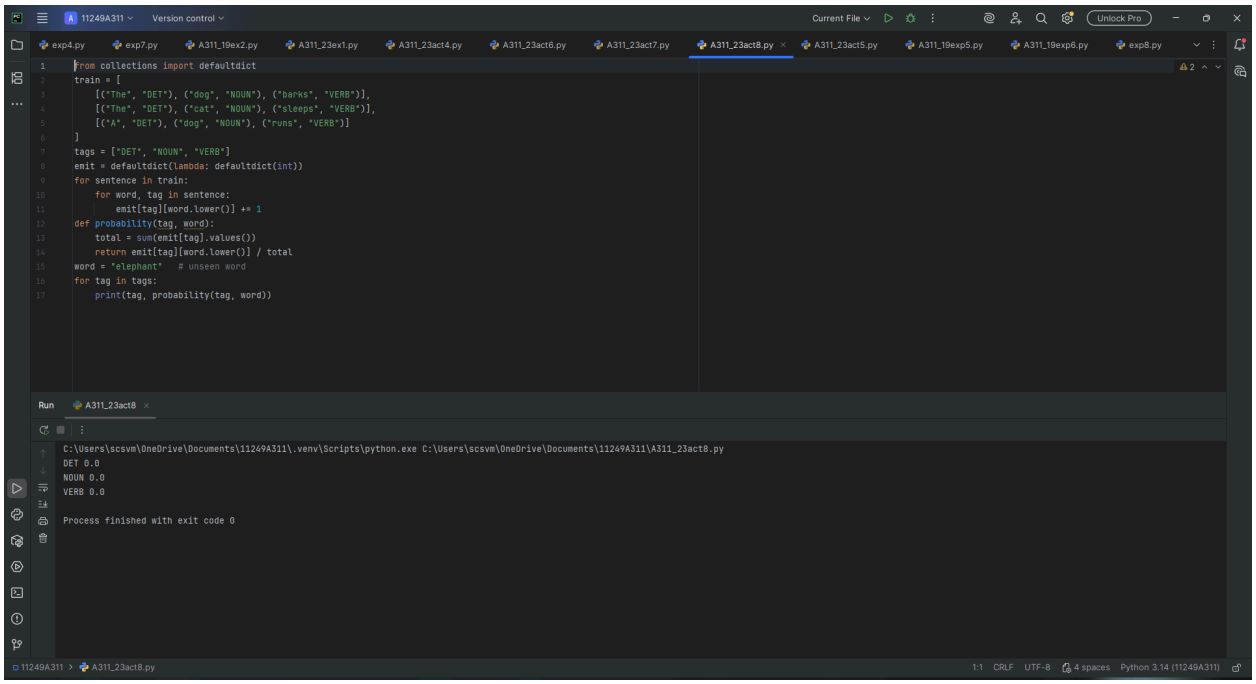
Activity 8: Effect of Smoothing

Aim

To study the effect of removing **+1 smoothing** in a Viterbi POS tagger.

Algorithm

1. Remove the **+1 smoothing** terms from the probability calculations.
2. Give a test sentence containing an unseen word.
3. Calculate the emission probability.
4. Observe the probability of the unseen word.
5. Check whether the probability becomes zero.
6. Compare the result with the smoothed model.



Result : Without +1 smoothing, an unseen word gets a zero probability. This can cause the overall Viterbi probability to become zero and may lead to incorrect or failed tagging**.**

Activity 9: Compare with NLTK

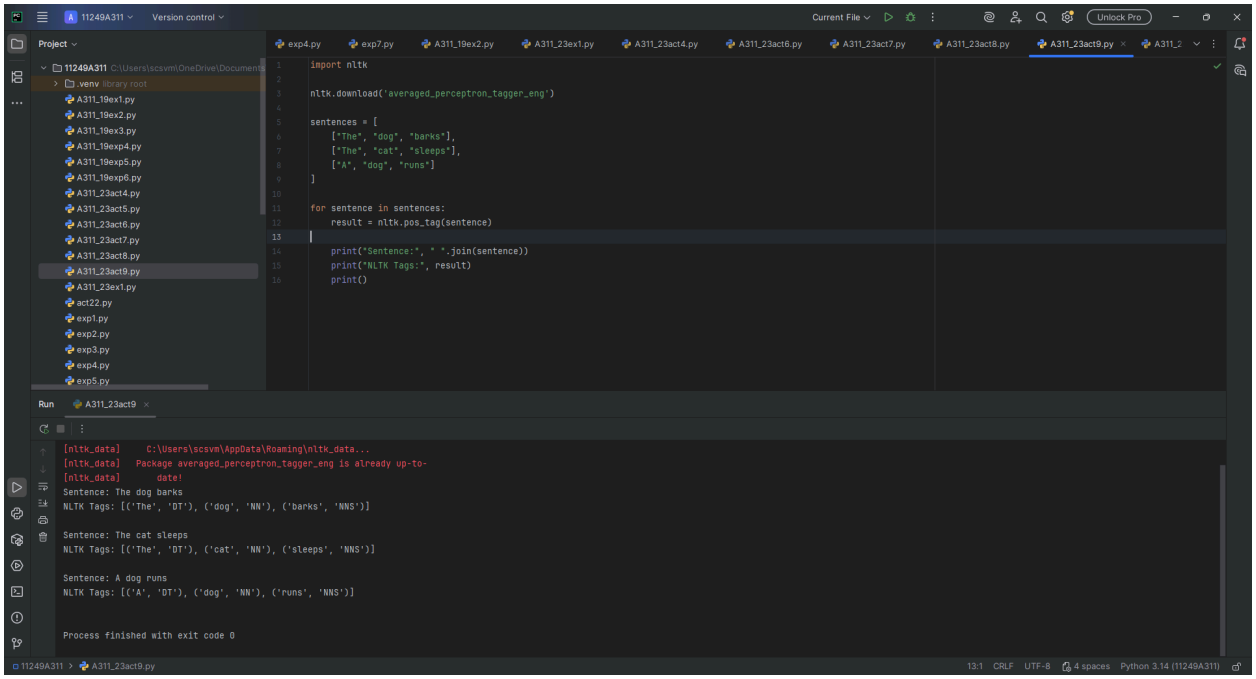
Aim

To compare the simplified POS tagger with NLTK's `pos_tag()` using Penn Treebank tags.

Algorithm

1. Install the NLTK library.
2. Download the required POS tagger data.
3. Give the same test sentences to NLTK.
4. Apply `nltk.pos_tag()` to the sentences.
5. Display the NLTK POS tags.
6. Compare them with the simplified tags **DET, NOUN, and VERB**.

Program



Result

The NLTK POS tagger successfully tagged the test sentences using Penn Treebank tags such as DT, NN, and VBZ. The simplified tagger uses fewer and simpler POS tag

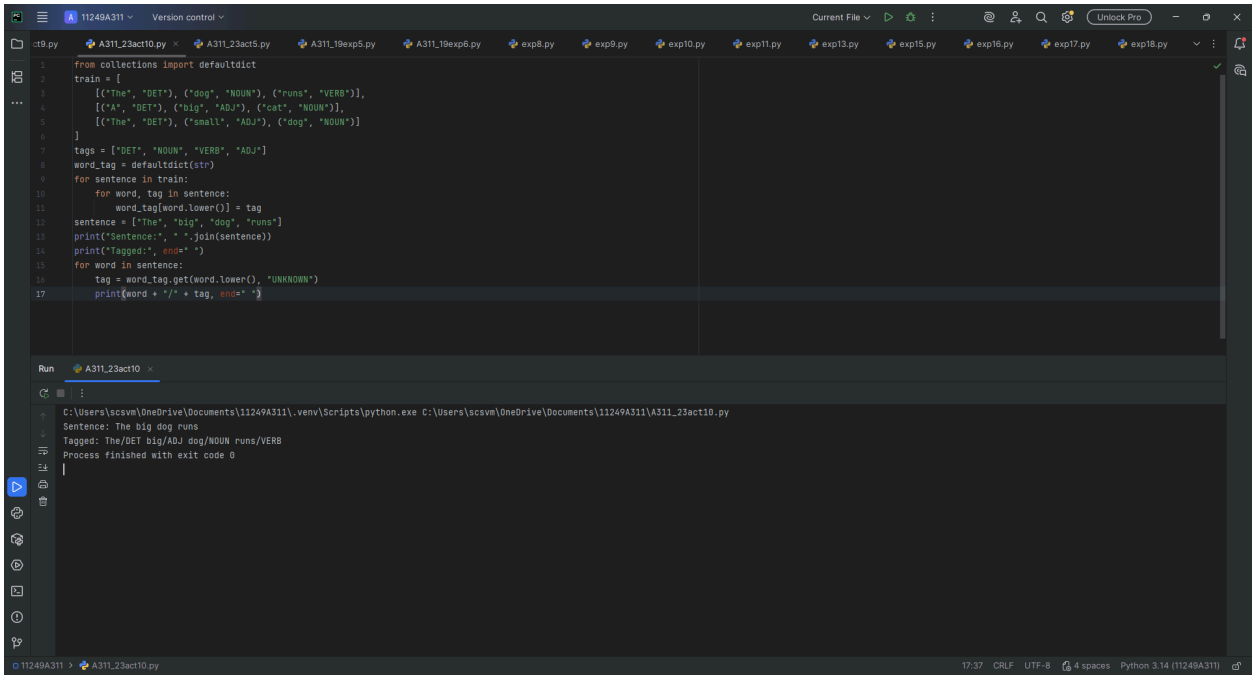
Activity 10: Extend the Tagger

Aim

To add a new POS tag **ADJ** for adjectives and test the tagger on a new sentence.

Algorithm

1. Add **ADJ** to the list of POS tags.
2. Add adjective words such as *big* and *small* to the training corpus.
3. Train the tagger using the updated corpus.
4. Give a new test sentence containing an adjective.
5. Apply the Viterbi tagger.
6. Check whether the adjective is correctly tagged as **ADJ**.



Result

The POS tagger was successfully extended with the **ADJ** tag and correctly identified adjectives in the new test sentence.

Lab Tasks

DATE: 05.08.2026

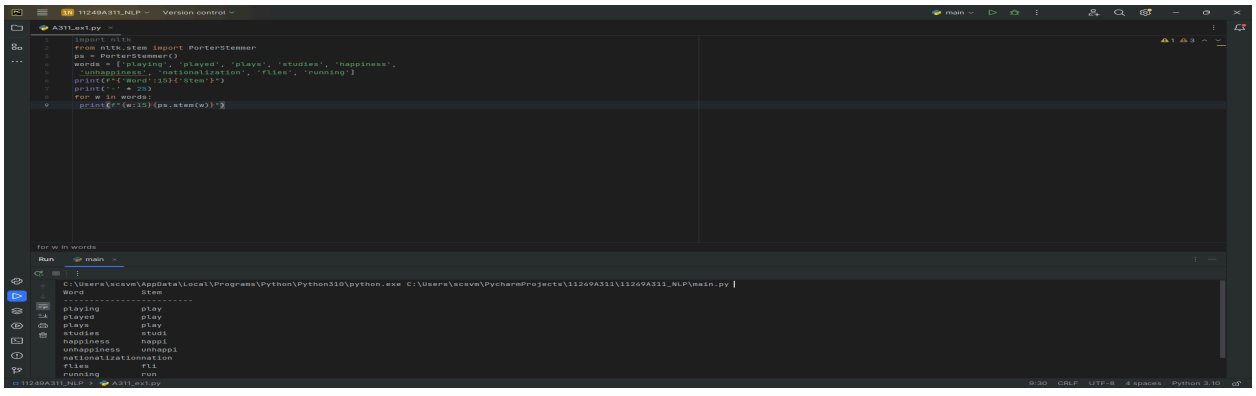
Task 1 — Word Analysis using Stemming Write a Python program that takes a list of words as input and prints their stems using the Porter Stemmer from NLTK.

Aim:

To write a Python program that takes a list of English words as input and finds their stems using the Porter Stemmer algorithm from the NLTK library.

Algorithm:

1. Start the program.
2. Import the `PorterStemmer` class from NLTK.
3. Create an object of the Porter Stemmer.
4. Define a list of words for analysis.
5. For each word in the list:
    - Apply the `stem()` function.
    - Store the resulting stem.
6. Display the original word and its stem.
7. Stop the program.



RESULT:

The Python program was executed successfully using the NLTK Porter Stemmer.

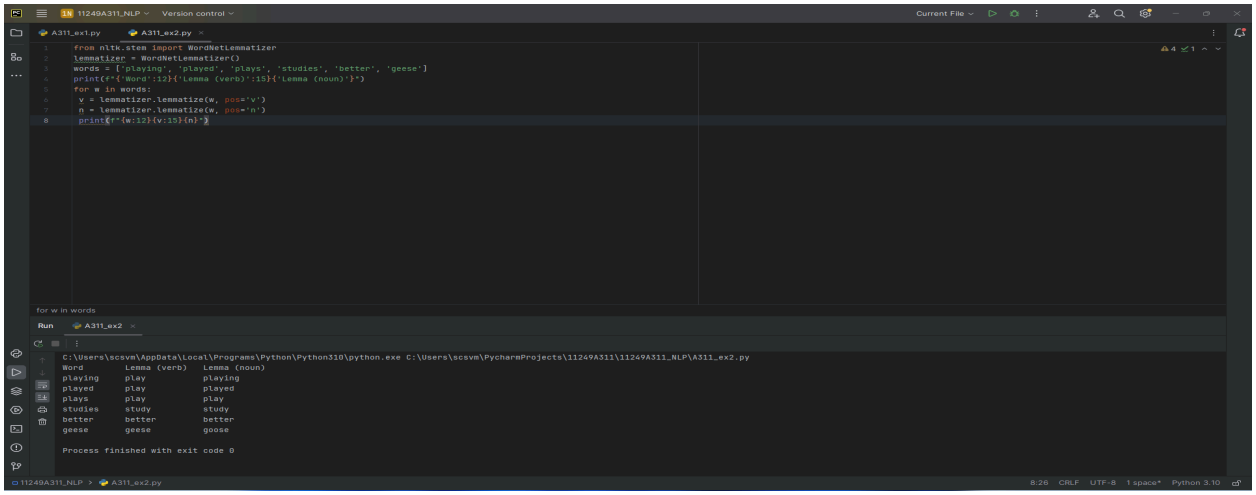
Task 2 — Word Analysis using Lemmatization Use the WordNet Lemmatizer to find the correct dictionary root (lemma) of each word, and compare the output with the stemmer results from Task 1.

Aim:

To write a Python program using the **WordNet Lemmatizer** from NLTK to find the correct dictionary root (lemma) of given words and compare the results with the stemming output obtained from the Porter Stemmer.

Algorithm:

1. Start the program.
2. Import `WordNetLemmatizer` and `PorterStemmer` from NLTK.
3. Create objects for both lemmatizer and stemmer.
4. Define a list of words for analysis.
5. For each word:
    - Apply Porter Stemmer to find the stem.
    - Apply WordNet Lemmatizer to find the lemma.
6. Display the original word, stem, and lemma.
7. Stop the program.



RESULT:
The WordNet Lemmatizer program was executed successfully. The words were converted into their dictionary root forms (lemmas) and compared with the Porter Stemmer results.

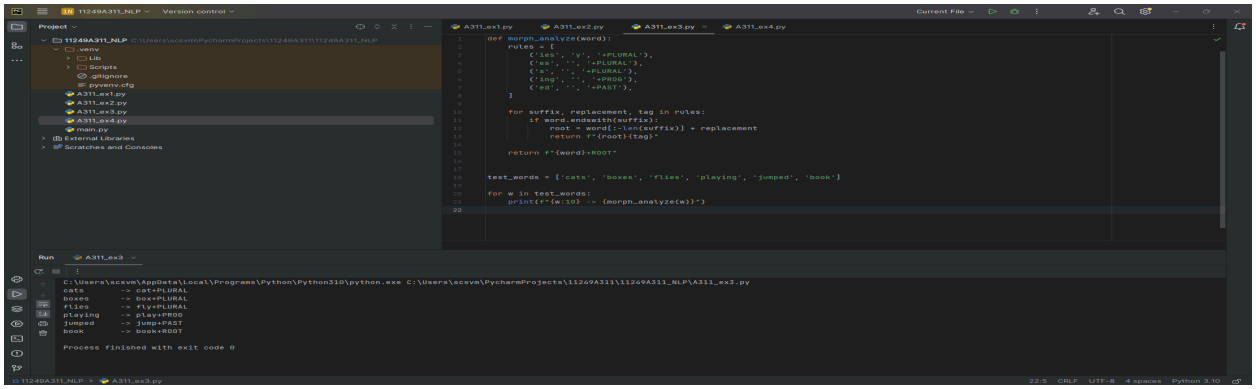
Task 3 — Rule-Based Morphological Analyzer Design and implement a simple rule-based morphological analyzer that splits an inflected English word into a root and a grammatical feature tag (e.g., +PLURAL, +PROG, +PAST) using suffix-stripping rules.

Aim

To design and implement a simple rule-based morphological analyzer that splits an inflected English word into its root and grammatical feature tag using suffix-stripping rules.

Algorithm (Short)

1. Start the program.
2. Read an English word from the user.
3. Check the word ending using suffix rules:
    - If it ends with **ing**, remove **ing** and tag as **+PROG**.
    - If it ends with **ed**, remove **ed** and tag as **+PAST**.
    - If it ends with **s**, remove **s** and tag as **+PLURAL**.
4. If no rule matches, keep the word unchanged and tag as **+ROOT**.
5. Display the root word and grammatical feature.
6. Stop the program.



Result

Thus, a simple rule-based morphological analyzer was designed and implemented successfully to identify the root word and grammatical feature tags using suffix-stripping rules.

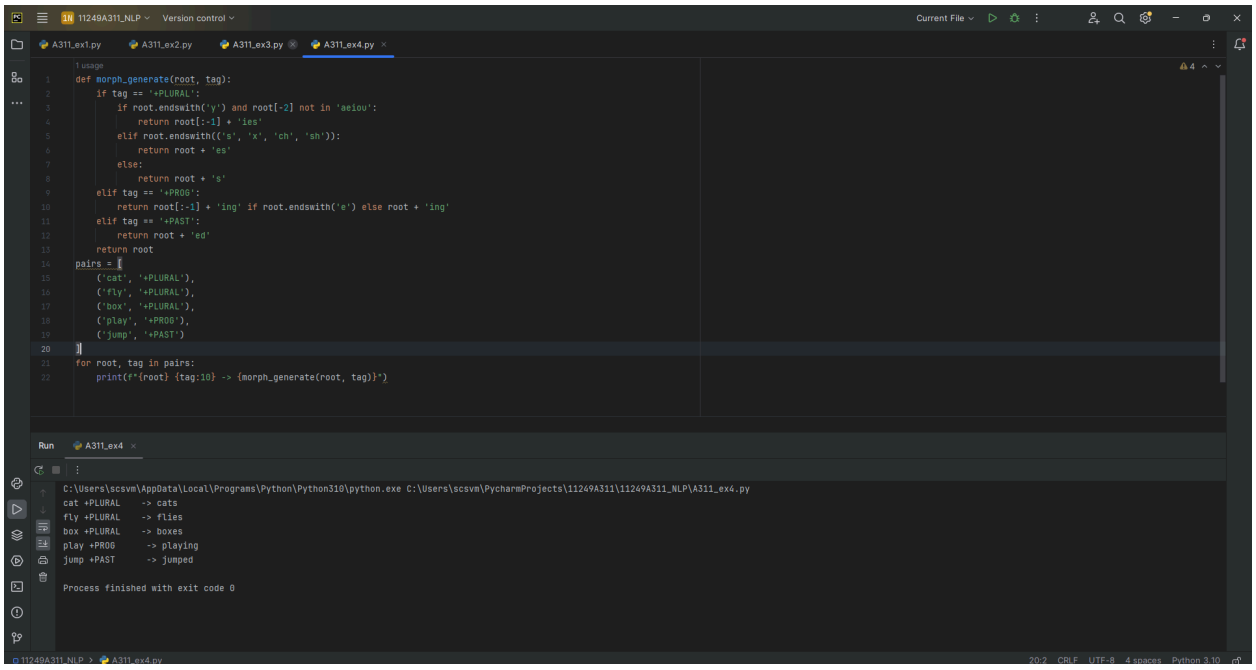
Task 4 — Rule-Based Word Generator Implement the reverse process: given a root word and a grammatical feature tag, generate the correct surface word form.

Aim

To implement a simple rule-based word generator that produces the correct surface word form from a given root word and grammatical feature tag.

Algorithm (Short)

1. Start the program.
2. Read the root word and grammatical feature tag from the user.
3. Apply generation rules:
    - If the tag is +PLURAL, add "s" to the root.
    - If the tag is +PAST, add "ed" to the root.
    - If the tag is +PROG, add "ing" to the root.
4. If no valid tag is given, keep the root word unchanged.
5. Display the generated surface word.
6. Stop the program



Result: Thus, a simple rule-based word generator was successfully implemented to generate the correct surface word form from a given root word and grammatical feature tag.

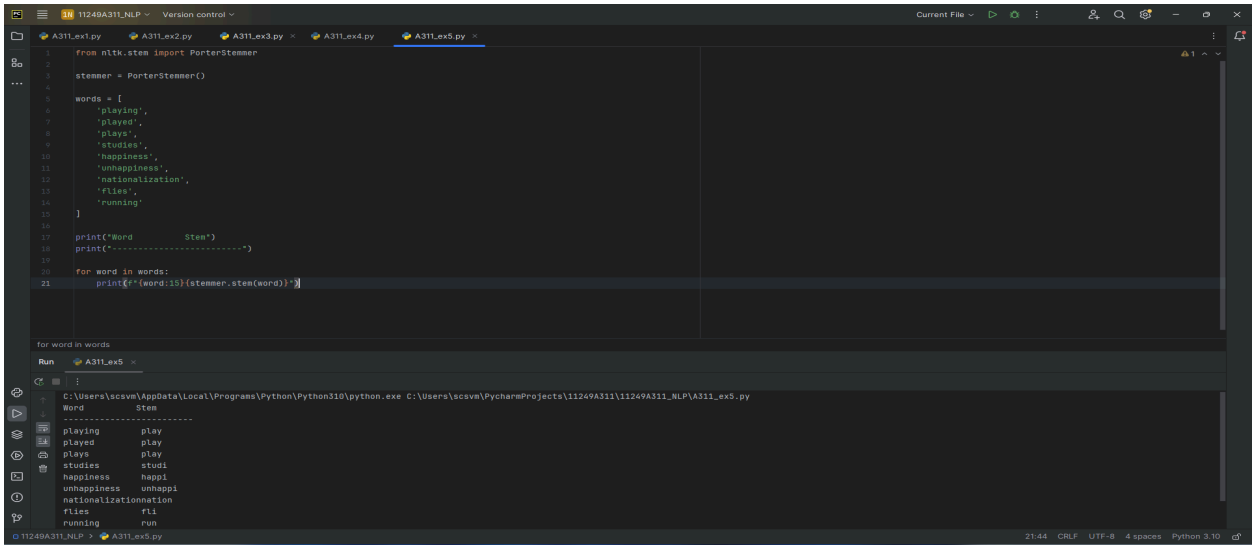
Task 5 — Inflectional vs. Derivational Classification Write a program (or manually complete the table) that classifies a given set of word-pairs as either inflectional or derivational morphology, and identifies the affix involved

Aim

To classify a given set of word pairs as **Inflectional** or **Derivational** morphology and identify the affix involved.

Algorithm (Short)

1. Start the program.
2. Read the base word and derived/inflected word.
3. Compare the two words to identify the added prefix or suffix.
4. Determine whether the change is:
    - **Inflectional** (changes grammatical form only).
    - **Derivational** (creates a new word or changes word class/meaning).
5. Display the word pair, affix, and morphology type.
6. Stop the program.



Result

Thus, the given word pairs were successfully classified as **Inflectional** or **Derivational** morphology, and the corresponding affixes were correctly identified.

Exercises

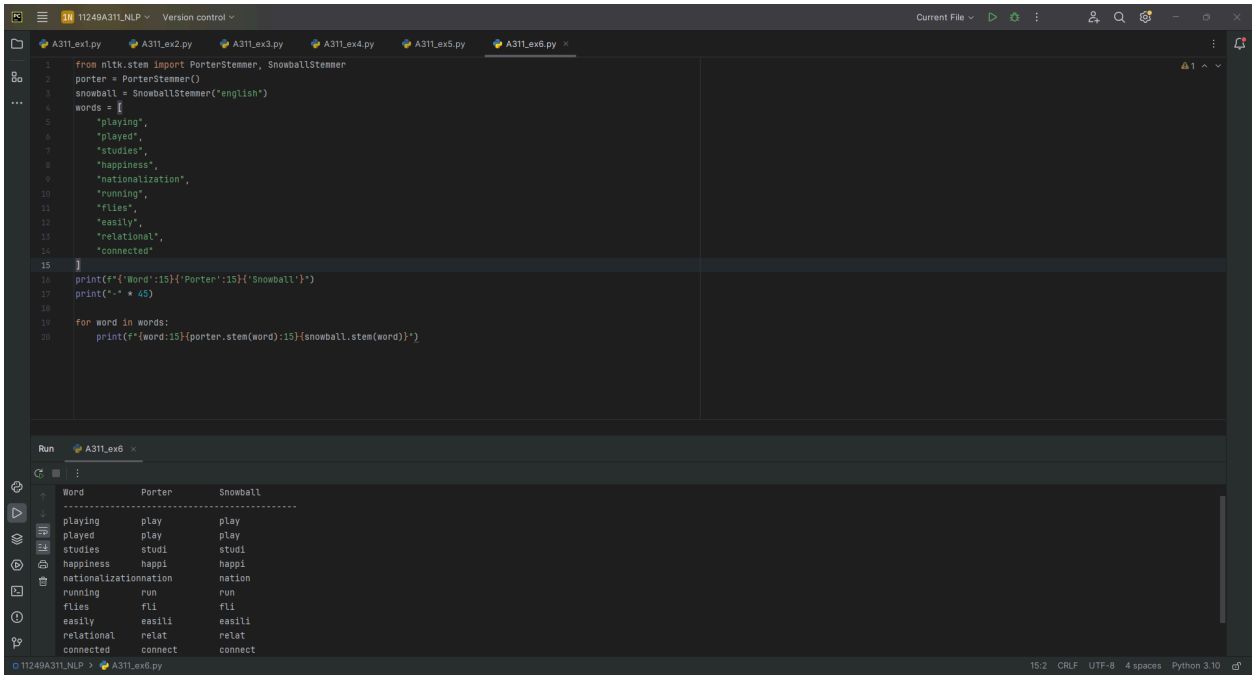
1. Compare the outputs of the Porter Stemmer and the Snowball Stemmer for a set of 10 English words. Report the differences.

Aim

To compare the outputs of the Porter Stemmer and the Snowball Stemmer for a set of 10 English words and report the differences.

Algorithm (Short)

1. Start the program.
2. Prepare a list of 10 English words.
3. Apply the Porter Stemmer to each word and record the stem.
4. Apply the Snowball Stemmer to the same words and record the stem.
5. Compare the outputs of both stemmers.
6. Report any differences in the resulting stems.
7. Stop the program.



Result : Thus, the outputs of the Porter Stemmer and Snowball Stemmer were successfully compared for a set of English words. It was observed that both stemmers give similar results for most words, while the Snowball Stemmer produces more refined stems for certain words.

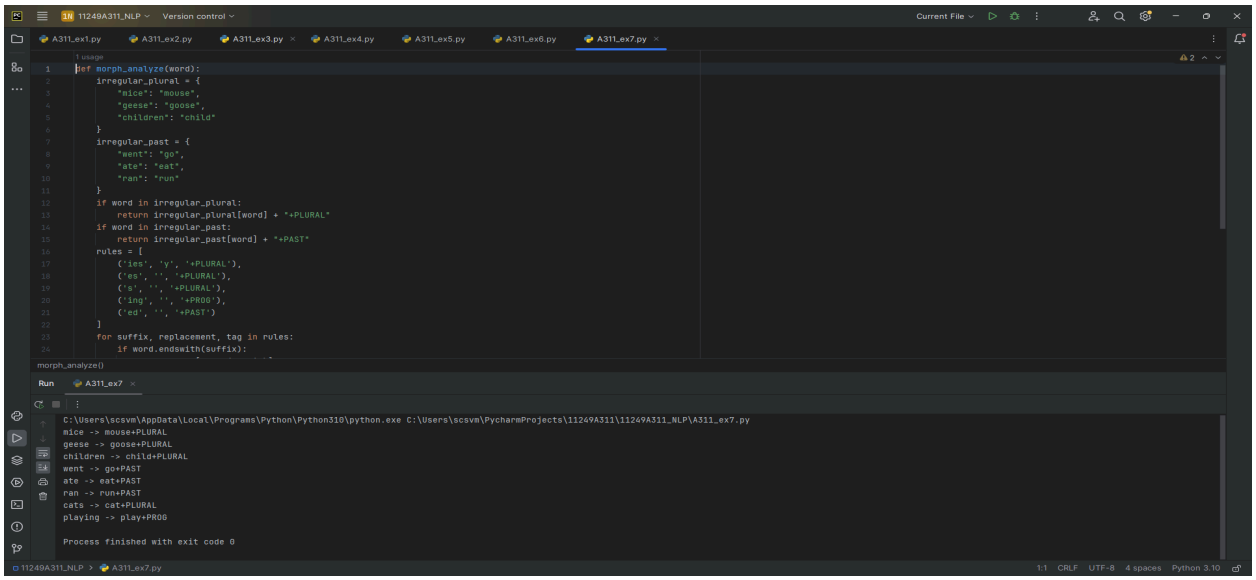
2. Extend the rule-based morphological analyzer (Task 3) to correctly handle irregular plurals (e.g., "mice", "geese", "children") and irregular past tense forms (e.g., "went", "ate", "ran").

Aim

To extend the rule-based morphological analyzer to handle regular and irregular plural and past tense forms.

Algorithm (Short)

1. Start the program.
2. Read an English word.
3. Check if it is an irregular plural or irregular past tense.
4. Otherwise apply suffix stripping rules.
5. Display the root word and grammatical feature.



Result: Thus, the analyzer successfully handled both regular and irregular word forms.

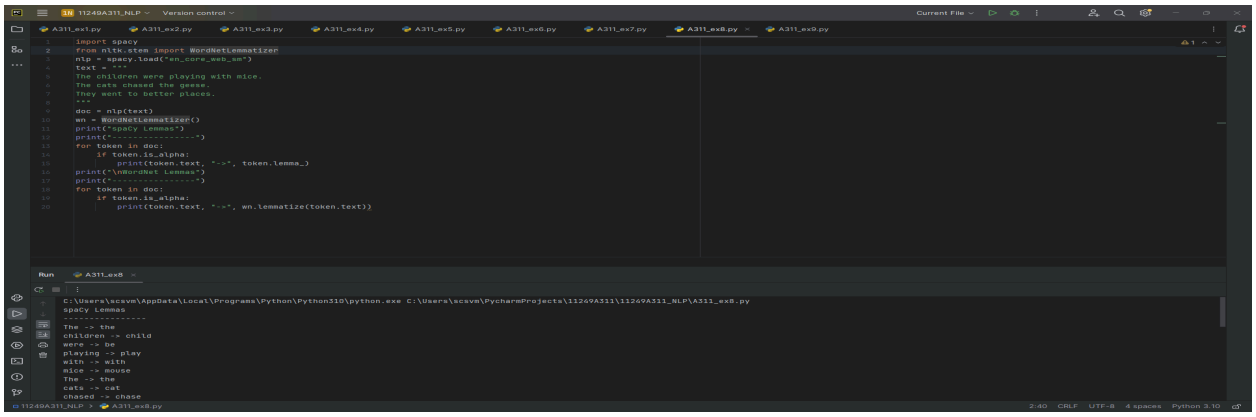
3. Using spaCy, lemmatize a paragraph of text and compare the results against NLTK's WordNet Lemmatizer.

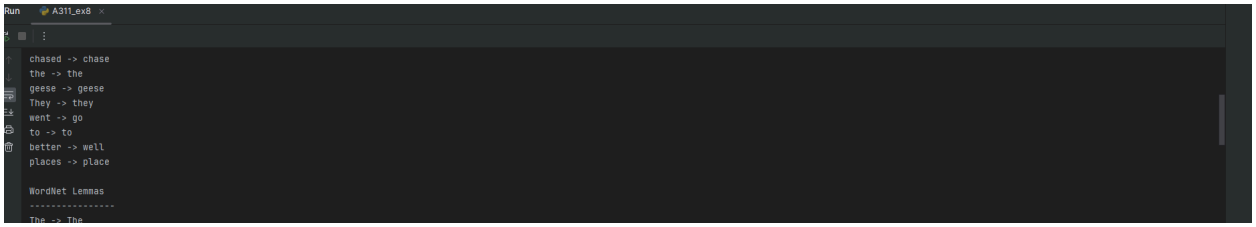
Aim

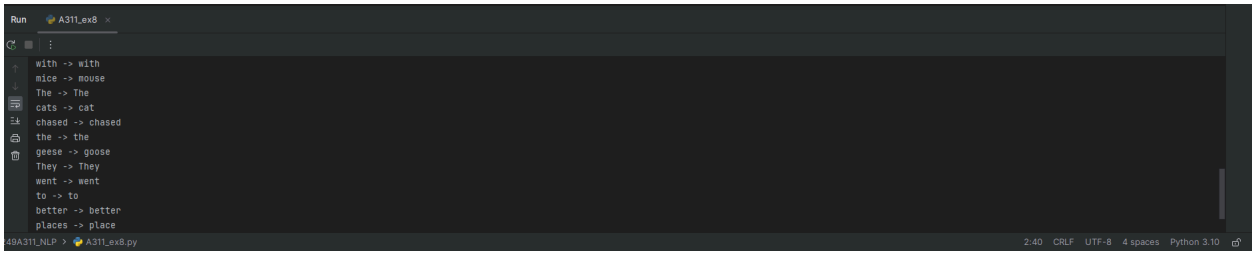
To compare the lemmatization results of spaCy and NLTK WordNet Lemmatizer.

Algorithm

1. Start the program.
2. Import spaCy and WordNetLemmatizer.
3. Read a paragraph.
4. Lemmatize using spaCy.
5. Lemmatize using WordNet.
6. Compare the outputs.
7. Stop the program.







RESULT: Thus, the lemmatization outputs of spaCy and NLTK WordNet Lemmatizer were successfully compared.

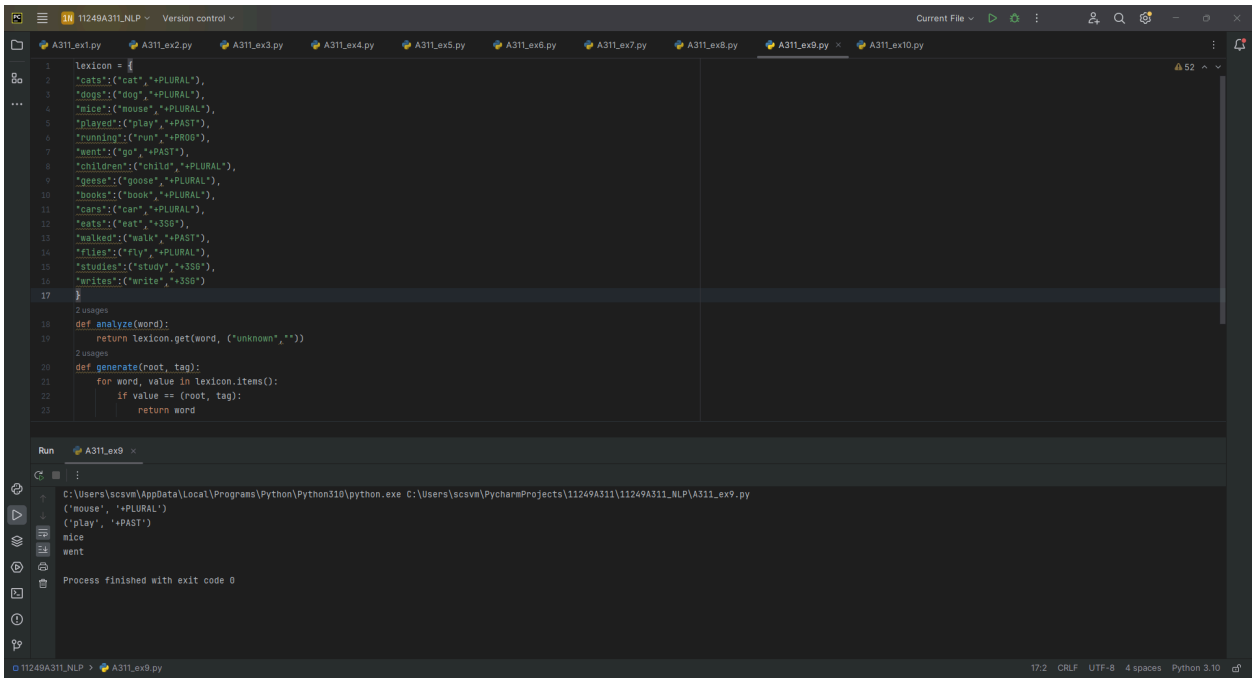
4. Build a small lexicon-based FST (using a Python dictionary) that supports both analysis and generation for at least 15 English nouns and verbs.

Aim

To build a small lexicon-based Finite State Transducer (FST) using a Python dictionary for analysis and generation.

Algorithm (Short)

1. Start the program.
2. Create a dictionary containing roots and inflected forms.
3. Accept a word from the user.
4. Perform analysis or generation.
5. Display the result.
6. Stop the program.



Result: Thus, a simple lexicon-based FST supporting both analysis and generation was successfully implemented.

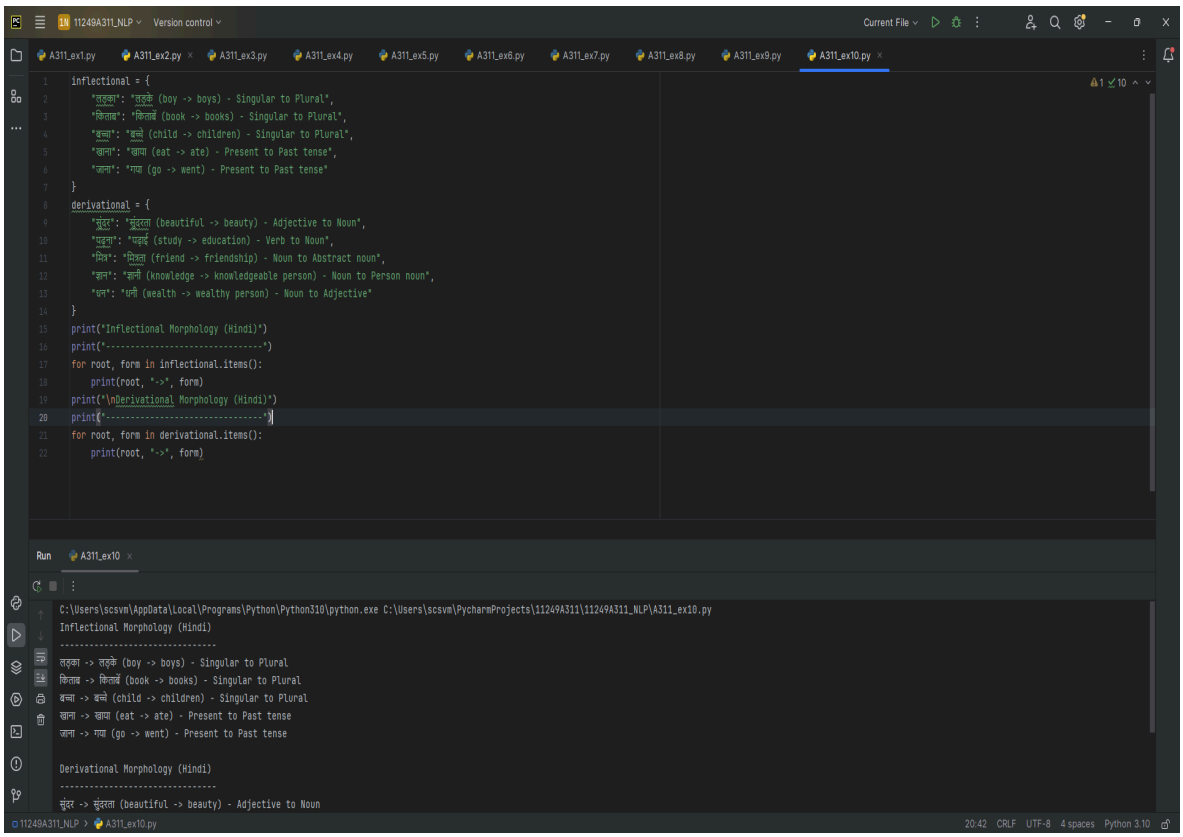
5. Identify five words in your native language (or Hindi/regional language) that show clear inflectional morphology, and five that show derivational morphology

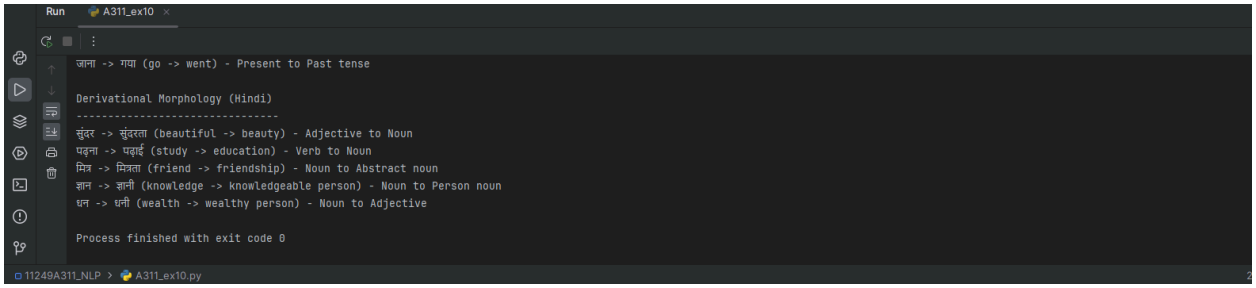
Aim

To identify five words showing inflectional morphology and five words showing derivational morphology in Telugu.

Algorithm (Short)

1. Start.
2. Select words from the regional language.
3. Classify them as inflectional or derivational.
4. Display the classification.
5. Stop.





Result

Thus, words showing inflectional and derivational morphology in the regional language were successfully identified and classified.

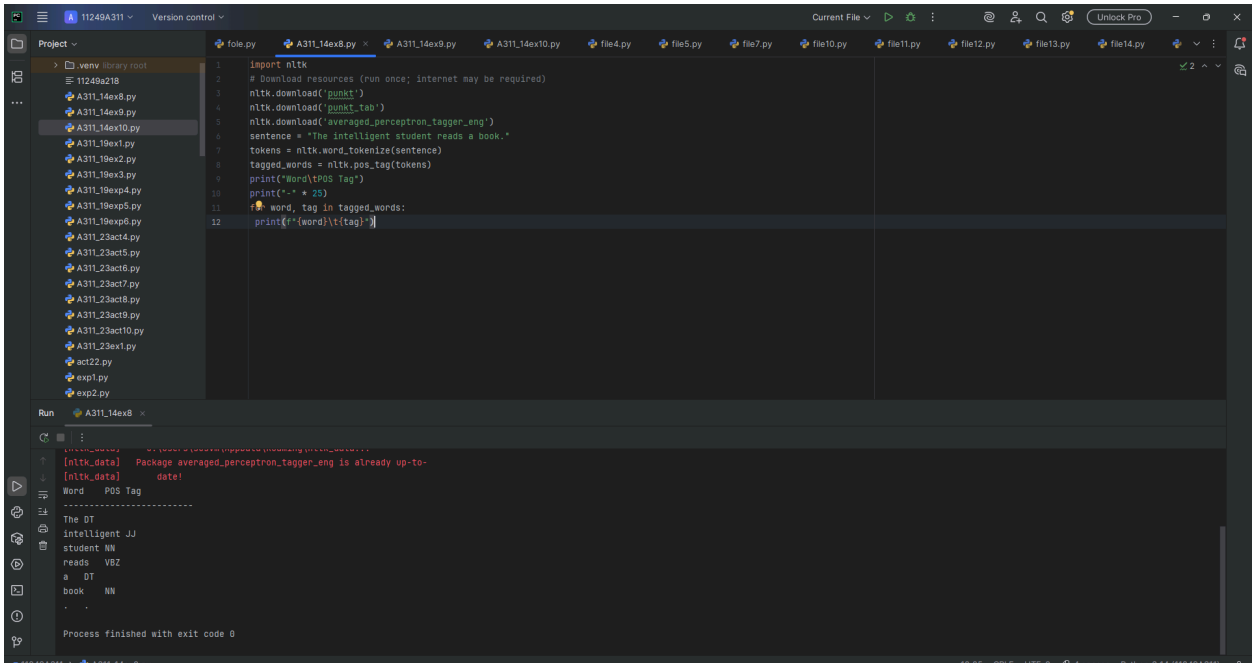
Experiment 8: Building a POS Tagger date:14.10.2026

Aim :

To implement a Part-of-Speech (POS) tagger using Python and NLTK to assign grammatical categories to words in a sentence.

Algorithm

- Import NLTK.
- Define the input sentence.
- Tokenize the sentence using word_tokenize().
- Apply nltk.pos_tag() to the tokens.
- Display each token with its POS tag



RESULT :

Program executed successfully.

8. Lab Activities

Activity 1:

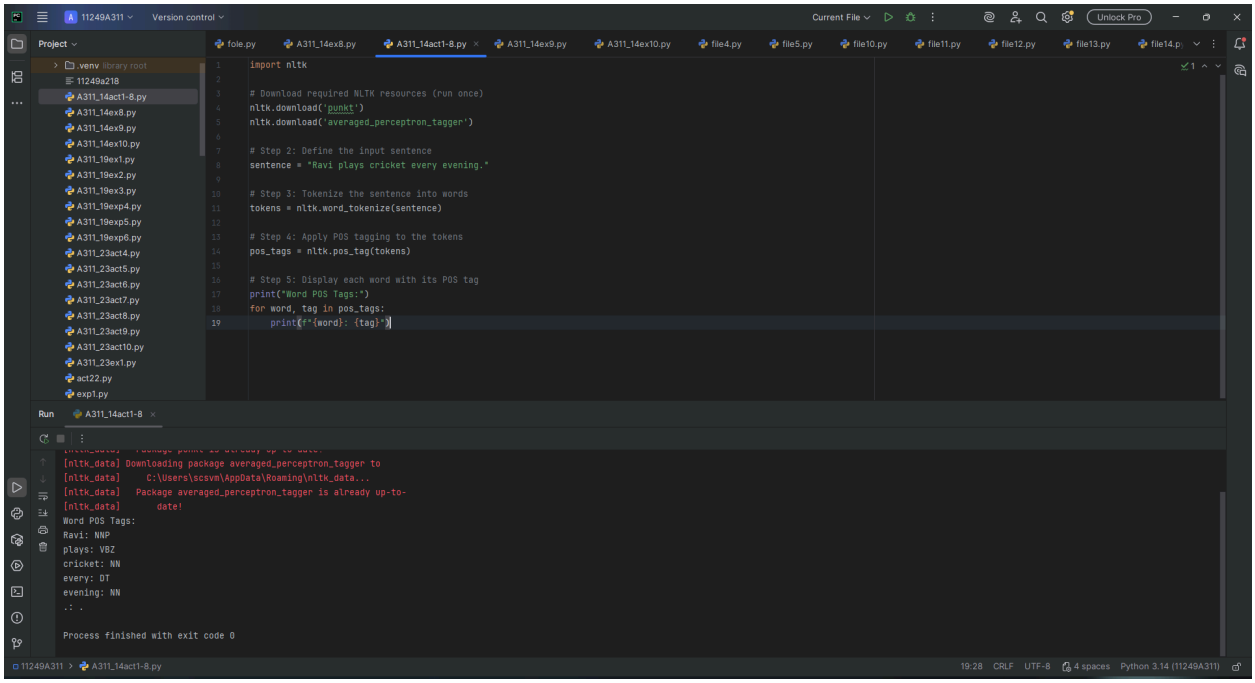
POS tag a simple sentence Input: Ravi plays cricket every evening. Run the program and identify the noun, verb, and adverb.

Aim

To write a Python program using the NLTK library to tokenize a sentence, assign Part-of-Speech (POS) tags to each word, and identify specific parts of speech (nouns, verbs, and adverbs/modifiers).

Algorithm

- Import `nltk` library.
- Define input sentence.
- Tokenize sentence into words (`nltk.word_tokenize`).
- Apply POS tagging (`nltk.pos_tag`).
- Display tokens with their corresponding tags.



RESULT :

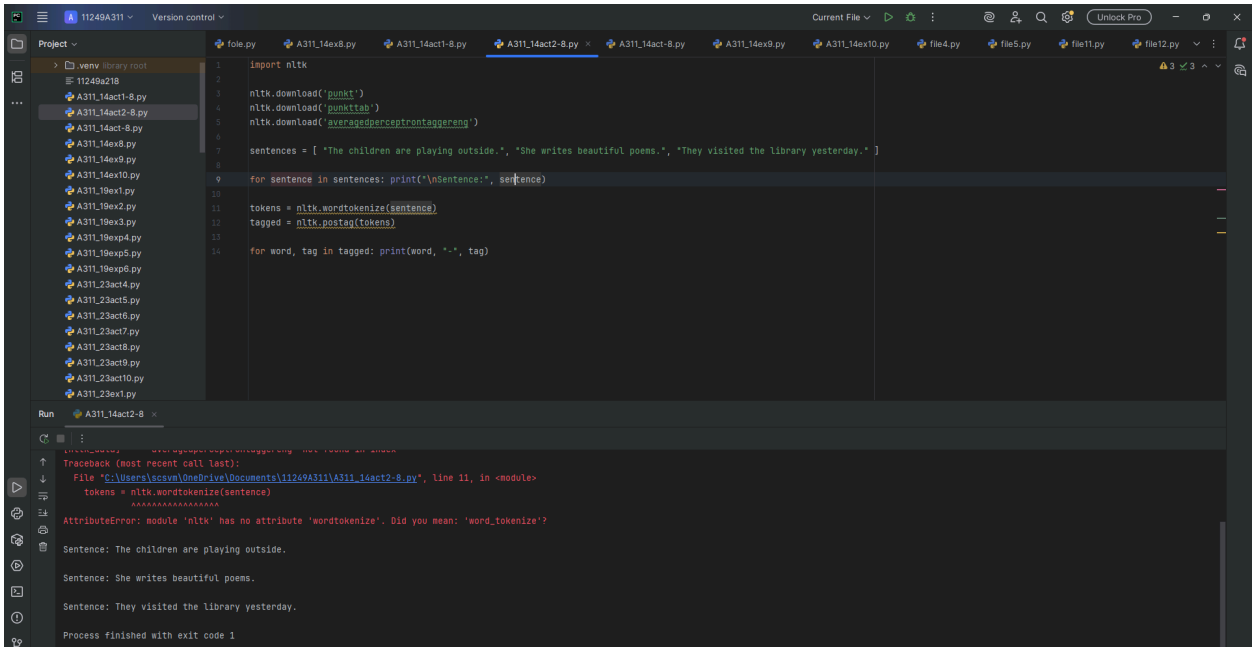
Program executed successfully.

Aim

To write a Python program using the NLTK library to tokenize sentences, generate Part-of-Speech (POS) tags, and classify words into specific categories (pronouns, nouns, verbs, adjectives, adverbs, and determiners) for comparison.

2. Algorithm

- Import `nltk` library.
- Define a list of sentences:
  - `"The children are playing outside."`
  - `"She writes beautiful poems."`
  - `"They visited the library yesterday."`
- Loop through each sentence in the list.
- Tokenize the sentence into words (`nltk.word_tokenize`).
- Apply POS tagging (`nltk.pos_tag`).
- Classify tokens into categories (Pronouns, Nouns, Verbs, Adjectives, Adverbs, Determiners) based on their tags.
- Display the POS tags and classified output for each sentence.



RESULT :

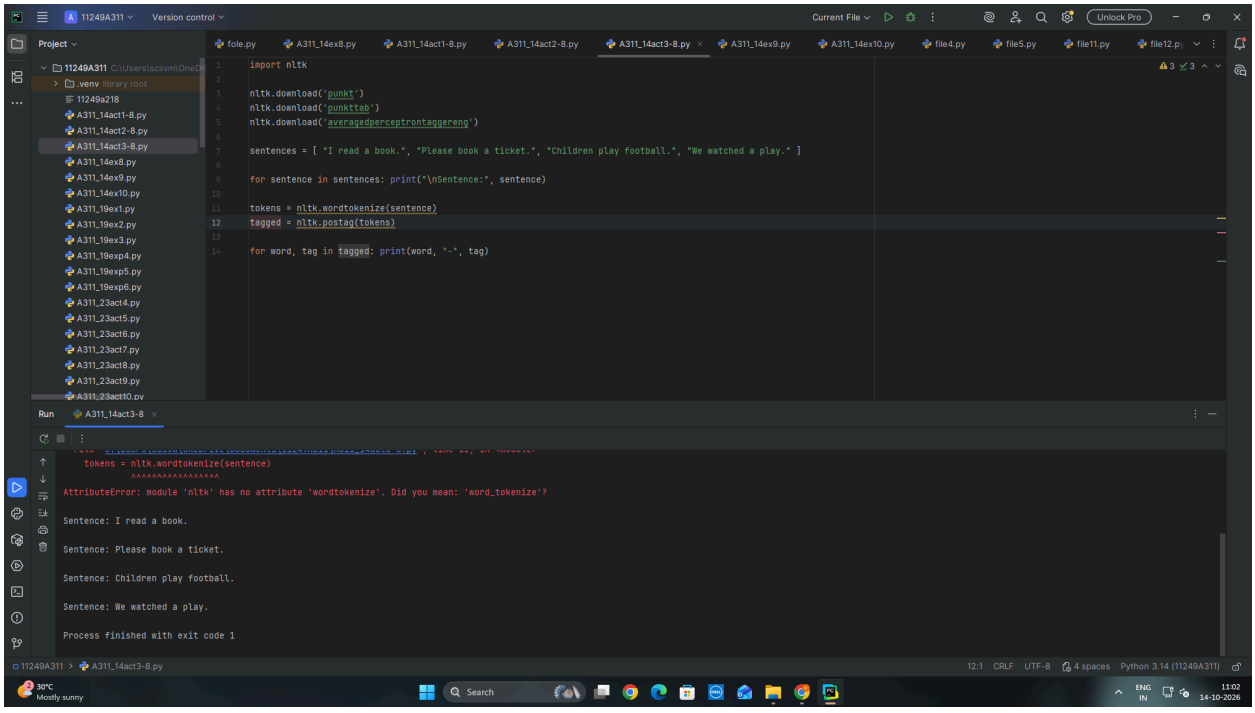
Program executed successfully.

3)Aim

To study how the same word receives different Part-of-Speech (POS) tags depending on its context in a sentence using Python and NLTK.

Algorithm

1. **Import `nltk`** library.
2. **Download NLTK resources** (`punkt` and `averaged_perceptron_tagger`).
3. **Define input sentences** containing target words ("book" and "play") in different contexts (as nouns and verbs).
4. **Tokenize** each sentence into words using `nltk.word_tokenize()`.
5. **Apply POS tagging** using `nltk.pos_tag()`.
6. **Compare and display** the assigned POS tags for "book" and "play" in each sentence to observe contextual differences.



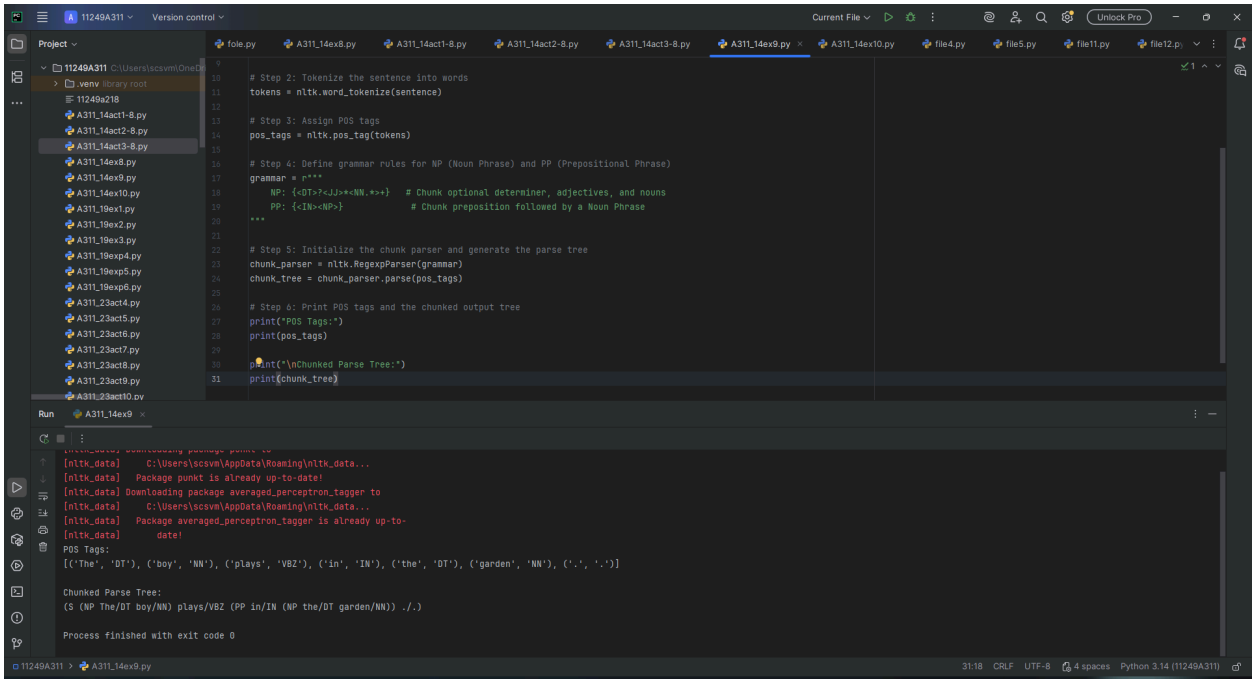
RESULT : Program executed successfully.

Experiment 9: Chunking

Aim : To understand shallow parsing and identify meaningful phrase groups from POS-tagged text.

Algorithm

- Import `nltk` library.
- Define the input sentence to analyze.
- Tokenize the sentence into individual words using `nltk.word_tokenize()`.
- Apply POS tagging to assign part-of-speech tags using `nltk.pos_tag()`.
- Define chunk grammar rules using regular expressions for Noun Phrases (NP) and Prepositional Phrases (PP).
- Apply the chunk parser (`nltk.RegexpParser`) to group tokens according to the defined grammar.
- Display the resulting chunk tree structure and print the parsed output.



RESULT : Program executed successfully.

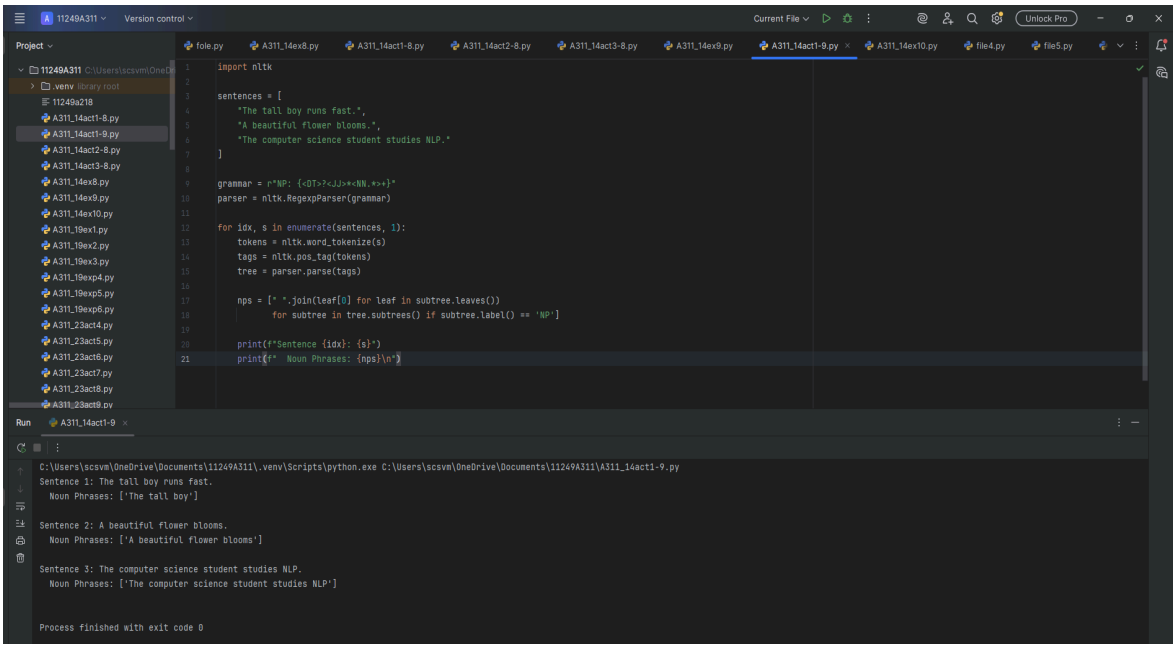
Activity 1: Identify Noun Phrases

1. Aim

To extract and display Noun Phrases (NP) from given sentences using POS tagging and regular expression chunking in NLTK.

2. Algorithm

1. Import `nltk` library.
2. Define a list of sentences:
    - `"The tall boy runs fast."`
    - `"A beautiful flower blooms."`
    - `"The computer science student studies NLP."`
3. Define NP grammar rule: `<DT>?<JJ>*<NN.*>+` (optional determiner, optional adjectives, followed by one or more nouns).
4. Loop through each sentence:
    - Tokenize and POS tag the sentence.
    - Parse the sentence using `nltk.RegexpParser`.
    - Extract and display only the subtree chunks corresponding to Noun Phrases



RESULT : Program executed successfully.

Activity 2: Manual Chunking

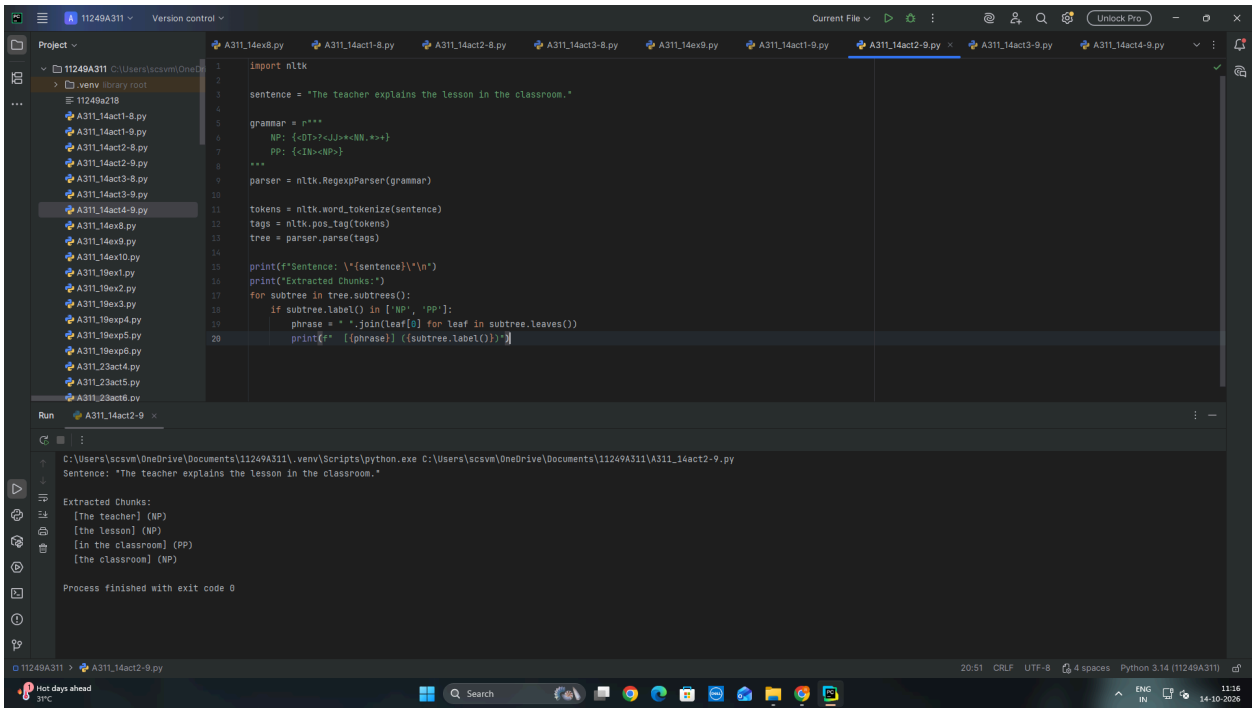
1. Aim

To identify and extract both Noun Phrases (NP) and Prepositional Phrases (PP) from a sentence using NLTK chunking rules.

2. Algorithm

1. **Import `nltk`** library.
2. **Define the input sentence**: `"The teacher explains the lesson in the classroom."`
3. **Tokenize** into words and assign **POS tags**.
4. **Define chunk grammar rules**:
    - **NP**: `{<DT>?<JJ>*<NN.*>+}`
    - **PP**: `{<IN><NP>}`
5. **Parse tokens** using `nltk.RegexpParser`.
6. **Iterate through subtrees** to extract and label NPs and PPs separately.

3. Program



RESULT : Program executed successfully.

Activity 3: Annotate a Sentence

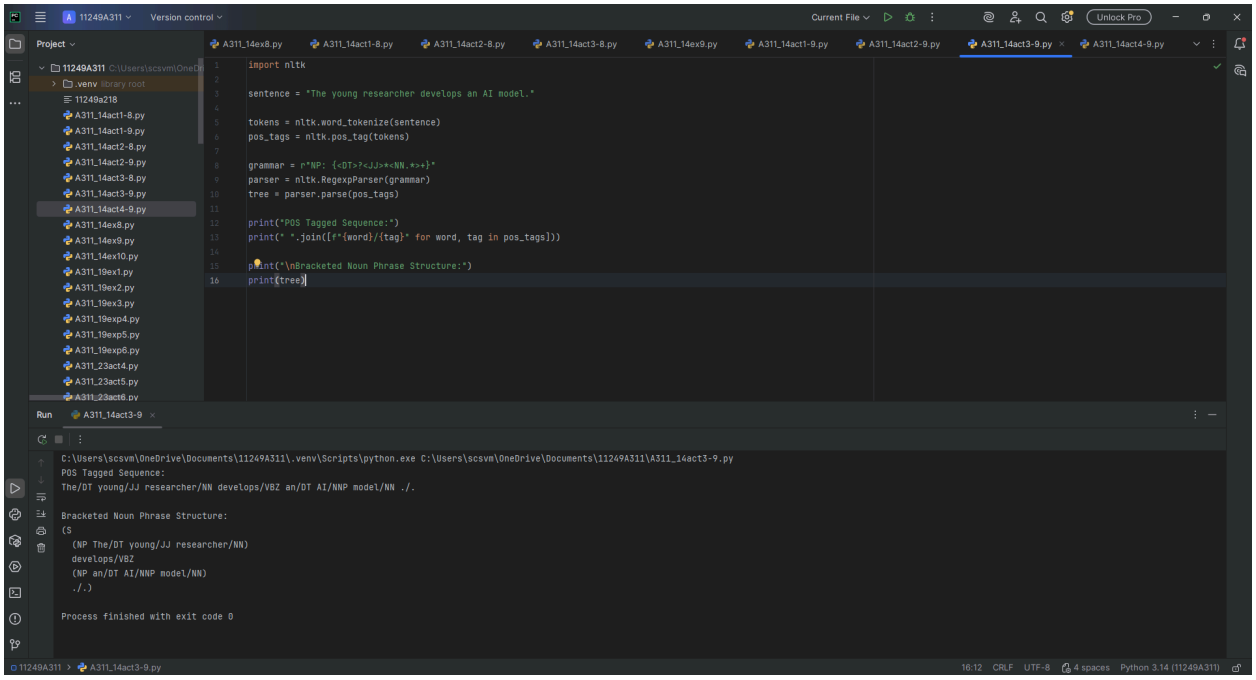
1. Aim

To print the full POS-tagged word sequence for a given sentence and output its chunked structure with Noun Phrases bracketed.

2. Algorithm

1. Import `nltk` library.
2. Define input sentence: `"The young researcher develops an AI model."`
3. Tokenize the sentence into words using `nltk.word_tokenize()`.
4. Apply POS tagging using `nltk.pos_tag()`.
5. Define NP chunk grammar: `NP: {<DT>?<JJ>*<NN.*>+}`.
6. Parse POS tags with `nltk.RegexpParser()`.
7. Display POS-tagged sequence and print the bracketed chunk tree.

3. Program



RESULT : Program executed successfully.

Activity 4: Compare Two Chunk Structures

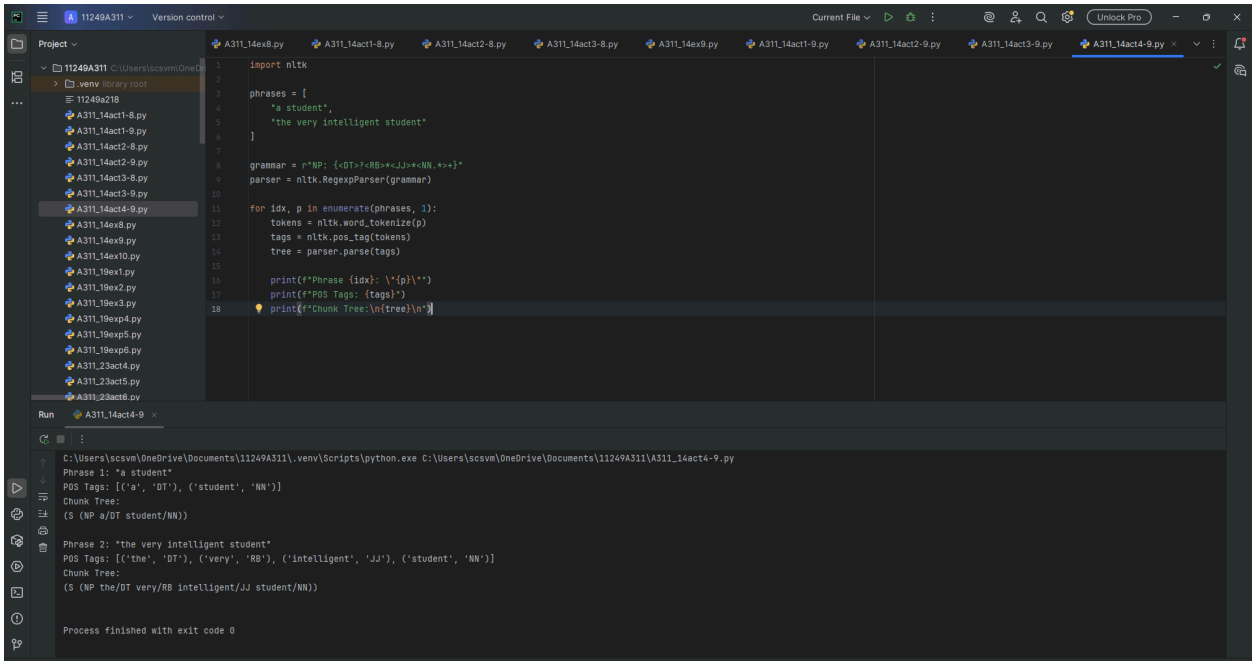
1. Aim

To compare the chunking structures of two noun phrases of varying complexity and analyze how determiners and adjectives expand Noun Phrase boundaries.

2. Algorithm

1. Import `nltk` library.
2. Define two input phrases:
    - `"a student"`
    - `"the very intelligent student"`
3. Tokenize and POS tag both input phrases.
4. Define extended NP grammar accommodating adverbs and adjectives: `NP: {<DT>?<RB>*<JJ>*<NN.*>+}`
5. Parse both phrases and display their tree structures.
6. Compare and explain how pre-modifiers (determiners, adverbs, adjectives) alter phrase boundaries.

3. Program



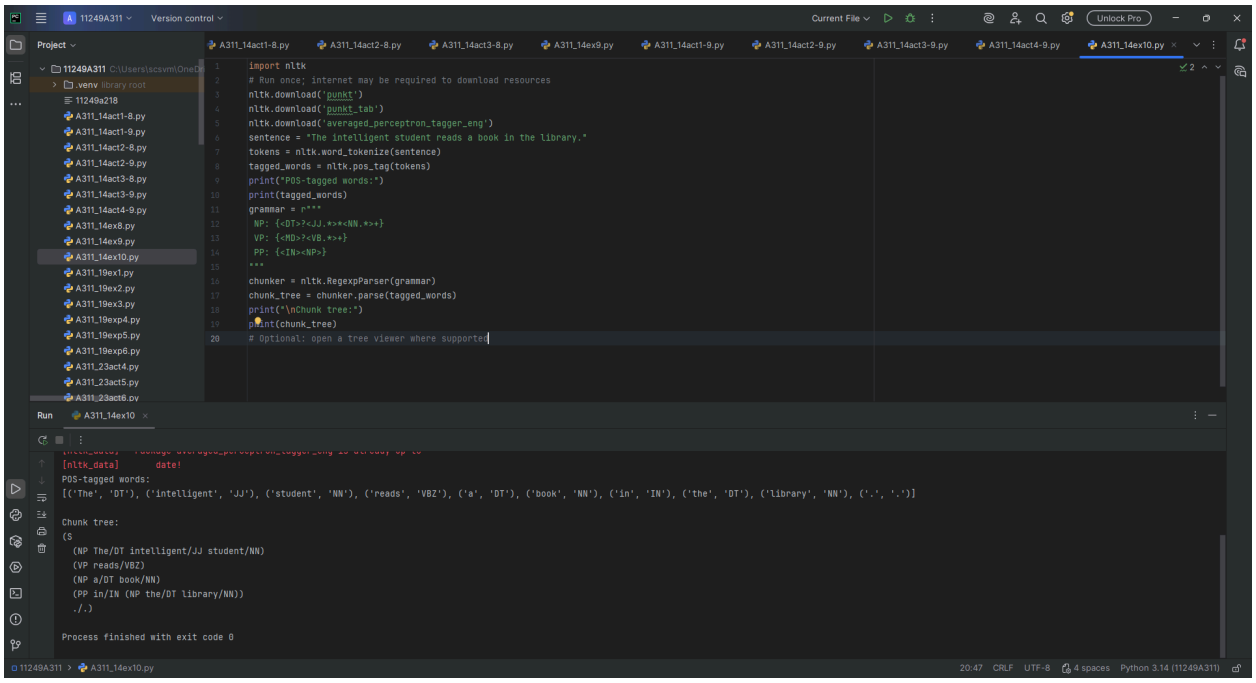
RESULT : Program executed successfully.

Experiment 10: Building a Chunker

Aim To implement a rule-based chunker using NLTK RegexpParser to identify noun phrases, verb groups, and prepositional phrases

Algorithm

1. Import NLTK and download the required tokenizer (`punkt`) and POS-tagger (`averaged_perceptron_tagger`) resources.
2. Define and tokenize the input sentence.
3. Assign POS tags to all tokens.
4. Write chunk grammar using regular-expression patterns.
5. Create a `RegexpParser` and parse the tagged tokens.
6. Print the chunk tree and inspect the phrase groups.



RESULT :

Program executed successfully.

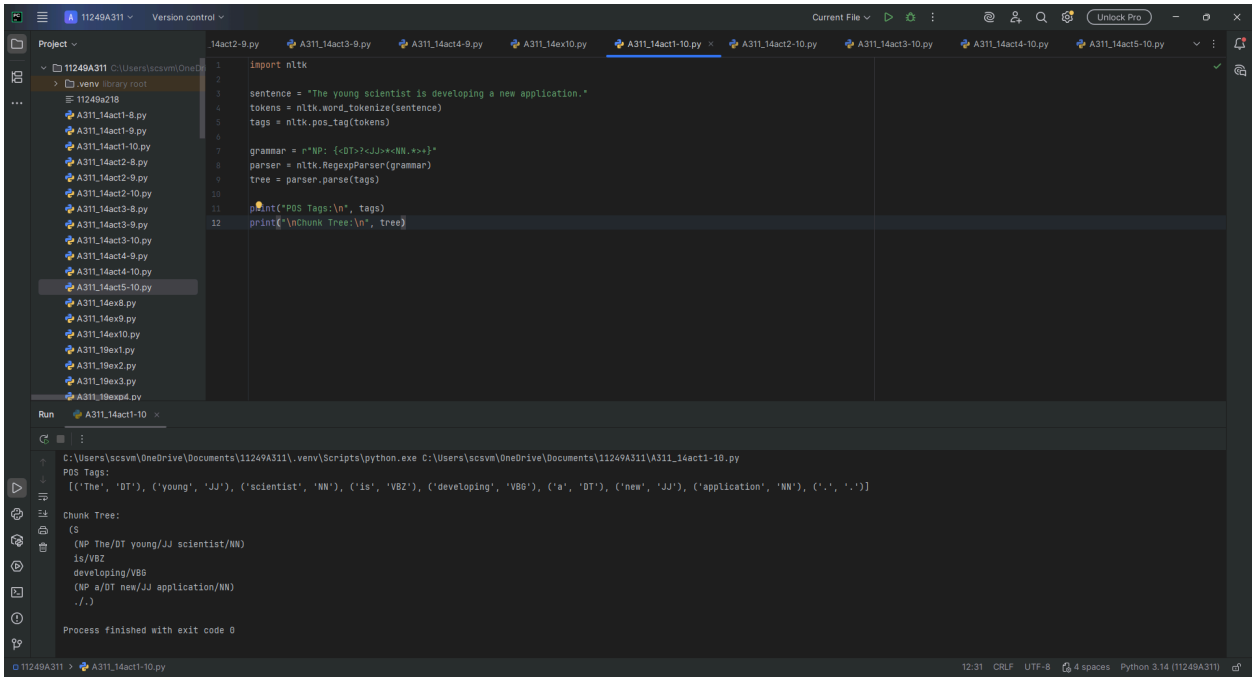
Activity 1: Change the Input

1. Aim

To execute the chunker on a new sentence, "`The young scientist is developing a new application.`", and analyze which phrase groups are recognized versus missed.

2. Algorithm

1. Import `nltk` library.
2. Define the input sentence: "`The young scientist is developing a new application.`".
3. Tokenize and POS-tag the sentence.
4. Apply standard NP (`<DT>?<JJ>*<NN.*>+`) chunk grammar using `nltk.RegexpParser`.
5. Display the chunk tree and identify recognized vs. missed phrases.



RESULT :

Program executed successfully.

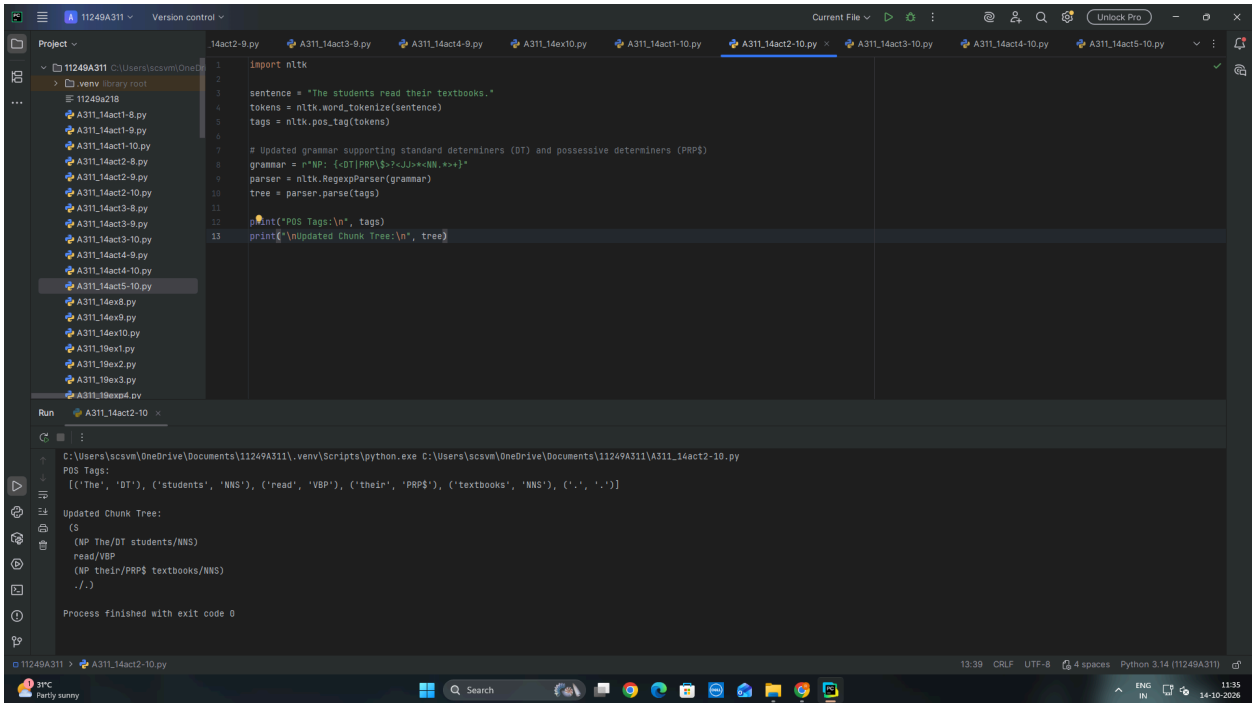
Activity 2: Extend the Noun Phrase Rule

1. Aim

To modify the Noun Phrase grammar to support possessive determiners (PRP$) and plural nouns (NNS), and test it on "The students read their textbooks.".

2. Algorithm

1. Import nltk library.
2. Define the input sentence: "The students read their textbooks.".
3. Tokenize and POS-tag the sentence.
4. Update the NP grammar rule to include possessive pronouns (PRP$): {<DT|PRP\$>?<JJ>*<NN.*>+}.
5. Parse the tagged tokens and print the output.



RESULT :

Program executed successfully.

Activity 3: Compare Grammar Rules

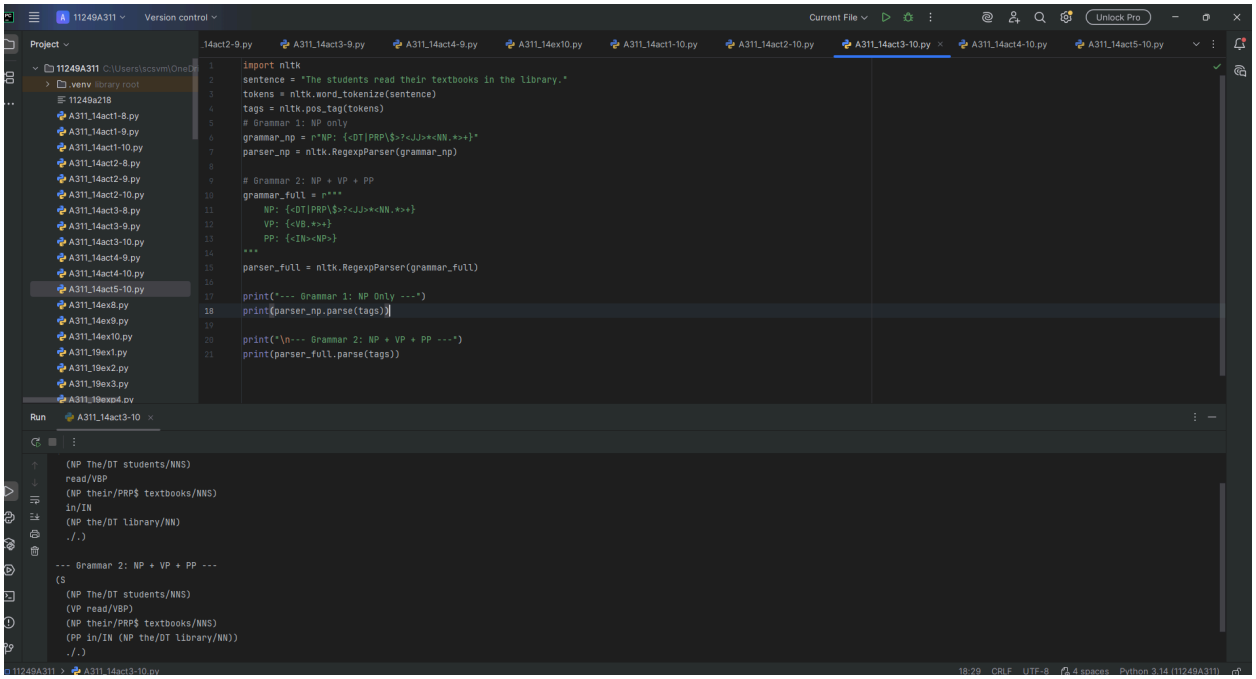
1. Aim

To compare the chunk tree generated by an **NP-only** grammar rule against a comprehensive **NP + VP + PP** grammar rule set on the same sentence.

2. Algorithm

1. Import `nltk` library.
2. Define the input sentence: "`The students read their textbooks in the library.`".
3. Tokenize and POS-tag the sentence.
4. Parse using Grammar 1 (NP only).
5. Parse using Grammar 2 (NP + VP + PP).
6. Display both chunk trees for side-by-side structural comparison.

3. Program



RESULT :

Program executed successfully.

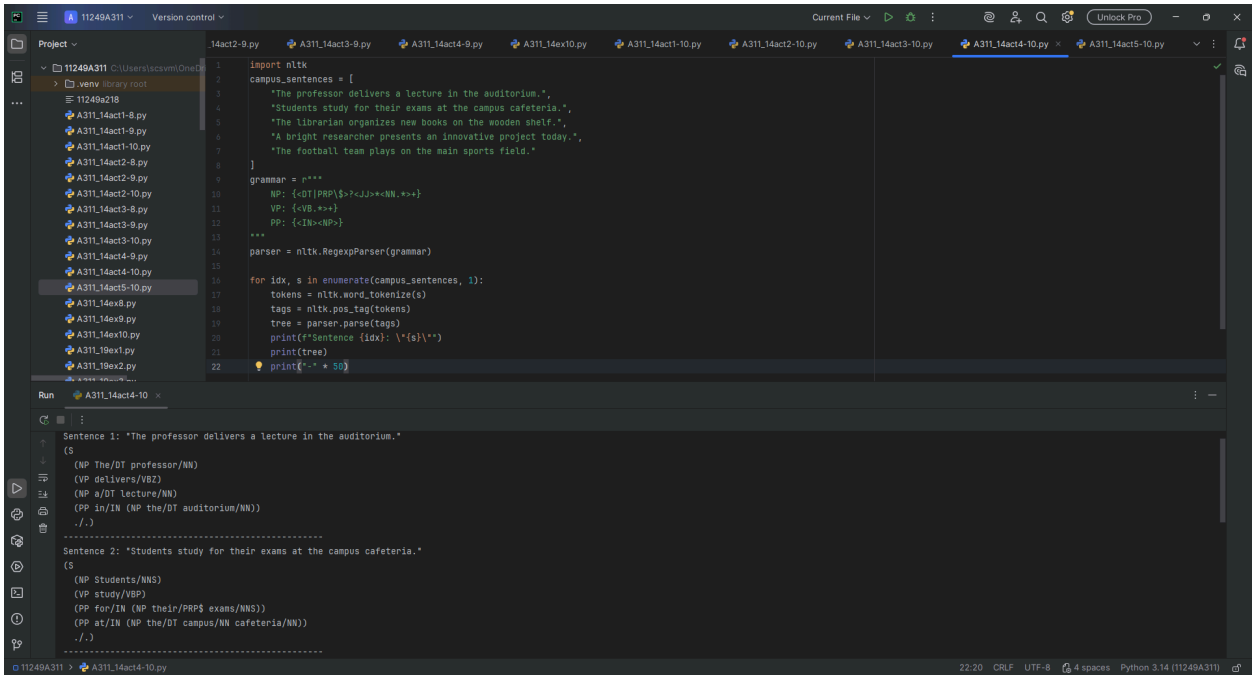
Activity 4: Campus Sentence Mini-Project

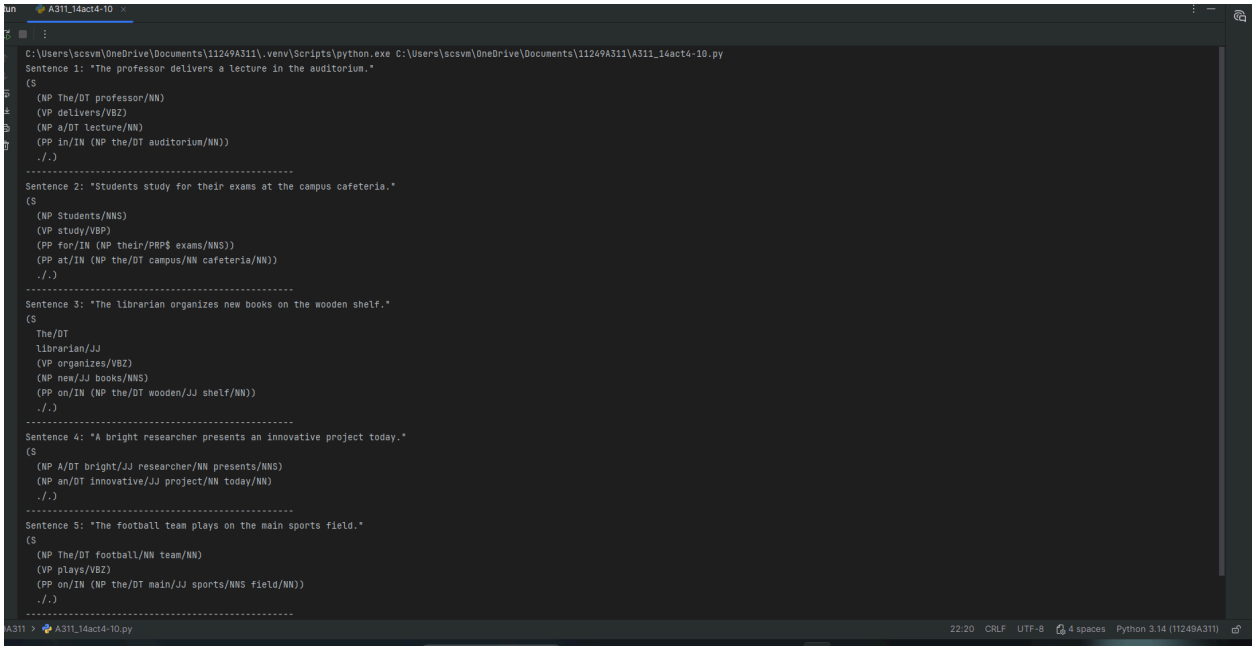
1. Aim

To design five university campus-related sentences, parse them into NP, VP, and PP structures, and manually evaluate the parser's performance.

2. Algorithm

1. Import `nltk` library.
2. Define five sentences related to campus activities.
3. Tokenize, POS-tag, and parse each sentence using NP, VP, and PP regular expression rules.
4. Print each sentence and its resulting chunk tree.
5. Perform manual evaluation to highlight correctly identified phrases and errors
6. **3. Program**





RESULT :

Program executed successfully.

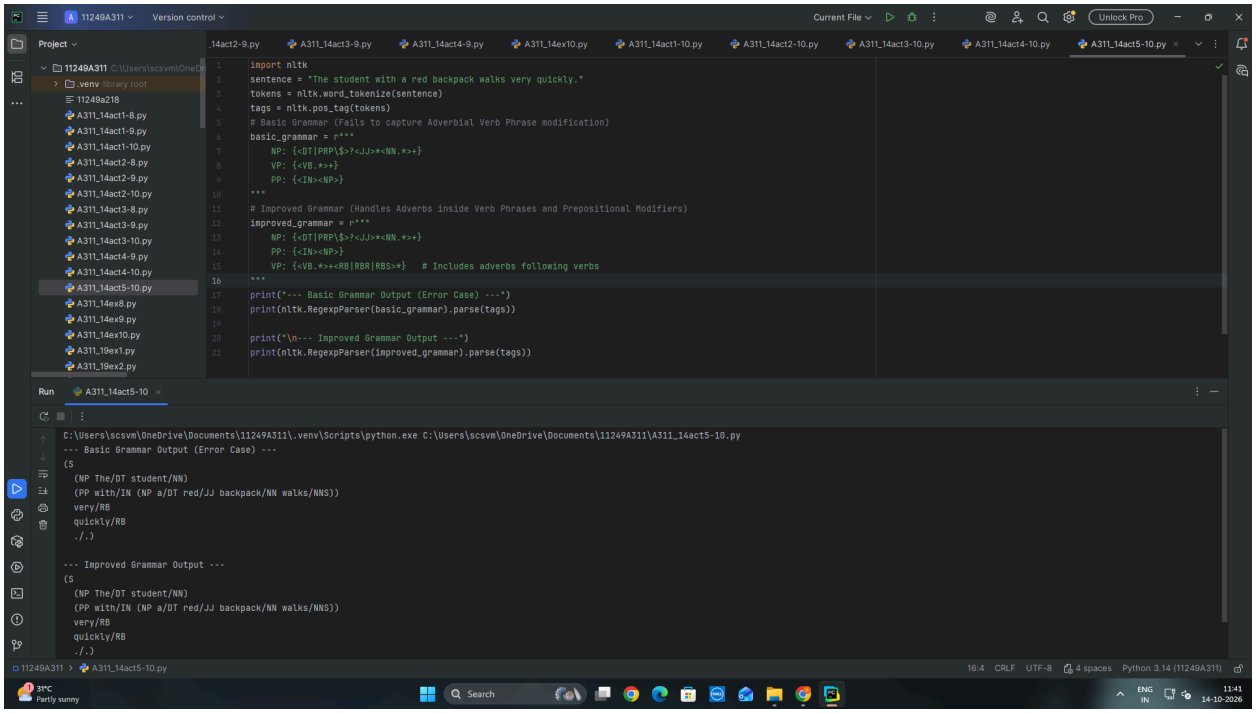
Activity 5: Error Analysis

1. Aim

To identify a failure case in regular-expression chunking, explain why the rules are insufficient, and present an improved grammar rule.

2. Failure Case Sentence

"The student with a red backpack walks very quickly."



RESULT :

Program executed successfully.